



**Electronics and Communications Department
Faculty of Engineering
Capital University**

Design and Implementation of UC1e 3.0 Adapter Layer

*Graduation project is submitted to Electronics and Communications
Department in Partial fulfilment of the requirements for the Degree of
Bachelor of Electronics and Communications Engineering – June 2026*

Prepared by

Fatma Ashraf Fawzy Ahmed

Shahd Mohamed Qassem

Ali Noureldin Abdelaziz

Ashraf Sherif Mohamed

Ali Mohamed Ali Sakr

Under Supervision of

Dr. Shimaa I. Sayed

2026

Acknowledgements

In the name of Allah, the most gracious, the most merciful. All praise and thanks are due to Allah, the lord of the worlds, who has guided and blessed us throughout our academic journey. First and foremost, we would like to thank Allah for his countless blessings and for granting us the opportunity to undertake this project. without his guidance and support, we would not have been able to achieve this accomplishment.

We would also like to express our sincere gratitude to our supervisors Dr. Shimaa I. Sayed and Eng. Abdelrahman for their guidance, support, and encouragement throughout this project. Their expertise and feedback have been invaluable in shaping and refining our work. We would also like to extend our sincere thanks to our families for their unwavering encouragement, understanding, and patience during this project. Their constant support has been a source of motivation and inspiration. Furthermore, we are grateful to all individuals who have directly or indirectly contributed to this project, whether through their expertise, assistance, or encouragement. Your contributions have played a vital role in shaping the outcome of this work.

Thank you all for your support and encouragement. May Allah bless you all and guide us to the path of righteousness.

Abstract

The Universal Chiplet Interconnect Express (UCIe) is an open, high-bandwidth interconnect standard designed to enable heterogeneous, disaggregated System-in-Package (SiP) architectures. Within the UCIe protocol stack, the Die-to-Die (D2D) Adapter layer serves as the critical intermediary between the Protocol Layer and the Physical Layer.

This project focuses on the comprehensive design and hardware implementation of the UCIe D2D Adapter, realizing its core responsibilities to establish a robust, low-latency, and power-efficient data path. The implementation encompasses the design of the standard interfaces bridging the layers: the Flit-aware Die-to-Die Interface (FDI) connecting to the Protocol Layer, and the Raw Die-to-Die Interface (RDI) connecting to the Physical Layer. A primary focus of the design is ensuring reliable data transfer across these interfaces using main data path through the implementation of precise Cyclic Redundancy Check (CRC) and Retry mechanisms. This requires complex state management, including the rigorous tracking of explicit and implicit Flit Sequence Numbers to maintain perfect synchronization between the transmitter and receiver during both normal operation and error recovery windows.

Furthermore, the project involves the implementation of the Adapter Link State Machine (LSM), which coordinates Link bring-up, and crucial parameter negotiations with remote link partners. Using Sideband interface, an out-of-band communication channel that operates independently from the main data path. During the Link bring-up phase, the Adapter Link State Machine (LSM) utilizes the sideband to negotiate operational parameters and protocol capabilities with the remote Link partner through standardized messages. By isolating control, configuration, and state-transition messages from the mainband, the sideband ensures robust operation and system control even before the high-speed data lanes are fully active.

Ultimately, this project delivers a fully functional D2D Adapter layer that successfully abstracts the physical complexities of chiplet-to-chiplet communication from the upper protocol layers, enabling seamless and reliable data transport for advanced chiplet ecosystems.

TABLE OF CONTENTS

Chapter 1: Introduction	17
1.1 Background and Motivation	17
1.2 Problem Overview	17
1.3 What is UCle.....	17
1.4 UCle Architecture Overview	18
1.5 UCle D2D Adapter Layer	19
1.6 Objectives of this Project.....	20
Chapter 2: UCle 3.0 Specification Deep Dive.....	21
2.1 Overview of Protocol Layer.....	21
2.2 Overview of Physical Layer.....	22
2.3 Overview of Raw Die-to-Die Interface (RDI)	23
2.4 Overview of Flit-aware Die-to-Die Interface (FDI)	24
2.5 Overview of Adapter Layer	26
2.5.1 <i>Link Initialization (Stage 3 — Adapter Initialization)</i>	26
2.5.2 <i>Operation Formats (Flit Formats)</i>	27
2.5.3 <i>State Machine Hierarchy</i>	29
2.5.4 <i>CRC Computation</i>	30
2.5.5 <i>Retry Rules</i>	30
2.6 System Total Flow	31
Chapter 3: Sub-block: MainBand using Retry Mechanism	33
3.1 Overview of Mainband block	33
3.2 Format 3	34
3.2.1 <i>Flit Header of Format 3</i>	34
3.2.2 <i>Flit Structure Description</i>	34
3.3 Mainband Block.....	36
3.3.1 <i>Mainband Block Overview</i>	36
3.3.2 <i>TX Path</i>	37
3.3.3 <i>RX Path</i>	37
3.4 Packer.....	38
3.4.1 <i>Overview</i>	38

3.4.2	<i>Signal Description</i>	38
3.4.3	<i>TX Path Operation</i>	39
3.5	CRC Generator	40
3.5.1	<i>Overview</i>	40
3.5.2	<i>Signal Description</i>	40
3.5.3	<i>CRC-16 Architecture (64 Bytes per Clock)</i>	41
3.5.4	<i>CRC Generator Operation</i>	41
3.6	TX Operation	42
3.6.1	<i>TX Path Pipeline</i>	42
3.6.2	<i>TX Flow Chart</i>	43
3.7	Unpacker	44
3.7.1	<i>Overview</i>	44
3.7.2	<i>Signal Description</i>	44
3.7.3	<i>RX Path Operation</i>	45
3.7.4	<i>Flit Cancel Mechanism</i>	46
3.8	CRC Check	47
3.8.1	<i>Overview</i>	47
3.8.2	<i>Signal Description</i>	47
3.9	RX Operation	48
3.9.1	<i>RX Flow Chart</i>	48
3.9.2	<i>CRC Check Operation</i>	49
3.10	Implementation	50
3.10.1	<i>Overview</i>	50
3.10.2	<i>Byte Map</i>	50
3.10.3	<i>Chunk 3 Bit-Field Offsets (within 512-bit chunk)</i>	50
3.10.4	<i>Flit Header Encoding</i>	51
3.10.5	<i>Architecture & Timing</i>	51
3.10.6	<i>Cycle-by-Cycle Timing</i>	51
3.10.7	<i>Pipeline Timing</i>	52
3.10.8	<i>FSM State Descriptions TX</i>	52
3.10.9	<i>Data Source per Mode</i>	52

3.10.10	<i>Pipeline Timing</i>	52
3.10.11	<i>FSM State Descriptions RX</i>	53
3.10.12	<i>Chunk 3 Field Extraction</i>	53
3.10.13	<i>Formation Conventions</i>	53
3.10.14	<i>Modules Description</i>	53
3.11	Results	54
3.11.1	<i>TX</i>	54
3.11.2	<i>RX</i>	55
3.11.3	<i>Mainband After Integration</i>	56
3.12	Overview of Retry Block	57
3.12.1	<i>Retry External Interfaces</i>	58
3.12.2	<i>Retry Counters and Flags</i>	59
3.13	Precedence of Retry	60
3.13.1	<i>Transmitter Rules</i>	61
3.13.2	<i>Receiver Rules</i>	62
3.13.3	<i>Replay Schedule and Transmit</i>	64
3.13.4	<i>Retry Internal Block</i>	65
3.13.5	<i>Retry Flow Chart</i>	74
3.13.6	<i>Test Cases</i>	75
3.13.7	<i>Results of Retry</i>	76
Chapter 4:	Sub-Block: Sideband	77
4.1	Overview	77
4.2	Sideband Final Architecture	78
4.3	Block Documentation	79
4.3.1	<i>Parameter Exchange Block</i>	79
4.3.2	<i>Access Arbiter</i>	81
4.3.3	<i>Mailbox Controller</i>	83
4.3.4	<i>FDI packer</i>	85
4.3.5	<i>FDI Controller</i>	86
4.3.6	<i>RDI controller</i>	88
4.3.7	<i>Msg Controller TX</i>	90

4.3.8	<i>Remote die request controller</i>	91
4.3.9	<i>Credit loop</i>	93
4.3.10	<i>Comp controller</i>	94
4.3.11	<i>Msg controller RX</i>	96
4.3.12	<i>RX Controller</i>	97
4.3.13	<i>Tag Manager</i>	99
4.3.14	<i>FIFO</i>	100
4.4	Results	102
4.4.1	<i>Sideband TX</i>	102
4.4.2	<i>Sideband RX TOP</i>	103
4.4.3	<i>Sideband Full Test</i>	104
Chapter 5: sub-block: ALSM & Register File		105
5.1	Block Diagram	105
5.2	Purpose	105
5.3	Functionality	107
5.4	Signal Description	107
5.5	Implmentation	109
5.5.1	<i>Link Bringup</i>	109
5.5.2	<i>Retrain</i>	111
5.5.3	<i>LinkError</i>	112
5.5.4	<i>LinkDown</i>	114
Results		116
5.6	Overview of Register File	117
5.6.1	<i>Register Discovery and Architectural Mapping</i>	117
5.6.2	<i>Register Attributes and Field Types</i>	117
5.6.3	<i>Configuration Space: UCle Link DVSEC</i>	117
5.6.4	<i>MMIO Space: D2D Register Block</i>	118
5.6.5	<i>Remote Access via the Sideband Mailbox Mechanism</i>	118
5.6.6	<i>UCle Registers Offsets</i>	119
Chapter 6: Adapter Top Level Integration & Verification		122
6.1	Overview of Final Architecture	122

6.2	Testing Methodology	123
6.3	Results	124
Chapter 7: ASIC Flow		127
7.1	RTL Linting using synopsys Spyglass Linting Tool	127
7.1.1	<i>Linting Results for Adapter Architecture</i>	127
7.2	Synthesis	127
7.2.1	<i>Design Constraints</i>	127
7.2.2	<i>Results</i>	129
7.3	Design For Testability (DFT)	130
7.3.1	<i>DFT Design Constraints</i>	130
7.3.2	<i>DFT Results</i>	132
7.3.3	<i>Summary of Design Report</i>	133
7.4	Formal Verification	133
Chapter 8: Conclusions and Future Work		135
8.1	Conclusion	135
8.2	Future Work	135

TABLE OF FIGURES

Figure 1 - UCle connecting different packages	18
Figure 2 – UCle Layers.....	19
Figure 3 – Adapter Layer Functionalities	20
Figure 4 - Project Design Flow	20
Figure 5 -Physical Layer Functionality.....	22
Figure 6 - UCle Link Stages	27
Figure 7 - Summary of all flit formats	29
Figure 8 - CRC Computation.....	30
Figure 9 – format 3 flit header	34
Figure 10 – flit structure of format 3	35
Figure 11 – Mainband block Top level	36
Figure 12 – TX block Path.....	37
Figure 13 – RX block Path.....	37
Figure 14 – Packer Block Top level.....	38
Figure 15 – CRC Gen sub-block Top Level	40
Figure 16 – CRC Computation	41
Figure 17 – TX Path Pipeline in Mainband	42
Figure 18 – TX Flow Chart in Mainband	43
Figure 19 – Unpacker block Top level in Mainband	44
Figure 20 – Flit Cancel Mechanism.....	46
Figure 21 – CRC Check Block in Mainband Block	47
Figure 22 – RX Flow Chart	48
Figure 23 – transcript of TX of mainband top module	54
Figure 24 – waveform of mainband top module.....	54
Figure 25 – transcript of RX of mainband top module.....	55
Figure 26 – waveform of RX of mainband top module.....	55
Figure 27 – mainband Top level Module waveform.....	56
Figure 28 – Retry block in mainband	57
Figure 29 – Schedules implemented	63
Figure 30 – Reviewer flow in Retry	63

Figure 31 – Replay types	64
Figure 32 – Flit replay transmit implemented.....	65
Figure 33 – Retry block internal subblock architecture.....	65
Figure 34 – phases in retry.....	71
Figure 35 – TX Path in retry	74
Figure 36 – RX path in retry	74
Figure 37 – data flow in retry	74
Figure 38 - Retry Top Testbench Transcript	76
Figure 39 - Retry Top Waveform	76
Figure 40 - sideband workflow	77
Figure 41 – sideband architecture 2	78
Figure 42 – sideband architecture 1	78
Figure 43 – parameter exchange block	79
Figure 44 – access arbiter block.....	81
Figure 45 – mailbox controller block.....	83
Figure 46 – FDI packer block	85
Figure 47 – FDI controller block	86
Figure 48 – RDI controller.....	88
Figure 49 – msg controller TX.....	90
Figure 50 – remote die request controller block	91
Figure 51 – credit loop block	93
Figure 52 – comp controller block.....	94
Figure 53 – msg controller RX block	96
Figure 54 – RX controller block	97
Figure 55 – Tag manager block.....	99
Figure 56 – FIFO block.....	100
Figure 57 - sideband TX TOP transcript.....	102
Figure 58 - sideband TX TOP waveform.....	102
Figure 59 - sideband RX TOP transcript.....	103
Figure 60 - sideband RX TOP waveform	103
Figure 61 - Sideband TOP transcript	104

Figure 62 – ALSM block interfaces.....	105
Figure 63 – LSM of adapter layer.....	106
Figure 64 -RDI State Machine.....	106
Figure 65 – FDI State Machine.....	107
Figure 66 – Link bringup flow on RDI.....	109
Figure 67 – Link bringup flow on FDI	109
Figure 68 – Bringup implemented flow.....	110
Figure 69 – Retrain Implemented flow	111
Figure 70 – LinkError flow.....	112
Figure 71 – LinkError Implemented flow.....	113
Figure 72 – LinkDown flow	114
Figure 73 – LinkReset implemented flow	114
Figure 74 – LinkDisable implemented flow	115
Figure 75 – ALSM TOP level waveform 1	116
Figure 76 – ALSM TOP level waveform 2	116
Figure 77 - UCle DVSEC	119
Figure 78 – UCle Implementation Top level	122
Figure 79 - top module waveform 1	124
Figure 80 - top module waveform 2	124
Figure 81 - active bringup on top level 1	124
Figure 82 - link bringup on top level 2	125
Figure 85 - link bringup on top level 3	125
Figure 83 - linkreset entry and exit 2	125
Figure 84 - linkreset entry and exit 1	125
Figure 86 - request and completion on top level.....	126
Figure 87 - mailbox on top level.....	126
Figure 88 - linkreset entry and exit 3	126
Figure 89 - RTL linting result	127
Figure 90 - qor report before DFT	129
Figure 91 - setup time report before DFT	129
Figure 92 - qor report results for DFT	132

Figure 93 - setup timing report for DFT	132
Figure 94 - Critical Path Post DFT	133
Figure 95 - synthesis summary	133
Figure 96 - formal results.....	134

LIST OF TABLES

Table 1 - Terms and Definitions.....	16
Table 2 – flit structure of format 3	35
Table 3 - chucks details.....	35
Table 4 Mainband block sub-blocks functionality	36
Table 5 - Signal Description of Packer block in Mainband Block.....	39
Table 6 - Signal Description of CRC Gen block in Mainband block	41
Table 7 - Signal Description of unpacker block in mainband block.....	45
Table 8 - Signal Description of CRC Check in Mainband block.....	47
Table 9 – blocks functionality.....	50
Table 10 – byte mapping.....	50
Table 11 – Chunk 3 Bit-Field Offsets in format 3	51
Table 12 – flit header encoding in format 3	51
Table 13 – Cycle Timing in format 3).....	51
Table 14 – Pipeline Timing in format 3	52
Table 15 – FSM states in TX in mainband.....	52
Table 16 – Data source per mode in TX block in mainband.....	52
Table 17 – Pipeline Timing in TX block in mainband.....	52
Table 18 – FSM states in RX block in mainband	53
Table 19 – Chunk 3 Field Extraction	53
Table 20 - formation Conventions	53
Table 21 – Description of mainband system RTL modules	54
Table 22 – UCIE simplified version of Retry mechanism.....	57
Table 23 – Retry TOP interface.....	58
Table 24 – Retry mechanism counters	59
Table 25 – Retry Mechanism flags	60
Table 26 - signal description of implicit rx rules block in retry	66
Table 27 - receiver Rules for retry and schedules.....	67
Table 28 - signal description of receiver rules block in retry	68
Table 29 - signal description of ACK/NAK processing block in retry	68
Table 30 - replay schedule mechanism	69

Table 31 - signal description of replay schedule block in retry	70
Table 32 - retry buffer operation	70
Table 33- ram pointers in retry buffer	70
Table 34 - parameters of memory	70
Table 35- signal description of buffer block in retry	71
Table 36 - signal description of transmitter rules block in retry	73
Table 37 - signal description of SNH condition checker block in retry.....	73
Table 38 - Retry Test Cases.....	75
Table 39 - Signal Description of Parameter exchange block in sideband.....	80
Table 40 - signal description of access arbiter block in sideband.....	82
Table 41 - signal description of mailbox controller block in sideband.....	84
Table 42 - signal description of FDI packer block in sideband	86
Table 43 - signal description of FDI controller block in sideband	87
Table 44 - signal description of RDI controller in sideband.....	89
Table 45 - signal description of msg controller TX in sideband.....	91
Table 46 - signal description of remote die request controller block in sideband	92
Table 47 - signal description of credit loop block in sideband	94
Table 48 - signal description of comp controller block in sideband	95
Table 49 - signal description of msg controller RX block in sideband.....	97
Table 50 - signal description of RX controller block in sideband	98
Table 51 - signal description of tag manager block in sideband.....	100
Table 52 - signal description of FIFO block in sideband.....	101
Table 53 - singal description of ALSM block interface	109
Table 54 - UCle Register File Offset	121
Table 55 - constraints without DFT	128
Table 56 - constraints for DFT	131

TERM AND DEFINITIONS

Term	Definition
Ack	Acknowledge.
Addr	Address
BAR	Base Address Register
BE	Byte Enable
BEI	BAR Equivalent Indicator
BER	Bit Error Ratio
bubble	Gap in data transfer and/or signal transitions. Measured in number of clock cycles.
CA	Completer Abort
chiplet	Integrated circuit die that contains a well-defined subset of functionality that is designed to be combined with other chiplets in a package.
CPU	Central Processing Unit
CXL	Compute eXpress Link
CXL 68B Flit Mode	This term is used to reference 68B Flit Mode related Protocol features defined in CXL Specification.
CXL 256B Flit Mode	This term is used to reference 256B Flit Mode related Protocol features defined in CXL Specification.
D2D	Die-to-Die
DDR	Double Data Rate Memory
DevID	Device ID
DLLP	Data Link Layer Packet (as defined in PCIe Base Specification)
DLP	In Flit modes, the Data Link Layer Payload within a flit (as defined in PCIe Base Specification).
DP	Downstream Port
DSP	Downstream Switch Port (as defined in CXL Specification).
DVSEC	Designated Vendor-Specific Extended Capability (as defined in PCIe Base Specification)
DWORD	Double Word. Four bytes. When used as an addressable quantity, a Double Word is four bytes of data that are aligned on a four-byte boundary (i.e., the least significant two bits of the address are 00b).
E2E	End to end
EP	Endpoint
FIFO	First In, First Out

FM	Flit Marker (as defined in PCIe Base Specification).
HW	Hardware
I/O	Input/Output
IP	Generic term used to refer to architecture blocks that are defined within the specification (e.g., D2D adapter, PHY, etc.).
LCLK	Refers to the clock at which the Logical Physical Layer, Adapter and RDI/FDI are operating.
Link UCIe Link	A Link or UCIe Link refers to the set of two UCIe components and their interconnecting Lanes which forms a dual-simplex communications path between the two components.
LSM	Adapter Link State Machine
LSB	Least Significant Bit
Mainband MB	Connection that constitutes the main data path of UCIe.
MMIO	Memory mapped Input/Output
MSB	Most Significant Bit
NOP	No Operation
P2P	Peer to peer
Packet	A block of data transmitted across a network.
PCIe (PCI Express)	Peripheral Component Interconnect Express (defined in PCIe Base Specification)
PCIe Flit Mode	This term is used to reference Flit Mode related Protocol features defined in PCIe Base Specification.
PCIe non-Flit Mode	This term is used to reference non-Flit Mode related Protocol features defined in PCIe Base Specification.
PHY	Physical Layer (PHY and Physical Layer are used interchangeable)
PM	Power Management states, used to refer to behavior and/or rules related to Power Management states (covers both L1 and L2).
PMO	Sideband Performant Mode Operation
QWORD	Quad Word. Eight bytes. When used as an addressable quantity, a Quad Word is eight bytes of data that are aligned on an eight-byte boundary (i.e., the least significant three bits of the address are 000b).

RAC	Read Access Control
RCiEP	Root Complex Integrated Endpoint
RCRB	Root Complex Register Block
RDI	Raw Die-to-Die Interface
remote Link partner	This term is used to denote the logic associated with the far side of the UCIe Link; to denote actions or messages sent or received by the Link partner of a UCIe die.
Replay Retry	sRetry and Replay are used interchangeably to refer to the Link level reliability mechanisms.

Table 1 - Terms and Definitions

Chapter 1: Introduction

1.1 Background and Motivation

The semiconductor industry has long followed Moore's Law, scaling transistor counts on a single monolithic die to achieve higher performance. However, as process node scaling becomes increasingly difficult, expensive, and power-limited, the industry is shifting toward a new paradigm: disaggregated die architectures, where multiple smaller chiplets — each implementing a well-defined subset of functionality — are integrated together inside a single package to form a complete System-in-Package (SiP). This approach offers several advantages, including improved manufacturing yield for individual dies, the freedom to mix and match dies from different process nodes, and faster time-to-market by reusing proven IP blocks across products.

To make this chiplet ecosystem viable, a standardized, open, and high-performance die-to-die (D2D) interconnect is essential. Proprietary die-to-die interfaces hinder interoperability between chiplets from different vendors, creating lock-in and limiting the diversity of the ecosystem. Universal Chiplet Interconnect express (UCIe) was created to address this gap — it is an open, multi-protocol capable, on-package interconnect standard for connecting multiple dies on the same package, designed to enable a vibrant, vendor-neutral chiplet marketplace.

1.2 Problem Overview

This graduation project focuses on the design and RTL implementation of the UCIe Die-to-Die (D2D) Adapter Layer, which is the central intelligence block of the UCIe stack. The D2D Adapter sits between the Protocol Layer above (PCIe, CXL, Streaming) and the Physical Layer below, coordinating link bring-up, protocol parameter negotiation, multi-protocol multiplexing, CRC computation, link-level retry, and power management state transitions. The adapter minimizes logic on the main data path to achieve low-latency, optimized Flit transfer while still providing the reliability and manageability features required for a production-grade chiplet interconnect. The implementation is guided by the UCIe Specification Revision 3.0, Version 1.0 (released August 5, 2025).

1.3 What is UCIe

Universal Chiplet Interconnect Express (UCIe) is an open, multi-protocol capable, on-package interconnect standard designed to connect multiple semiconductor dies within the same package. Its primary purpose is to enable a vibrant ecosystem for disaggregated chiplet architectures, allowing components from different vendors with varying performance characteristics to seamlessly interoperate. UCIe operates as a layered protocol composed of three main components: a Protocol Layer, a Die-to-Die (D2D) Adapter, and a Physical Layer. It is highly versatile and supports the transmission of multiple established protocols—including PCIe, CXL, and custom Streaming protocols—over a common physical and link layer. To accommodate a wide spectrum of low-cost to high-performance system designs, UCIe supports three different packaging options: Standard Package (2D), Advanced Package (2.5D), and UCIe-3D.

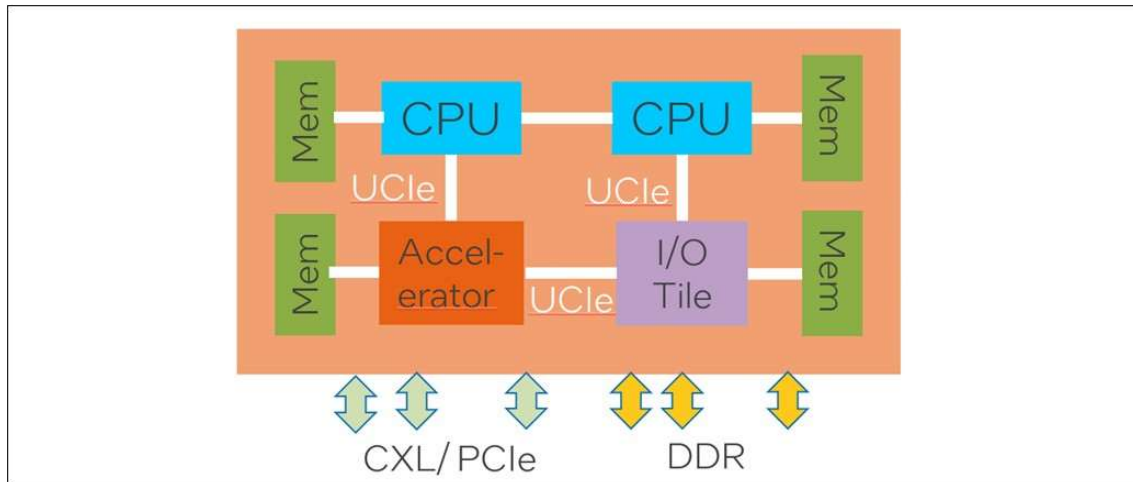


Figure 1 - UCIe connecting different packages

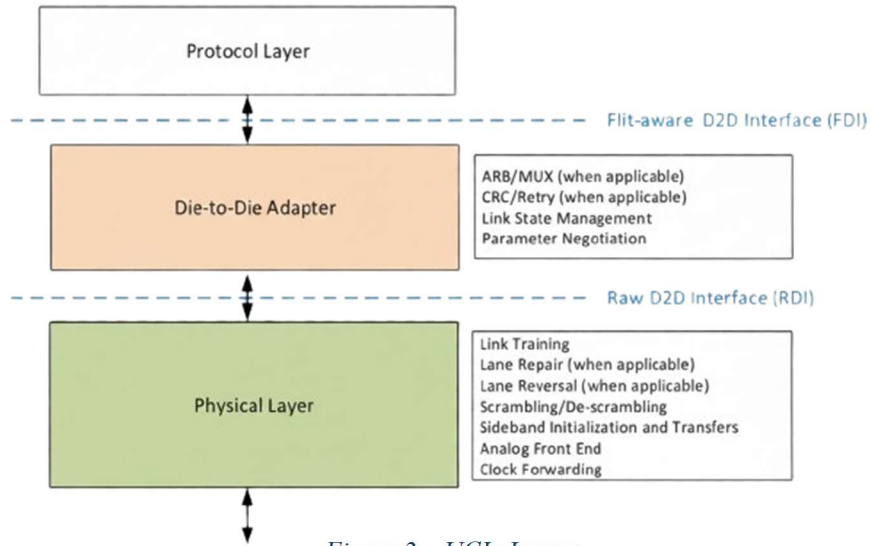
1.4 UCIe Architecture Overview

UCIe is a layered protocol stack consisting of three main components:

Protocol Layer: Carries the actual data traffic using one of the supported protocols — PCIe (Flit and non-Flit modes), CXL 68B/256B Flit Mode, Streaming protocol, or a Raw Format for user-defined protocols. Each protocol maps its transactions into UCIe Flit formats for transport.

Die-to-Die (D2D) Adapter: The focus of this project. It performs CRC and Retry, Link State Machine (LSM) management, and parameter exchange with the remote link partner. It interfaces to the Protocol Layer through the Flit-Aware D2D Interface (FDI) and to the Physical Layer through the Raw D2D Interface (RDI).

Physical Layer: Consists of a Logical PHY (link training, lane repair, lane reversal, scrambling, sideband transfers) and an Analog Front End (AFE). The physical link uses two connections: a Mainband (forwarded clock, data valid, N data lanes) for the main data path, and a Sideband (fixed 800 MHz forwarded clock and data pin, always-on domain) for parameter exchange, register access, and link training coordination.



1.5 UCle D2D Adapter Layer

The Die-to-Die (D2D) Adapter is a crucial intermediary layer in the UCle architecture that sits between the Protocol Layer and the Physical Layer. It interfaces with the Protocol Layer using the Flit-aware Die-to-Die Interface (FDI) and communicates with the Physical Layer via the Raw Die-to-Die Interface (RDI).

The D2D Adapter is specifically designed to minimize logic on the main data path, which provides a highly optimized, low-latency route for protocol Flits. Its core responsibilities include:

Reliable Data Transfer: It ensures successful data transmission across the UCle Link by implementing Cyclic Redundancy Check (CRC) and Retry mechanisms (or parity computation) to protect data, which is mandatory for options where the raw Bit Error Rate (BER) is higher than $1e-27$. **Link State and Power Management:** It orchestrates the higher-level Link state machine, controls link bring-up, and handles power management coordination with the remote Link partner. **Parameter Negotiation:** During the initialization phase, the adapter negotiates protocol options, operating modes, and Flit formats with the remote Link partner.

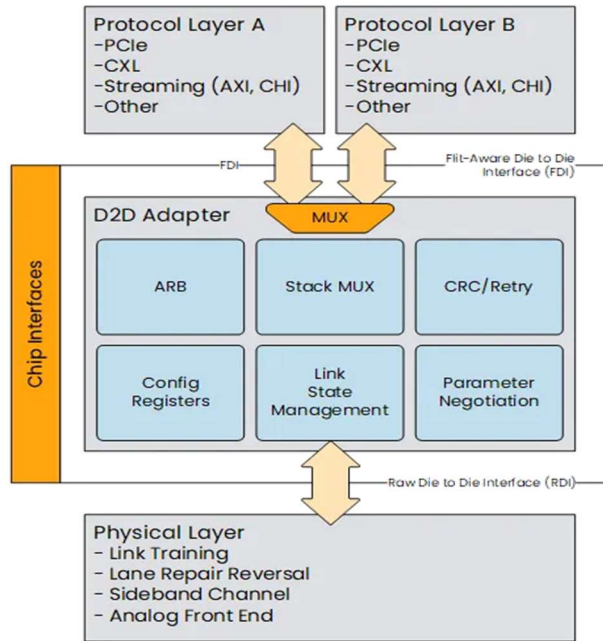


Figure 3 – Adapter Layer Functionalities

1.6 Objectives of this Project

The specific objectives of this graduation project are:

- Study and understand the UCle 3.0 D2D Adapter specification in depth, covering Flit formats, Link State Machine, CRC/Retry, and the FDI/RDI interface definitions.
- Design the RTL architecture of the UCle D2D Adapter Layer.
- Implement the design in SystemVerilog following the UCle 3.0 specification compliance requirements.
- Verify the implementation using functional simulation.
- Report area, timing, and bandwidth results against UCle 3.0 performance targets.

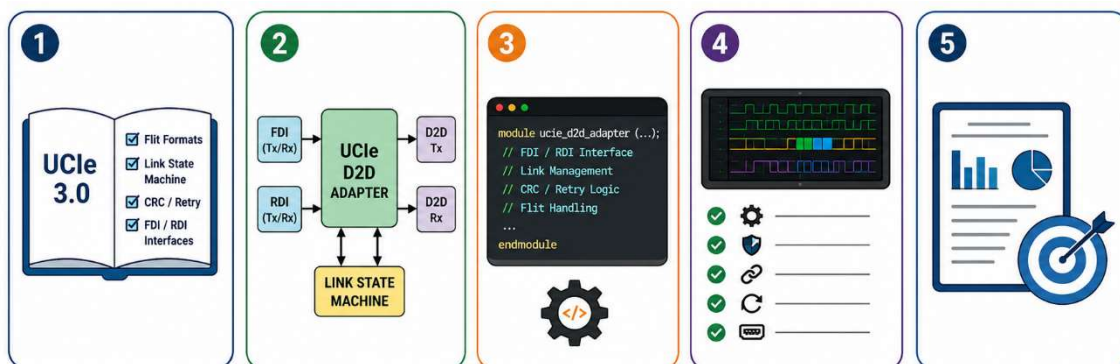


Figure 4 - Project Design Flow

Chapter 2: UCle 3.0 Specification Deep Dive

In the Universal Chiplet Interconnect Express (UCle) architecture, the Protocol Layer serves as the uppermost layer in the protocol stack, sitting above the Die-to-Die (D2D) Adapter and the Physical Layer. Its primary responsibility is to act as the application-specific engine that generates and consumes the specific protocol traffic traversing the interconnect. To enable an open chiplet ecosystem, the Protocol Layer is decoupled from the physical complexities of chiplet-to-chiplet communication. Instead, it interfaces with the underlying D2D Adapter using a standardized Flit-aware Die-to-Die Interface (FDI).

2.1 Overview of Protocol Layer

The UCle Protocol Layer natively supports transferring multiple industry-standard and custom protocols over a common physical Link. The protocols supported over the UCle mainband include: PCI Express (PCIe): The Protocol Layer supports PCIe Flit mode. In a PCIe context, this layer behaves as the Transaction Layer, responsible for the assembly and disassembly of Transaction Layer Packets (TLPs), handling transaction-level mechanisms like virtual channels, credit-based flow control, and PCI-compatible ordering rules. PCIe non-Flit mode is also supported but uses CXL 68B transport mechanisms. Compute Express Link (CXL): The Protocol Layer supports CXL 68B Flit mode and CXL 256B Flit mode. When CXL is negotiated, the specific sub-protocols (CXL.io, CXL.cache, and CXL.mem) are negotiated independently. Note that specific configurations like RCD/RCH/eRCD/eRCH are not supported. Streaming Protocol: This offers a generic mode for users to map any custom, proprietary, or vendor-defined protocol over UCle. Management Transport Protocol: An end-to-end media-independent protocol used to transfer manageability packets across the system's Management Network.

The Protocol Layer does not arbitrarily decide how to pack its data. During Link initialization, the D2D Adapter negotiates the operating mode with the remote Link partner and communicates the selected protocol and Flit Format to the Protocol Layer over the FDI. The Protocol Layer must adhere to the negotiated format: Raw Format: The Protocol Layer populates all bytes of the data transfer, taking full responsibility for reliable transport, Retry, Forward Error Correction (FEC), and CRC. This is often used by UCle Retimers transporting PCIe or CXL. Standard 256B Flit Formats (Start Header / End Header): Mandatory when CXL 256B Flit Mode or PCIe Flit Mode is supported. The Protocol Layer presents 256 bytes per Flit on the FDI but drives 0s on the specific bit fields reserved for the Adapter. 68B Flit Format: Mandatory for CXL and PCIe if the negotiated data rate is 32 GT/s or lower. The Protocol Layer presents 64 bytes of payload on the FDI, while the Adapter adds a 2-byte header and 2-byte CRC to complete the 68B Flit. Latency-Optimized 256B Flit Formats: An optional capability (with or without optional bytes) providing independent operating points that allow the Protocol Layer to achieve better Flit packing efficiency and lower accumulation latency.

The Protocol Layer utilizes the FDI to execute data transfers and state transitions seamlessly. Data Transfer: The Protocol Layer transmits data to the Adapter using the `lp_data` signal, asserting `lp_valid` and `lp_irdy` to indicate the data is ready to be sampled. It

receives data from the Adapter via the pl_data and pl_valid signals. Link State and Power Management: The Protocol Layer can request transitions into power-saving states (such as L1 or L2) by changing its lp_state_req signal after internal idle criteria are met. It also supports dynamic clock gating, utilizing the lp_wake_req and lp_clk_ack handshakes to wake the Adapter or respond to the Adapter waking the Protocol Layer. DLLP and Flow Control: In formats that support it, the Protocol Layer manages credit updates by passing Data Link Layer Packets (DLLPs) via dedicated FDI signals (lp_dllp, pl_dllp) out-of-band from the main data Flits.

2.2 Overview of Physical Layer

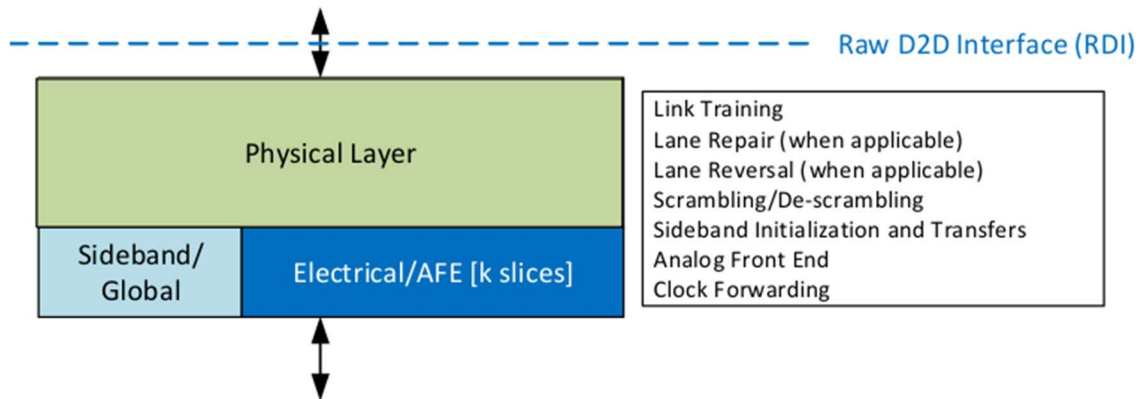


Figure 5 -Physical Layer Functionality

In the Universal Chiplet Interconnect Express (UCIE) architecture, the Physical Layer is the lowest layer of the protocol stack. It is responsible for the actual electrical transmission and reception of data across the physical chiplet-to-chiplet interconnect. It interfaces with the Die-to-Die (D2D) Adapter via the Raw Die-to-Die Interface (RDI) and shields the upper layers from the complexities of the physical signaling media. The Physical Layer is architected into three main sub-components: the Logical Physical Layer, the Electrical/Analog Front End (AFE), and the Sideband/Global control.

The Physical Layer organizes the data path on the physical bumps into groupings called a Module, which forms the atomic granularity for the structural design. Depending on the packaging technology, Advanced Packages utilize x64 or x32 Modules, while Standard Packages utilize x16 or x8 Modules. The physical link is divided into two distinct connections: Mainband: This is the high-speed primary data path. It consists of multiple data Lanes, a forwarded clock, a Data Valid pin (used to frame data and enable rapid clock gating), and a Track pin (used for runtime recalibration against voltage and temperature drifts). Sideband: This is an "always-on," low-speed connection used for link training, parameter exchange, register accesses for debugging, and power management. It consists of single-ended data and clock pins in each direction, and operates at a fixed 800 MHz clock rate, completely independent of the mainband's speed.

The Logical Physical Layer coordinates functions and relative sequencing to ensure proper link bring-up, management, and error recovery. Its core responsibilities include:

Byte-to-Lane Mapping: Data is transmitted byte-by-byte per Lane, with bit 0 transmitted first. **Scrambling and Training:** It implements a Linear Feedback Shift Register (LFSR) to scramble data for electrical reliability and generate training patterns during link bring-up. **Interconnect Redundancy and Repair:** Because physical bumps can fail, Advanced Package Modules include extra redundant pins. The Logical PHY detects failing Lanes (Data, Clock, Track, or Valid) and dynamically remaps the signals to the redundant Lanes. Standard Package Modules do not have redundant pins; instead, they rely on symmetric Width Degradation (e.g., degrading a x16 link to x8) if Lanes fail. **Link Training State Machine (LTSM):** The Logical PHY orchestrates the state machine required to bring the link online. Key states include RESET, SBINIT (Sideband Initialization), MBINIT (Mainband Initialization/Repair), MBTRAIN (Mainband Training and Clock-to-Data centering), LINKINIT, ACTIVE (normal operation), and low-power states (L1/L2).

The Electrical Layer (or AFE) handles the analog transmission of bits over the 2D, 2.5D, or 3D physical media.

Signaling: The interface utilizes single-ended, unidirectional data signaling over a DC-coupled, point-to-point interconnect. It leverages a forwarded clock architecture for transmit jitter tracking. **Data Rates:** It supports a highly scalable range of operating data rates per Lane: 4, 8, 12, 16, 24, 32, 48, and up to 64 GT/s. **Equalization and Termination:** To meet bit error rate (BER) targets, Receivers on Advanced Packages remain unterminated up to 32 GT/s to save power, but are terminated to ground for 48 GT/s and 64 GT/s. At higher speeds (48/64 GT/s), the Transmitter applies multi-tap equalization (FIR) to compensate for channel loss. **Power and Clock Gating:** The Physical Layer employs dynamic clock gating. By utilizing the Data Valid signal, the transmitter can gate the forwarded clock to save power during idle periods, allowing the link to quickly enter and exit idle states without tearing down the connection

2.3 Overview of Raw Die-to-Die Interface (RDI)

In the Universal Chiplet Interconnect Express (UCIe) architecture, the Raw Die-to-Die Interface (RDI) serves as the critical standardized boundary between the Die-to-Die (D2D) Adapter and the Physical Layer. RDI abstracts the complexities of the physical medium, allowing the D2D Adapter to seamlessly coordinate Link state management and data transfers. A single RDI instance can map to a single Physical Layer module, or it can coordinate across multiple modules using Multi-module PHY Logic.

RDI requires both sides of the interface to share the same reset domain and operate primarily on the same clock domain, (lclk). Below is a breakdown of all the primary interfaces and signal groups that comprise RDI:

Data is transferred across RDI using a byte-oriented approach where the data width (lp_data and pl_data) is determined by the implementation. **Transmit (Adapter to PHY):** To send data, the Adapter asserts lp_valid and lp_iridy. The Physical Layer indicates its

readiness to accept data by asserting `pl_trdy`. A successful data transfer occurs on the rising edge of `lclk` when all three signals are asserted. The Adapter is not permitted to insert bubbles (gaps) in the middle of a continuous Flit transfer. Receive (PHY to Adapter): The Physical Layer sends data to the Adapter by asserting `pl_valid`. Notably, there is no backpressure mechanism (like a `trdy` signal) from the Adapter to the Physical Layer on the main data path, meaning the Adapter must always be ready to sink incoming data.

RDI utilizes a hierarchical state machine featuring states such as Reset, Active, Retrain, LinkError, Disabled, LinkReset. `lp_state_req` and `pl_state_sts`: The Adapter requests state transitions using the `lp_state_req` signal, and the Physical Layer communicates the current RDI status back to the Adapter using the `pl_state_sts` signal. `lp_linkerror`: This provides an immediate, decoupled indication from the Adapter that a fatal error has occurred, forcing the Physical Layer to immediately transition to the LinkError state for error containment.

To save power, RDI permits dynamic coarse clock gating when the interface is in Reset, LinkReset, Disabled, or PM states. This is safely coordinated using two distinct four-way handshakes: `lp_wake_req` and `pl_wake_ack`: The Adapter asynchronously requests the Physical Layer to ungate its internal clocks; the PHY acknowledges once the clocks are active. `pl_clk_req` and `lp_clk_ack`: The Physical Layer asynchronously requests the Adapter to ungate its clocks; the Adapter acknowledges once ready.

The Physical Layer continuously updates the Adapter on the health and configuration of the physical Link: Link Training & Configuration: `pl_inband_pres` asserts when the PHY has successfully finished Link training and is ready to transition to the Active state. `pl_phyinrecenter` warns the Adapter if the PHY drops into training or retraining. Furthermore, `pl_speedmode` and `pl_lnk_cfg` dynamically report the current Link data rate (e.g., 4 GT/s to 64 GT/s) and active Link width (e.g., x8 to x256). Error Reporting: The PHY flags specific hardware issues to the Adapter. `pl_error` indicates a recoverable framing error that requires Link retraining. `pl_nfererror` flags non-fatal implementation-specific errors. `pl_trainerror` indicates a fatal error that automatically transitions the Link to the LinkError state.

RDI includes a robust out-of-band communication channel for configuration and parameter exchanges: Standard Sideband (`lp_cfg` / `pl_cfg`): These buses (ranging from 8 to 32 bits wide), alongside their respective valid (`*_vld`) and credit (`*_crd`) signals, pass register accesses and Link management packets between the Adapter and Physical Layer.

2.4 Overview of Flit-aware Die-to-Die Interface (FDI)

In the Universal Chiplet Interconnect Express (UCIe) architecture, the Flit-Aware Die-to-Die Interface (FDI) serves as the standardized boundary between the Protocol Layer and the Die-to-Die (D2D) Adapter. While the Raw Die-to-Die Interface (RDI) handles raw data over the physical medium, FDI is specifically designed to be "Flit-aware," meaning it incorporates mechanisms for handling Protocol Flits, out-of-band Data Link Layer Packets (DLLPs), and multi-protocol stack multiplexing.

FDI requires both sides of the interface to share the same reset domain and operate primarily on the same clock domain (lclk). Below is a breakdown of the primary interfaces and signal groups that comprise FDI: Data is transferred across FDI using a byte-oriented approach where the data width (lp_data and pl_data) is determined by the implementation (e.g., 64B, 128B, or 256B).

Transmit (Protocol Layer to Adapter): To send data, the Protocol Layer asserts lp_valid and lp_irdy. The Adapter indicates readiness by asserting pl_trdy. A successful data transfer occurs on the rising edge of lclk when all three signals are asserted. The Protocol Layer is not permitted to insert bubbles (gaps) in the middle of a continuous Flit transfer. **Receive (Adapter to Protocol Layer):** The Adapter sends data to the Protocol Layer by asserting pl_valid. Unlike the transmit side, there is no backpressure mechanism from the Protocol Layer to the Adapter on the main data path; the Protocol Layer must always be ready to sink incoming data, and the Adapter is permitted to insert bubbles. **Flit Cancellation (pl_flit_cancel):** To optimize latency, the Adapter can forward data to the Protocol Layer before CRC checks are fully complete. If the Adapter subsequently detects a CRC error, it asserts pl_flit_cancel to instruct the Protocol Layer to discard (dump) the corrupted Flit or half-Flit.

FDI includes dedicated signaling to handle out-of-band control packets and support protocol multiplexing: **Out-of-Band DLLPs (lp_dllp, pl_dllp):** For protocols like PCIe and CXL.io 256B Flit Modes, credit updates and flow control must bypass the Adapter's Retry Buffer. FDI provides dedicated DLLP buses (lp_dllp and pl_dllp), along with their respective valid signals, so the Protocol Layer and Adapter can exchange DLLPs independently of the main data path. The lp_dllp_ofc and pl_dllp_ofc signals indicate when these DLLPs are using the Optimized_Update_FC format. **Stream IDs (lp_stream, pl_stream):** Because a single Adapter can multiplex traffic from multiple Protocol Layers (e.g., PCIe, CXL.io, CXL.cachemem), these 8-bit signals identify the specific protocol and stack ID associated with the current data transfer.

FDI utilizes a state machine mirroring the upper-level Link states (Reset, Active, LinkReset, Retrain, Disabled, LinkError).

State Requests and Status: The Protocol Layer requests state transitions using lp_state_req, and the Adapter reports the current status using pl_state_sts. **Protocol Negotiation:** During initialization, the Adapter negotiates the operating parameters with the remote link partner. It uses pl_protocol, pl_protocol_flitfmt, and pl_protocol_vld to inform the Protocol Layer exactly which protocol (e.g., PCIe, CXL) and Flit Format (e.g., 68B, Standard 256B, Latency-Optimized 256B) have been negotiated. pl_inband_pres asserts when the Adapter is ready to transition to the Active state. **Receiver Active Handshake:** The Adapter ensures the Protocol Layer's receiver is ready using the pl_rx_active_req and lp_rx_active_sts handshake. **Stall Mechanism (pl_stallreq, lp_stallack):** The Adapter uses this handshake to force the Protocol Layer to flush its pipeline and halt on a clean Flit boundary prior to major state transitions, such as entering power management states. Similar to RDI, FDI allows dynamic coarse clock gating to save power during idle states (Reset, LinkReset, Disabled). **Adapter Clock Request (lp_wake_req / pl_wake_ack):** The Protocol Layer

asynchronously requests the Adapter to ungate its internal clocks; the Adapter acknowledges once its clocks are active. Protocol Layer Clock Request (pl_clk_req / lp_clk_ack): The Adapter asynchronously requests the Protocol Layer to ungate its clocks; the Protocol Layer acknowledges once its clocks are active.

FDI includes a robust out-of-band communication channel for error reporting and sideband buses: Sideband Bus (lp_cfg, pl_cfg): A robust out-of-band communication channel (ranging from 8 to 32 bits wide) accompanied by valid and credit signals, allowing the Protocol Layer indirect access to remote registers via the Adapter's mailbox mechanisms. Error Reporting: The Adapter flags specific hardware issues to the Protocol Layer. pl_error indicates a framing or CRC error. pl_cerror flags correctable internal errors. pl_nfererror indicates a non-fatal error, and pl_trainerror indicates a fatal error that forces the interface into the LinkError state. Furthermore, the Protocol Layer can trigger an immediate link-down transition by asserting the lp_linkerror signal.

2.5 Overview of Adapter Layer

The D2D Adapter is responsible for the following high-level functions:

Link Initialization (Stage 3): Performing capability determination, parameter exchange with the remote link partner, and FDI bring-up after Physical Layer training is complete

Operation Format Management: Selecting and enforcing the correct Flit Format (Raw, 68B, Standard 256B, Latency-Optimized 256B) based on the negotiated protocol

Link State Machine (LSM) Management: Coordinating link state transitions (Active, Retrain, PM, LinkError, LinkReset, Disabled) with the remote link partner via sideband messages.

CRC Computation and Retry: Computing Link CRC over transmitted Flits and managing the Retry Buffer for link-level error recovery

The Adapter interfaces to the Protocol Layer through the Flit-Aware D2D Interface (FDI) and to the Physical Layer through the Raw D2D Interface (RDI). These standardized interfaces allow IP from different vendors to be mixed and matched at low integration cost.

2.5.1 Link Initialization (Stage 3 — Adapter Initialization)

UCIe link initialization consists of four sequential stages. Stage 0 is die-specific (independent reset), Stage 1 is sideband initialization, Stage 2 is mainband training and lane repair (both handled by the Physical Layer), and Stage 3 is the Adapter Initialization, which begins once the RDI state machine reaches the Active state following successful Stage 2 completion. Stage 3 proceeds in three sequential parts:

The Adapter first determines the results of Physical Layer training — specifically whether Retry must be enabled for the negotiated link speed and configuration. If the Adapter is capable of supporting Retry, it advertises this capability. It also determines its supported Flit Formats, protocol types (PCIe, CXL, Streaming, Raw), and CXL-specific capabilities (CXL.io, CXL.cachemem), and packages these into an Advertised Capability structure.

Both sides exchange their Advertised Capabilities over the sideband link. After receiving the remote partner's capabilities, each Adapter computes the Finalized Capabilities (FinCap) — the intersection of what both sides support. The parameter exchange covers: Adapter capabilities: Flit Format support, Retry enable/disable, CXL capabilities (if applicable): 68B vs 256B Flit Mode, CXL.io, CXL.cachemem sub-protocol support

After the sideband negotiation succeeds and both sides agree on the final parameters, the adapter can safely derive the Finalized Capabilities and continue the bring-up process. Only then does the link move toward FDI activation, allowing the Protocol Layer to begin sending and receiving Flits over the operational UCle link.

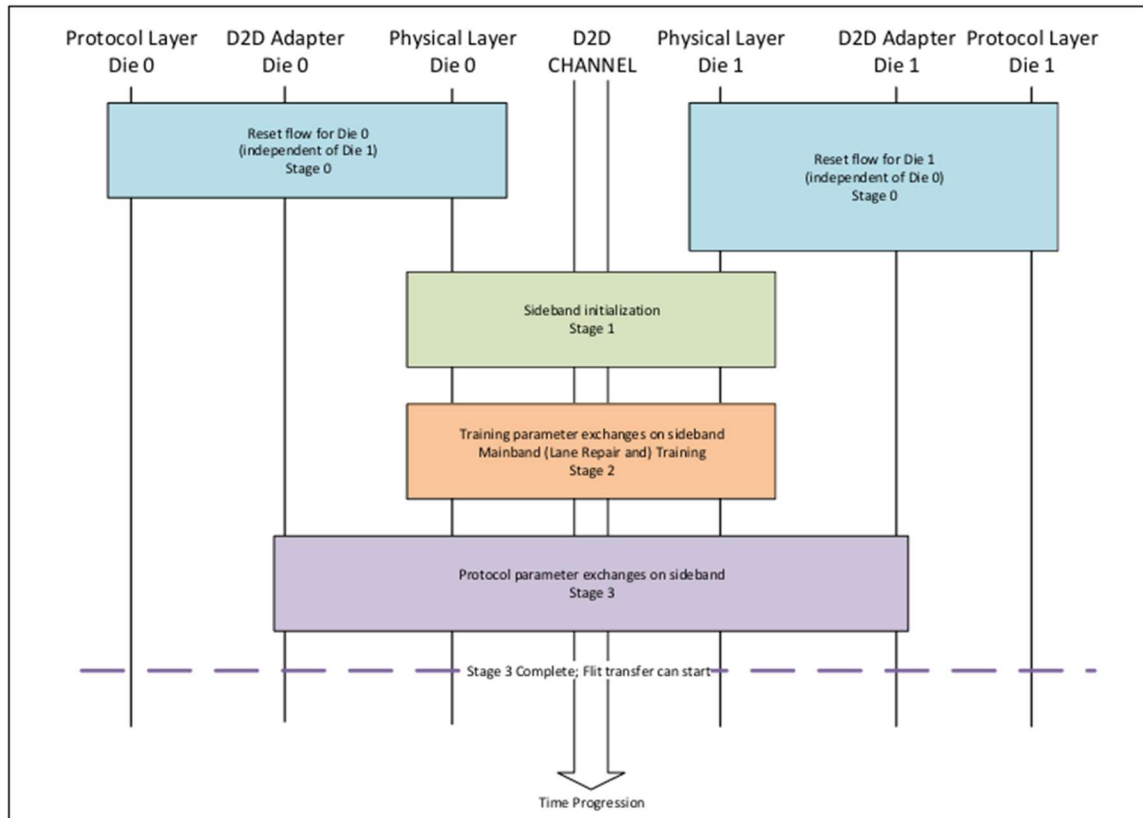


Figure 6 - UCle Link Stages

2.5.2 Operation Formats (Flit Formats)

The D2D Adapter supports six Flit Formats, selected based on the negotiated protocol and capabilities. Understanding these formats is critical to the adapter implementation because they define the exact byte layout of every unit of data transferred across the link.

- **Format 1: Raw Format:** all bytes of the Flit are populated by the Protocol Layer. The D2D Adapter passes the Flit through without inserting CRC or performing Retry. This format is used when the Protocol Layer takes full responsibility for

reliable transport. Raw Format and Multi_Protocol_Enable (or Enhanced Multi_Protocol_Enable) are mutually exclusive.

- Format 2: 68B Flit Format: 68-byte unit consisting of a 2-byte Flit Header, 64 bytes of Data Link Layer Payload (DLP), and a 2-byte Link CRC (LCRC). It is used for PCIe and CXL 68B Flit Mode, and for the Streaming protocol. The Adapter inserts the LCRC field and performs Retry when this format is used. Alignment and padding rules ensure that when the data stream is paused (PDS — Pause of Data Stream), extra zero-padded bytes are appended to make the total transfer a multiple of 256 bytes.
- Format 3: Standard 256B End Header Flit Format: This format packs multiple protocol packets into a 256-byte Flit where the header appears at the end of the Flit body. It is used for PCIe, Streaming Protocol, and Management Transport Protocol. The Adapter inserts the CRC field and manages Retry for this format.
- Format 4: Standard 256B Start Header Flit Format: Similar to Format 3 but with the header located at the beginning of the Flit. Used for CXL.cachemem, CXL.io, PCIe, Streaming, and Management Transport. This allows the receiver to parse the header immediately upon receiving the first bytes of the Flit.
- Format 5: Latency-Optimized 256B without Optional Bytes: Removes optional padding bytes to minimize latency on the critical path. Used for CXL.io, CXL.cachemem, Streaming, and Management Transport protocols. Suitable for implementations where latency is the primary concern.

- Format 6: Latency-Optimized 256B with Optional Bytes: Extends Format 5 by allowing optional byte fields (e.g., for DLLP transfer in PCIe 256B Flit Mode). Used for CXL.io, PCIe, CXL.cachemem, and Streaming protocols.

Format Number	Name	Notes	For Details, See Also
Format 1	Raw	Protocol Layer populates all the bytes on FDI. Adapter passes to RDI without modifications or additions.	• Section 3.3.1 • Figure 3-10
Format 2	68B Flit	Protocol Layer transmits 64B per Flit on FDI. Adapter inserts two bytes of Flit header and two bytes of CRC and performs the required barrel shifting of bytes before transmitting on RDI. On the Rx, Adapter strips out the Flit header and CRC only sending the 64B per Flit to the Protocol Layer on FDI.	• Section 3.3.2 • Figure 3-11 • Figure 3-12
Format 3	Standard 256B End Header Flit	Protocol Layer transmits 256B of Flit on FDI, while driving 0b on the bits reserved for the Adapter. Adapter fills in the relevant Flit header and CRC information before transmitting on RDI. On the Rx, Adapter forwards the Flit received from the Link to the Protocol Layer without modifying any bits applicable to the Protocol Layer, and the Protocol Layer must ignore any bits not applicable for it. Flit Header is located on Byte 236 and Byte 237 of the Flit.	• Section 3.3.3 • Figure 3-15
Format 4	Standard 256B Start Header Flit	Protocol Layer transmits 256B of Flit on FDI, while driving 0b on the bits reserved for the Adapter. Adapter fills in the relevant Flit header and CRC information before transmitting on RDI. On the Rx, Adapter forwards the Flit received from the Link to the Protocol Layer without modifying any bits applicable to the Protocol Layer, and the Protocol Layer must ignore any bits not applicable for it. Flit Header is located on Byte 0 and Byte 1 of the Flit.	• Section 3.3.3 • Figure 3-17 • Figure 3-18
Format 5	Latency-Optimized 256B without Optional Bytes Flit	Protocol Layer transmits 256B of Flit on FDI, while driving 0b on the bits reserved for the Adapter. Adapter fills in the relevant Flit header and CRC information before transmitting on RDI. On the Rx, Adapter forwards the Flit received from the Link to the Protocol Layer without modifying any bits applicable to the Protocol Layer, and the Protocol Layer must ignore any bits not applicable for it. CRC bytes sent with each 128B of the Flit. The optional Protocol Layer bytes are reserved in this format and not used by the Protocol Layer.	• Section 3.3.4 • Figure 3-20 • Figure 3-21
Format 6	Latency-Optimized 256B with Optional Bytes Flit	Protocol Layer transmits 256B of Flit on FDI, while driving 0b on the bits reserved for the Adapter. Adapter fills in the relevant Flit header and CRC information before transmitting on RDI. On the Rx, Adapter forwards the Flit received from the Link to the Protocol Layer without modifying any bits applicable to the Protocol Layer, and the Protocol Layer must ignore any bits not applicable for it. CRC bytes sent with each 128B of the Flit, and optional bytes are used by the Protocol Layer.	• Section 3.3.4 • Figure 3-23 • Figure 3-24 • Figure 3-25

Figure 7 - Summary of all flit formats

2.5.3 State Machine Hierarchy

UCIe uses a hierarchical Link State Machine (LSM) architecture to provide well-defined functionality partitioning between layers and enable coordinated state transitions at both FDI and RDI interfaces. Sideband Coordination: Major state transitions—such as moving to Active, Retrain, LinkReset, or Disabled—are heavily coordinated with the remote Adapter using sideband messages.

The hierarchy is as follows (from bottom to top): RDI State Machine (SM): Abstracts the Physical Layer states for upper layers. There is a single RDI SM per UCIe link instance (even for multi-module configurations). Transitions are coordinated with the remote link partner using {LinkMgmt.RDI*} sideband messages, originated and received by the Physical Layer. Adapter LSM: One LSM per protocol stack. Transitions are coordinated with the remote link partner using {LinkMgmt.Adapter*} sideband messages, originated

and received by the D2D Adapter. Exposed on FDI via `pl_state_sts` for PCIe and Streaming protocols.

The ordering rules for state transitions are strictly enforced: Active: RDI SM must reach Active before Adapter LSM can negotiate Active. Adapter LSM must reach Active before vLSMs can negotiate Active. Retrain: RDI SM must enter Retrain before propagating Retrain to Adapter LSMs. All Adapter LSMs in Active must be propagated to Retrain before the Adapter requests RDI Retrain exit. LinkError: RDI SM must enter LinkError before Adapter LSM transitions to LinkError. LinkError takes priority over LinkReset or Disabled transitions. LinkReset / Disabled: Adapter LSM negotiates these transitions via sideband. LinkReset or Disabled is propagated to RDI SM only after all Adapter LSMs associated with it have transitioned to that state.

2.5.4 CRC Computation

For Flit Formats that require CRC (Formats 2 through 6), the D2D Adapter computes a Link CRC (LCRC) over the Flit payload. The CRC polynomial used is the standard CRC-32 polynomial as defined in the UCIE specification. The CRC is appended to each Flit on the transmit side, and the receiver checks it upon receipt. A CRC mismatch triggers the Retry mechanism.

For the 68B Flit Format, the LCRC is a 16-bit field occupying the last 2 bytes of the 68B Flit. For the 256B Flit Formats, the LCRC field is embedded within the Flit according to the specific format's byte map.

The CRC calculation logic receives the Flit bytes, passes them through a combinational CRC engine, and appends the resulting checksum to the outgoing Flit before it is presented to the RDI. On the receive side, the Adapter re-computes the CRC over the received Flit bytes and compares with the received LCRC. If the comparison fails, a NAK is sent to the remote link partner to request a Retry.

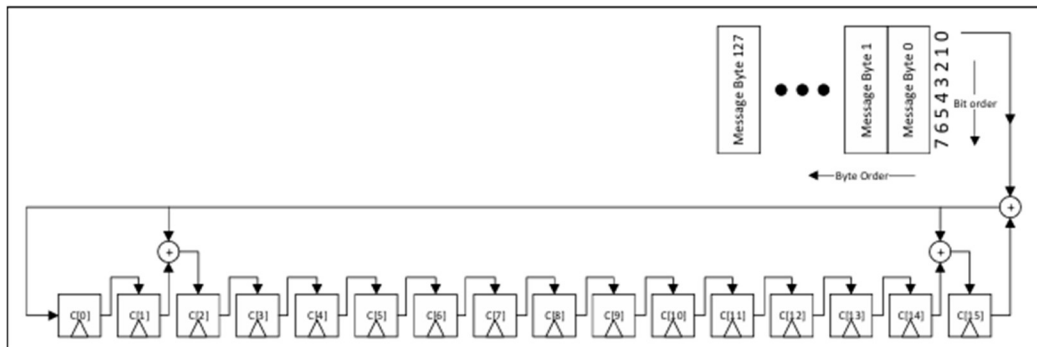


Figure 8 - CRC Computation

2.5.5 Retry Rules

The UCIE Retry mechanism provides link-level reliable transport for Flits whose Raw BER exceeds 1×10^{-27} at the configured link speed. Key rules governing the Retry mechanism

are: **Retry Buffer:** The transmitter maintains a Retry Buffer that stores all transmitted Flits until an ACK is received from the remote link partner. The buffer size is determined during the parameter exchange phase and must be sufficient to hold all in-flight Flits without causing backpressure on the Protocol Layer. **ACK/NAK Mechanism:** The receiver sends an ACK after successfully verifying the CRC of a received Flit. If a CRC error is detected, the receiver sends a NAK referencing the Flit sequence number. The transmitter then replays all Flits starting from the NAK'd Flit. **NOP Flit Handling:** NOP Flits bypass the Retry buffer — they are not stored and are not replayed during a Retry sequence. **When Retry is Required:** Retry must be enabled whenever the negotiated link speed and BER characteristics of the channel result in a Raw BER exceeding 1×10^{-27} for the given packaging option. The Adapter determines this after Physical Layer training completes in Stage 2.

2.6 System Total Flow

The complete packet flow traverses:

Packet Assembly (Transaction Layer): The flow begins in the upper Transaction Layer, whose primary responsibility is the assembly and disassembly of Transaction Layer Packets (TLPs). These TLPs are formed to communicate system-level transactions, such as Memory Reads, Memory Writes, and system events. Before a TLP can be sent, the Transaction Layer acts as a gatekeeper by strictly managing credit-based flow control to ensure the receiving device has enough buffer space to accept the packet.

Formatting and Reliability (Data Link / Adapter Layer): As the transmitted packets flow down through the subsequent layers, they are extended with additional information necessary to handle reliable transport. In a UCIE chiplet ecosystem, the Protocol Layer transmits the TLPs across the Flit-aware Die-to-Die Interface (FDI) to the D2D Adapter using a strict valid/ready handshake. Once the Adapter receives the data, it packs the TLPs into standardized fixed-size Flits. To protect the data from noise on the physical wire, the Adapter Layer generates and appends a Flit Header (for Sequence Numbering), dual Cyclic Redundancy Checks (CRC), and Forward Error Correction (FEC) bytes.

Electrical Transmission (Physical Layer): The Adapter passes the fully formed Flits over the Raw Die-to-Die Interface (RDI) to the Physical Layer. The Physical Layer organizes the data stream by mapping it byte-by-byte onto the available physical Lanes within the module. To improve the electrical characteristics of the Link and prevent electromagnetic interference, the Physical Layer XORs the data stream with a Linear Feedback Shift Register (LFSR) to scramble the bits. Depending on the operating speed, the data may be transmitted directly, or undergoes additional Gray Coding, Precoding, and PAM4 voltage translation before being driven as analog electrical signals across the physical chiplet bumps.

Signal Recovery and Decoding (Receive Physical Layer): At the receiving side, the exact reverse process occurs. The receiving Physical Layer captures the analog voltage levels and converts them back into a digital bitstream. For high-speed PAM4 links, it performs

Receive Precoding, Gray code decoding, and single-bit descrambling to extract the raw Flit. This data is then transferred up across the RDI to the receiving Adapter.

Error Checking and Extraction (Receive Adapter Layer): As the Adapter receives the raw Flit, it performs a 3-way interleaved FEC decode to mathematically correct any minor bit-flips caused by channel noise. Following the FEC correction, the Adapter evaluates the CRC bytes. If the CRC check passes, the Adapter acknowledges the Flit and forwards the extracted Protocol TLPs up across the FDI. If the CRC fails, the Adapter discards the corrupted data and sends a Nak to force the Transmitter to replay the Flit from its Retry Buffer.

Final Consumption (Receive Transaction Layer): The packet is ultimately transformed back into a form that can be processed by the Receive Transaction Layer. The Transaction Layer disassembles the TLP, routes the data to the appropriate internal memory or configuration address, and updates its local Flow Control credits so that new traffic can continue to flow across the Link.

Chapter 3: Sub-block: MainBand using Retry Mechanism

3.1 Overview of Mainband block

The Mainband is one of the core components of the UCle Adapter and is structured into three main blocks, each with a distinct and modular role in the data path: The TX block is responsible for constructing and transmitting flits. It receives data flits from the Protocol Layer through the FDI (Flit-aware Die-to-Die Interface). Based on the negotiated flit format, it: o Constructs the appropriate flit header, Calculates the CRC, Organizes the data into chunks (as per format requirements), and finally transmits the formatted flit through the RDI (Raw Die-to-Die Interface) to the Physical Layer. RX block handles incoming flits from the Physical Layer via the RDI. It parses the flit header to extract relevant information (e.g., format, sequence number, control info), Validates the CRC to ensure data integrity, then forwards the valid flit payload to the Protocol Layer through the FDI interface. The Retry block manages the retry mechanism, which provides reliable data transmission if this feature is enabled. It supports: Tracking sequence numbers and detecting errors or dropped flits, Storing transmitted flits temporarily in the Retry Buffer, Handling retransmission (replay) of unacknowledged or negatively acknowledged flits, and coordinating with RX and TX to maintain protocol compliance during recovery. This three-block modular design ensures clean separation between transmission, reception, and error recovery logic. It allows for efficient implementation and simplifies testing, integration, and future scalability of the Mainband.

3.2 Format 3

3.2.1 Flit Header of Format 3

Table 1-1 defines the flit header structure applicable to Format 3, Format 4, Format 5, and Format 6 with Retry. The diagram below shows the complete header layout as specified in the UCIE standard.

Byte	Bit	Description			
		CXL 256B Flit Mode	PCIe Flit Mode	Streaming Protocol	Management Transport Protocol
0	[7:6]	Protocol Identifier: 00b: D2D Adapter/CXL.io NOP Flit 01b: CXL.io Flit 10b: CXL.cachemem Flit 11b: ARB/MUX Flit	Protocol Identifier: 00b: D2D Adapter/PCIe NOP Flit 01b: PCIe Flit All other encodings are reserved	Protocol Identifier: 00b: D2D Adapter NOP Flit Remaining encodings are permitted to be used by Protocol Layer in a vendor defined manner. Protocol Layer must never set this to 00b for Flits sent across FDI.	Protocol Identifier: 00b: D2D Adapter NOP Flit 01b: Management Flit All other encodings are reserved.
	[5]	Stack Identifier: 0: Stack 0 1: Stack 1			
	[4]	Reserved for CXL.cachemem For CXL.io or PCIe Flit Mode: 0: DLLP Payload in DLP 2..5 1: Optimized_Update_FC or Flit_Marker in DLP2..5		Reserved	
	[3:0]	The upper four bits of Sequence number "S" (i.e., S[7:4])			
1	[7:6]	Flit Type: 00b: CXL/PCIe/Streaming Flit/D2D Adapter NOP Flit 01b: Test Flit (see Section 11.2 for details) 10b: Management Flit 11b: Reserved			
	[5:4]	Ack or Nak Information: 00b: Explicit Sequence number "S" of the current Flit is present. 01b: Ack. The sequence number "S" carries the Ack'ed sequence number. 10b: Nak. The sequence number "S" carries 255 if N=1; otherwise, it carries N-1; where N is the Nak'ed sequence number. 11b: Reserved			
	[3:0]	The lower four bits of Sequence number "S" (i.e., S[3:0]). Sequence number 0 is reserved and if present, it implies no Ack or Nak is sent.			

Figure 9 – format 3 flit header

3.2.2 Flit Structure Description

A complete 256-byte flit is structured as shown below. Each region serves a distinct purpose within the transmission pipeline.

Offset	Content
Byte 0	Flit Chunk 0 — 64B (from Protocol Layer)
Byte 64	Flit Chunk 1 — 64B (from Protocol Layer)
Byte 128	Flit Chunk 2 — 64B (from Protocol Layer)

Byte 192	44B of Flit Chunk 3 FH B0 FH B1 DLP B2–B5 10B Reserved CRC0 CRC1
-----------------	--

Table 2 – flit structure of format 3

Table 3 - chunks details

The individual flit fields are described as follows:

Part	Size / Location	Purpose
Flit Chunks 0–2	3 contiguous 64-byte regions, 192 bytes total	Carry unmodified payload data from the Protocol Layer.
Flit Chunk 3	Remaining 44 bytes	Completes the 256-byte flit size with protocol payload data and is included in CRC computation.
Flit Header (FH B0, FH B1)	2 bytes	Carries control metadata such as protocol type, sequence number, and Ack/Nak information when retry is enabled.
DLP Bytes (B2–B5)	4 bytes	Used by the Adapter to convey DLLP and Optimized Update Marker information.
Reserved bytes	10 bytes	Driven to zero; kept for format consistency and future protocol extensions.
CRC Bytes (CRC0 and CRC1)	2 x 16-bit CRC values	Generated by the Adapter to provide data integrity and validated by the receiver to detect transmission errors.

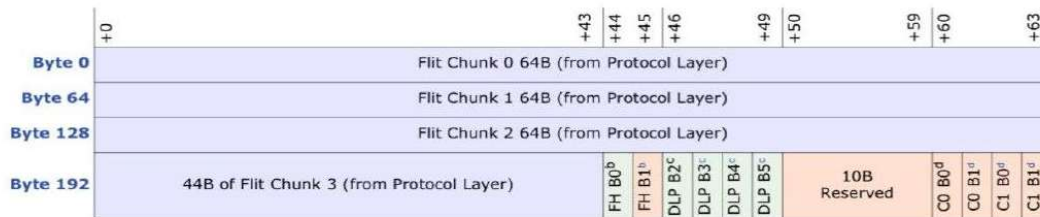


Figure 10 – flit structure of format 3

3.3 Mainband Block

3.3.1 Mainband Block Overview

The Mainband is the core data-path block within the UCIE Adapter. It handles flit-level processing in both the transmit (TX) and receive (RX) directions, bridging the Protocol Layer and the physical interface.

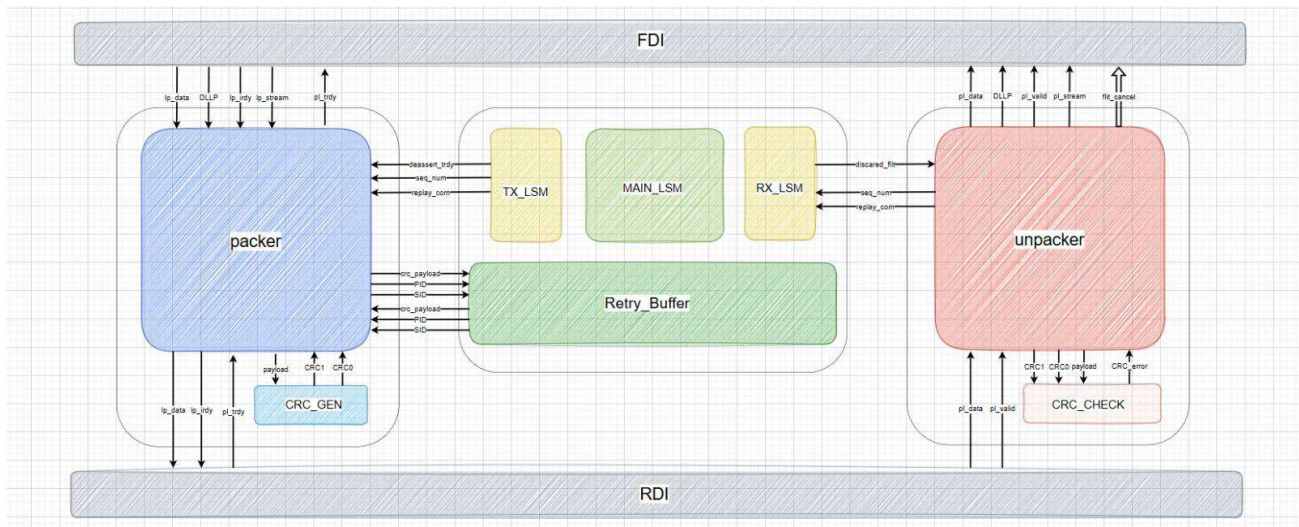


Figure 11 – Mainband block Top level

Its primary responsibilities include: Constructing and validating 256-byte flits in accordance with the UCIE protocol specification. Inserting and verifying CRC values to ensure end-to-end data integrity. Managing sequence numbers and replay commands to support the retry mechanism. Forwarding control signals and payload data across the FDI and RDI interfaces

The Mainband is composed of the following sub-blocks:

Sub-Block	Function
Packer	Assembles outgoing flits from protocol payload and control fields.
Retry Buffer	Stores transmitted flits for potential replay in the event of a detected error.
CRC Generator	Computes CRC0 and CRC1 over the assembled flit for error protection.
Unpacker	Decomposes incoming flits into payload, control, and diagnostic fields.
CRC Check	Recomputes and verifies CRC values on received flits to detect corruption.

Table 4 Mainband block sub-blocks functionality

3.3.2 TX Path

In the transmit direction, protocol payload enters through the FDI interface and is processed by the Packer. The CRC Generator computes integrity values over the assembled flit, which are then appended before transmission via the RDI interface.

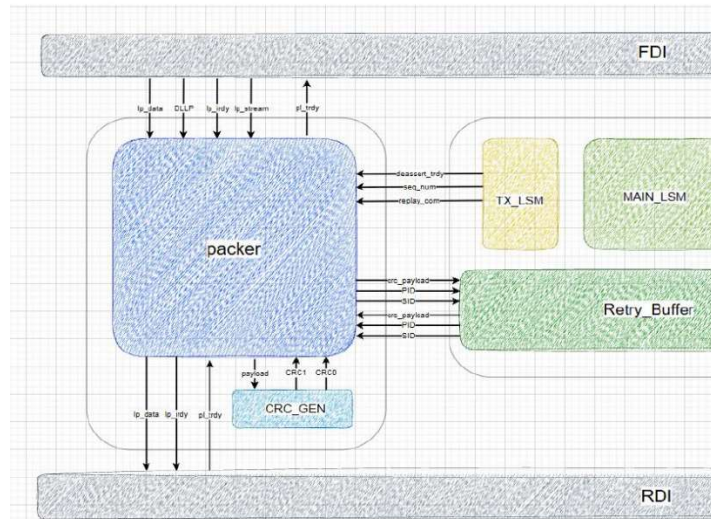


Figure 12 – TX block Path

3.3.3 RX Path

In the receive direction, incoming flit data arrives via the RDI interface and is processed by the Unpacker. The CRC Check block validates data integrity by recomputing and comparing CRC values.

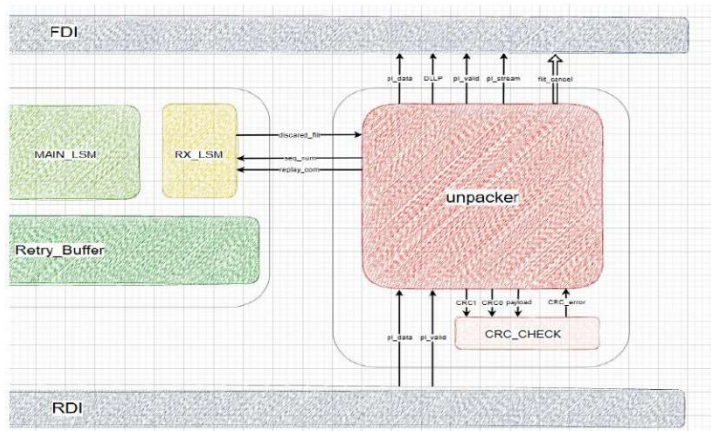


Figure 13 – RX block Path

3.4 Packer

3.4.1 Overview

The Packer operates exclusively in the transmit path and is responsible for constructing outgoing flits. It receives payload data from the Protocol Layer through the FDI interface and prepares complete flit structures for transmission.

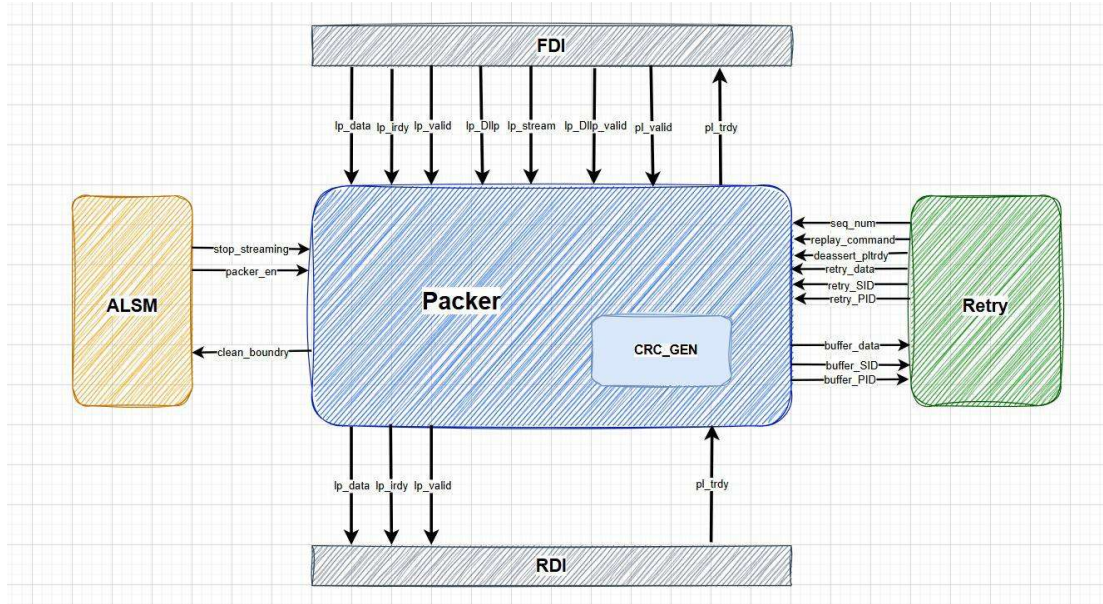


Figure 14 – Packer Block Top level

The Packer assembles the flit header using control metadata including the Protocol ID, Stack ID, sequence number, replay command, and flit type. The resulting header is combined with the payload data and any required control fields to form a complete flit, prior to CRC insertion by the CRC Generator.

By enforcing correct format and byte alignment, the Packer ensures protocol compliance and enables reliable, deterministic transmission behavior.

3.4.2 Signal Description

Port	Direction	Width	Connected	Description
Lp_irdy_fdi	IN	1	FDI	Indicates that the protocol layer is ready to send data.
Lp_valid_fdi	IN	1	FDI	Indicates that data from FDI is valid.
Lp_data_fdi	IN	512	FDI	Payload data received from the Protocol Layer via FDI.
Lp_dllp	IN	32	FDI	DLLP or optimized flow control information from FDI.

Lp_dllp_valid	IN	1	FDI	Indicates that the DLLP data is valid.
Lp_dllp_ofc	IN	1	FDI	Indicates that the DLLP carries optimized flow control info.
Lp_stream	IN	8	FDI	Contains Protocol ID and Stack ID information.
Pl_trdy	IN	1	RDI	Indicates that the Adapter is ready to accept a flit.
Packer_en	IN	1	LSM	Enables the packer operation in the TX path.
Stop_stream	IN	1	LSM	Stops data transmission from the protocol layer.
Seq_num	IN	8	Retry	Sequence number assigned to the current flit.
Replay_command	IN	2	Retry	Indicates replay or retry control command.
Deassert_trdy	IN	1	Retry	Requests deassertion of ready during retry handling.
Retry_data	IN	512	Retry	Payload data retrieved from retry buffer during replay.
Retry_SID	IN	1	Retry	Stack ID retrieved from retry buffer.
Retry_PID	IN	2	Retry	Protocol ID retrieved from retry buffer.
Pl_trdy_fdi	OUT	1	FDI	Indicates that the PHY is ready to receive data from Adapter.
Clean_boundry	OUT	1	LSM	Indicates a clean flit boundary after transmission.
Lp_data_rdi	OUT	512	RDI	Complete flit data sent to the RDI interface.
Lp_valid_rdi	OUT	1	RDI	Indicates that the flit data sent to RDI is valid.
Lp_irdy_rdi	OUT	1	RDI	Indicates that the packer is ready to transmit to RDI.
Buffer_data	OUT	512	Retry	Payload data stored for possible retry operation.
Buffer_PID	OUT	1	Retry	Protocol ID stored in the retry buffer.
Buffer_SID	OUT	1	Retry	Stack ID stored in the retry buffer.

Table 5 - Signal Description of Packer block in Mainband Block

3.4.3 TX Path Operation

The TX path begins by waiting for an enable assertion from the LSM. Once enabled, the Mainband asserts pl_trdy and waits until both lp_irdy and lp_valid are simultaneously asserted, completing the handshake. Upon handshake completion, lp_data is received from the FDI interface and stored in an internal buffer. In parallel, lp_stream and lp_dllp signals are captured in dedicated buffers. The payload data, Protocol ID, and Stack ID are forwarded to the Retry block for tracking and potential replay handling.

The Mainband then receives the sequence number and replay command from the Retry block, at which point it constructs the Flit Header. During a retry, payload and IDs are retrieved from the Retry block while DLLP information continues to be sourced from FDI. The complete flit — header, payload, and CRC — is then transmitted to the RDI interface.

3.5 CRC Generator

3.5.1 Overview

The CRC Generator protects transmitted data against corruption by computing two independent CRC values — CRC0 and CRC1 — over fixed, non-overlapping windows of the flit.

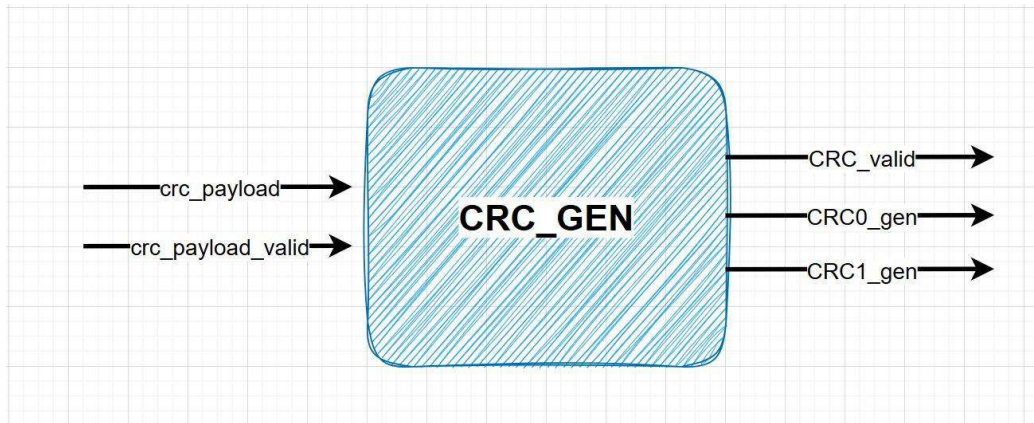


Figure 15 – CRC Gen sub-block Top Level

It receives the partially assembled flit from the Packer and includes the flit header, payload data, and any inserted DLLP bytes in its computation, while explicitly excluding the CRC fields themselves. The computed values are returned to the Packer for insertion into the final flit.

3.5.2 Signal Description

Port	Direction	Width	Connected	Description
CLK	IN	1	—	Clock input.
RST	IN	1	—	Reset input.
crc_payload_valid	IN	1	Packer	Indicates valid data for CRC computation.
crc_payload	IN	512	Packer	Flit data excluding CRC fields, used for CRC calculation.
crc0_gen	OUT	16	Packer	CRC value calculated for the first 128 bytes of the flit.
crc1_gen	OUT	16	Packer	CRC value calculated for the second CRC window.
crc_valid	OUT	1	Packer	Indicates that CRC calculation is complete and valid.

Table 6 - Signal Description of CRC Gen block in Mainband block

3.5.3 CRC-16 Architecture (64 Bytes per Clock)

The CRC block implements a 16-bit Cyclic Redundancy Check using the generator polynomial $x^{16} + x^{15} + x^2 + 1$ (0x1021). The design processes 64 bytes (512 bits) per clock cycle, making it well-suited for UC1e 256-byte flit processing.

Rather than processing one bit per cycle, this design unrolls the LFSR bit-by-bit logic into a single combinational function that processes all 512 bits in parallel. During synthesis, this produces a large parallel XOR network capable of updating the CRC register in a single clock cycle. Since each cycle handles 64 bytes, a full 256-byte flit requires four clock cycles to complete CRC computation.

3.5.4 CRC Generator Operation

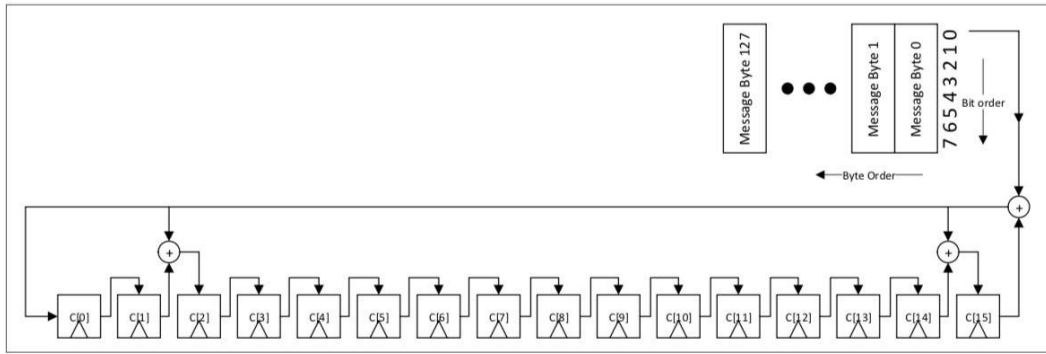


Figure 16 – CRC Computation

CRC generation proceeds immediately upon receiving data from the Packer. CRC0 covers the first 128 bytes of the flit; CRC1 covers the remaining 114 bytes of valid flit data. In both windows, the size is fixed and unused bytes are treated as zero. Upon completion, CRC0 and CRC1 are forwarded to the Packer and `crc_valid` is asserted.

3.6 TX Operation

3.6.1 TX Path Pipeline

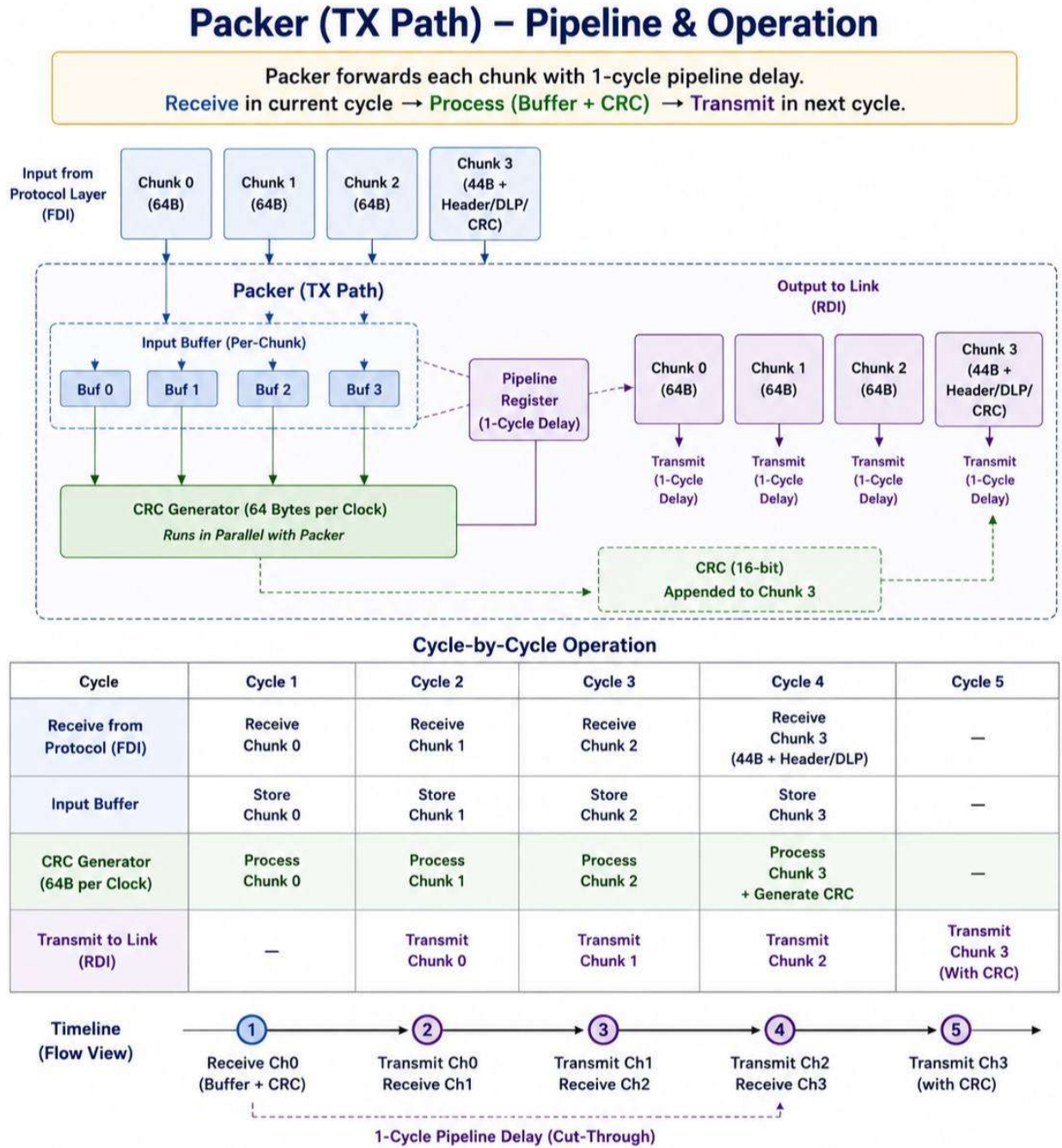


Figure 17 – TX Path Pipeline in Mainband

3.6.2 TX Flow Chart

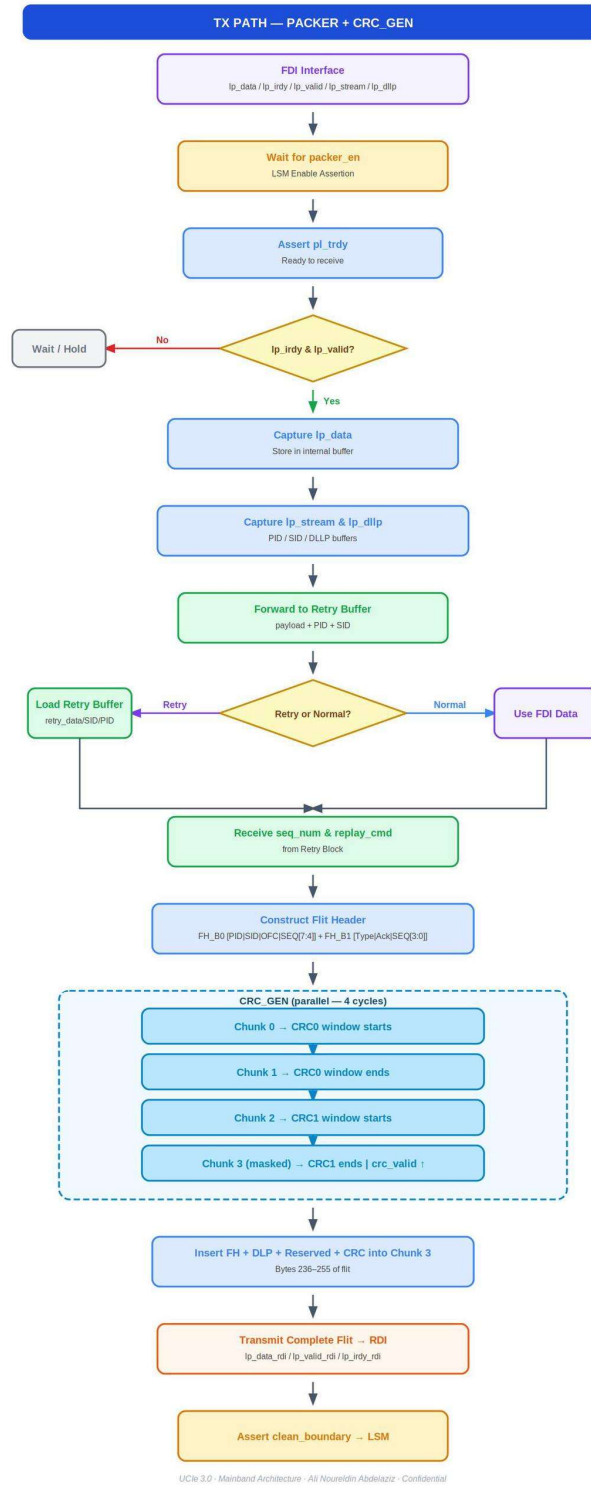


Figure 18 – TX Flow Chart in Mainband

3.7 Unpacker

3.7.1 Overview

The Unpacker operates in the receive path and processes incoming flits from the RDI interface. It buffers received data and decomposes each flit into its constituent components for independent handling.

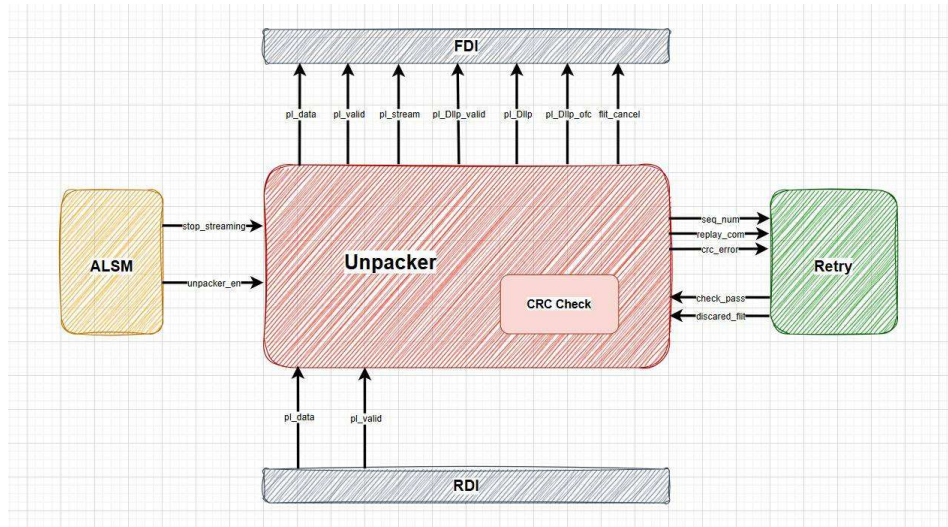


Figure 19 – Unpacker block Top level in Mainband

Extracted fields include the payload data, Protocol ID, Stack ID, DLLP information, sequence number, and replay command. By cleanly separating data and control information, the Unpacker enables correct protocol-layer interpretation and ensures valid payload delivery via the FDI interface.

3.7.2 Signal Description

Port	Direction	Width	Connected	Description
Pl_datd_rdi	IN	512	RDI	Full received flit data including header, payload, DLLP, and CRC.
Pl_valid_rdi	IN	1	RDI	Indicates that incoming flit data is valid.
Check_pass	IN	1	Retry	Asserted by Retry block when sequence and replay checks pass.
Discardd_flit	IN	1	Retry	Indicates that the current flit must be discarded due to retry failure.
Stop_stream	IN	1	LSM	Signal from LSM to stop unpacking and data forwarding.
Unpacker_en	IN	1	LSM	Enables the Unpacker to start processing incoming flits.
Pl_data_fdi	OUT	512	FDI	Valid payload data forwarded to the FDI interface.
Pl_valid_fdi	OUT	1	FDI	Indicates that the output payload data to FDI is valid.
Pl_stream	OUT	8	FDI	Carries Protocol ID and Stack ID information to FDI.

Pl_dllp	OUT	32	FDI	Extracted DLLP data forwarded to the FDI interface.
Pl_dllp_valid	OUT	1	FDI	Indicates that the DLLP data is valid.
Pl_dllp_ofc	OUT	1	FDI	Indicates that the DLLP carries optimized flow control information.
Flit_cancel	OUT	1	FDI	On CRC or sequence error, instructs FDI to discard the forwarded payload.
Seq_num	OUT	8	Retry	Sequence number extracted from the flit header, sent to Retry block.
Replay_com	OUT	2	Retry	Replay command extracted from the flit header for Retry handling.
Crc_err	OUT	1	Retry	Asserted when a CRC mismatch is detected in the received flit.

Table 7 - Signal Description of unpacker block in mainband block

3.7.3 RX Path Operation

When pl_valid is asserted and the Unpacker is enabled, it begins receiving pl_data from the RDI interface. The received data is stored in an internal buffer and decomposed into its constituent fields. The Stack ID and Protocol ID are extracted and driven on pl_stream; DLLP information is forwarded on pl_dllp; sequence number and replay command are sent to the Retry block.

CRC checking is initiated in parallel by recomputing CRC values over the received flit, excluding the CRC fields. If flit_cancel is asserted, FDI discards the forwarded payload.

3.7.4 Flit Cancel Mechanism

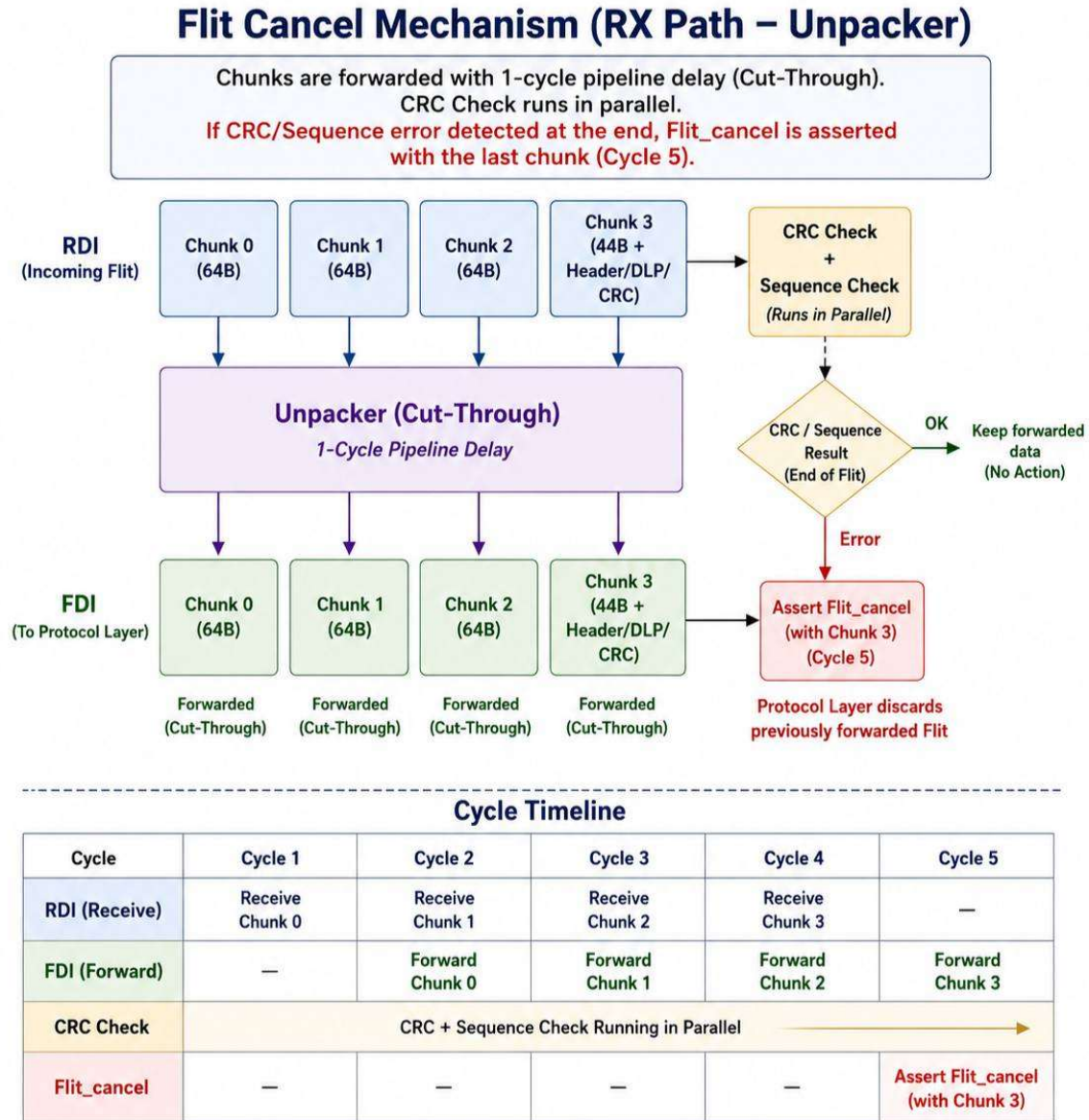


Figure 20 – Flit Cancel Mechanism

To minimize end-to-end latency, the receiver employs a cut-through forwarding architecture. Each 64-byte chunk is forwarded to the upper layer as soon as it is received (with a one-cycle pipeline delay), while the CRC engine and retry sequence checker operate in parallel.

The final validation results become available immediately after the last chunk is processed. If both checks pass, the flit is accepted. If a CRC error or sequence error is detected, a Flit Cancel signal is transmitted, instructing the receiving protocol layer to invalidate and discard the previously forwarded flit data.

3.8 CRC Check

3.8.1 Overview

The CRC Check block verifies the integrity of received flits by recomputing CRC0 and CRC1 using the same fixed windows applied during transmission and comparing them against the embedded CRC values.

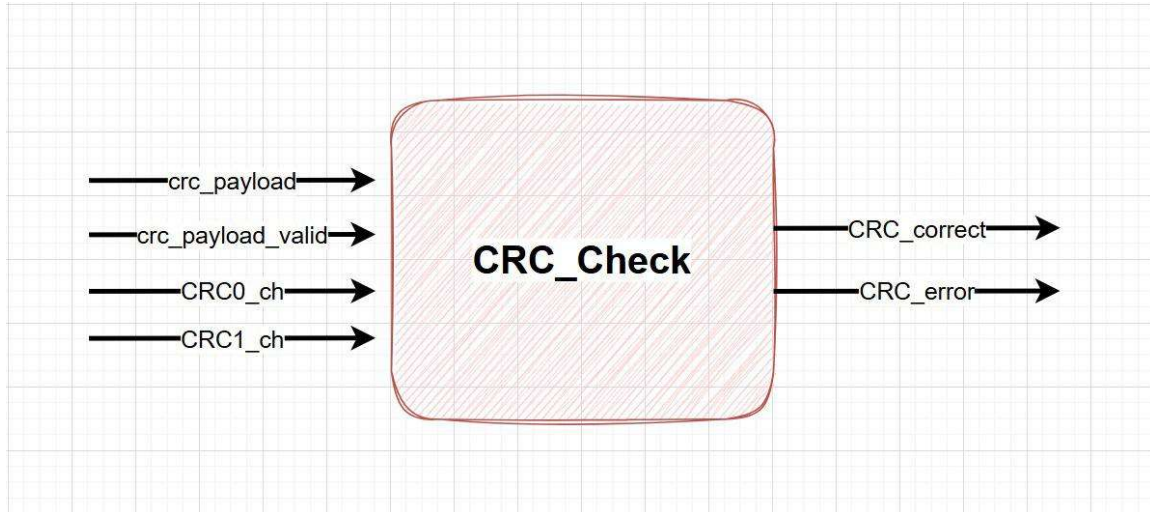


Figure 21 – CRC Check Block in Mainband Block

A successful match results in assertion of `crc_correct`. A mismatch causes the block to assert `crc_error`, preventing corrupted data from propagating and triggering the retry and recovery mechanisms.

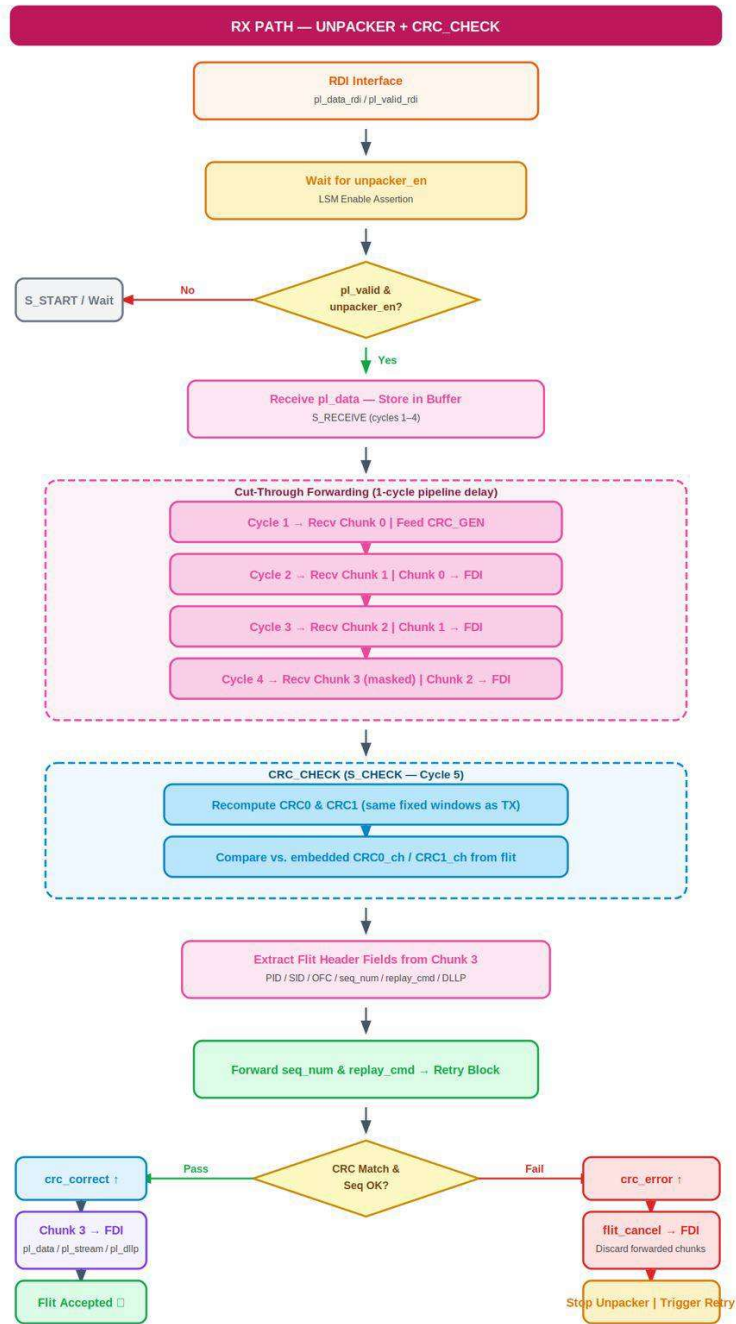
3.8.2 Signal Description

Port	Direction	Width	Connected	Description
CLK	IN	1	—	Clock input.
RST	IN	1	—	Reset input.
crc_payload	IN	512	Unpacker	Flit data excluding CRC fields, used as input to CRC calculation.
crc_payload_valid	IN	1	Unpacker	Indicates that the received flit data is valid for CRC checking.
crc0_ch	IN	16	Unpacker	CRC0 value received within the flit.
crc1_ch	IN	16	Unpacker	CRC1 value received within the flit.
crc_correct	OUT	1	Unpacker	Asserted when calculated CRC matches received CRC values.
crc_err	OUT	1	Unpacker	Asserted when a CRC mismatch is detected.

Table 8 - Signal Description of CRC Check in Mainband block

3.9 RX Operation

3.9.1 RX Flow Chart



UCle 3.0 - Mainband Architecture - Ali Noureldin Abdelaziz - Confidential

Figure 22 – RX Flow Chart

3.9.2 CRC Check Operation

Incoming data is processed to recompute CRC0 and CRC1 over the same fixed windows used during transmission. The received CRC0 and CRC1 values are supplied concurrently by the Unpacker and compared against the recomputed results. If both values match, `crc_correct` is asserted, confirming the flit was received without corruption. If a mismatch is detected, `crc_error` is forwarded to the Retry block and a stop signal is asserted to the Unpacker, halting further data forwarding until the error condition is resolved.

3.10 Implementation

3.10.1 Overview

The Mainband block is the core of the UCle 3.0 Adapter Layer, responsible for flit-level data processing in both TX and RX paths. It sits between the Protocol Layer (FDI interface) and the Physical Layer (RDI interface).

Block	Path	Function
Packer	TX	Assembles 256B flits, computes CRC, sends to RDI
Unpacker	RX	Receives flits from RDI, checks CRC, forwards to FDI
CRC Generator	TX + RX	16-bit CRC computation (LFSR-based, 512 bits/cycle)

Table 9 – blocks functionality

A complete 256-byte flit is laid out across four 64-byte chunks. The final chunk carries the protocol payload, flit header, DLP bytes, reserved region, and two 16-bit CRC values.

3.10.2 Byte Map

Region	Byte RSange	Size	Description
Chunk 0	0 – 63	64 B	Payload from Protocol Layer
Chunk 1	64 – 127	64 B	Payload from Protocol Layer
Chunk 2	128 – 191	64 B	Payload from Protocol Layer
Chunk 3 Payload	192 – 235	44 B	Remaining protocol payload
FH B0 / B1	236 – 237	2 B	Flit Header bytes
DLP B2–B5	238 – 241	4 B	DLLP / Optimized Update Marker
Reserved	242 – 251	10 B	Always zero; reserved for future use
CRC0	252 – 253	2 B	16-bit CRC over first 128 bytes
CRC1	254 – 255	2 B	16-bit CRC over second window

Table 10 – byte mapping

3.10.3 Chunk 3 Bit-Field Offsets (within 512-bit chunk)

Field	Bits	Size
Payload	[351:0]	44 B
FH Byte 0	[359:352]	1 B
FH Byte 1	[367:360]	1 B
DLP B2–B5	[399:368]	4 B
Reserved	[479:400]	10 B
CRC0	[495:480]	2 B

CRC1	[511:496]	2 B
------	-----------	-----

Table 11 – Chunk 3 Bit-Field Offsets in format 3

3.10.4 Flit Header Encoding

Byte	Bits	Field	Description
FH_B0	[7:6]	PID	Protocol Identifier
FH_B0	[5]	SID	Stack Identifier
FH_B0	[4]	OFC	Optimized Flow Control flag
FH_B0	[3:0]	SEQ[7:4]	Upper 4 bits of sequence number
FH_B1	[7:6]	Flit_Type	00b = normal flit
FH_B1	[5:4]	Ack/Nak	Retry acknowledgment / negative-ack
FH_B1	[3:0]	SEQ[3:0]	Lower 4 bits of sequence number

Table 12 – flit header encoding in format 3

The CRC Generator computes two independent 16-bit CRC values — CRC0 and CRC1 — over fixed, nonoverlapping windows of the 256-byte flit. The polynomial used is $x^{16} + x^{15} + x^2 + 1$ (0x1021).

3.10.5 Architecture & Timing

LFSR-based implementation, processes 512 bits (64 bytes) per clock cycle.

Full 256-byte flit requires four clock cycles to complete CRC computation.

CRC fields are zeroed before being fed to the generator (excluded from computation).

CRC0 covers the first 128 bytes; CRC1 covers the second 128-byte window.

3.10.6 Cycle-by-Cycle Timing

Cycle	State	Action
1	S_CHUNK0	Receive / process Chunk 0 — begin CRC0 computation
2	S_CHUNK1	Receive / process Chunk 1 — CRC0 window complete
3	S_CHUNK2	Receive / process Chunk 2 — begin CRC1 computation
4	S_CHUNK3	Receive / process Chunk 3 — assert o_crc_valid

Table 13 – Cycle Timing in format 3

The Packer operates exclusively in the transmit path, assembling 256-byte flits from protocol-layer payload chunks. It feeds the CRC Generator in parallel and delivers completed flits to the RDI interface with a total pipeline latency of five clock cycles.

3.10.7 Pipeline Timing

Cycle	Action
1	Receive Chunk 0 from FDI; feed to CRC Generator
2	Receive Chunk 1; feed CRC Gen Chunk 0 → RDI
3	Receive Chunk 2; feed CRC Gen Chunk 1 → RDI
4	Receive Chunk 3 (masked); feed CRC Gen Chunk 2 → RDI
5	CRC ready — insert FH + DLP + Reserved + CRC into Chunk 3; Chunk 3 → RDI

Table 14 – Pipeline Timing in format 3

3.10.8 FSM State Descriptions TX

State	Description
S_IDLE	Wait for packer_en assertion from LSM
S_COLLECT	Cycles 1–4: receive chunks, feed CRC, pipeline send to RDI
S_INSERT	Cycle 5: insert FH, DLP, reserved, CRC into Chunk 3; send to RDI
S_DONE	Check pending commands; prepare next flit
S_FLIT_BOUNDARY	Assert flit_boundary_done; return to S_IDLE
S_DRAIN	Send retry buffer with replay command until buffer is empty
S_FLUSH	Send retry buffer with replay_cmd forced to 00

Table 15 – FSM states in TX in mainband

3.10.9 Data Source per Mode

Mode	Data Source	replay_cmd
Normal	FDI	Real value from LSM
Retry	Retry buffer	Real value
Drain	Retry buffer	Real value

Table 16 – Data source per mode in TX block in mainband

3.10.10 Pipeline Timing

Cycle	Action
1	Receive Chunk 0; feed to CRC Generator
2	Receive Chunk 1; feed CRC Gen Chunk 0 → FDI
3	Receive Chunk 2; feed CRC Gen Chunk 1 → FDI
4	Receive Chunk 3 (masked); feed CRC Gen Chunk 2 → FDI
5	S_CHECK — CRC compare + forward Chunk 3 to FDI + assert flit_cancel if error

Table 17 – Pipeline Timing in TX block in mainband

3.10.11FSM State Descriptions RX

State	Description
S_START	Wait for unpacker_en assertion from LSM
S_RECEIVE	Cycles 1–4: receive chunks, feed CRC Generator, cut-through forward to FDI
S_CHECK	Cycle 5: CRC comparison; forward Chunk 3; assert flit_cancel on error

Table 18 – FSM states in RX block in mainband

3.10.12 Chunk 3 Field Extraction

Register	Source Bits	Field Extracted
r_pid	FH_B0[7:6]	Protocol Identifier (PID)
r_sid	FH_B0[5]	Stack Identifier (SID)
r_dllp_ofc	FH_B0[4]	Optimized Flow Control flag
r_replay_cmd	FH_B1[5:4]	Ack / Nak replay command
r_seq_num	{FH_B0[3:0], FH_B1[3:0]}	Full 8-bit sequence number
r_dllp_buf	i_pl_data_rdi[C3_DLP +: 32]	Extracted DLLP bytes
r_crc0_ch	i_pl_data_rdi[C3_CRC0 +: 16]	Received CRC0
r_crc1_ch	i_pl_data_rdi[C3_CRC1 +: 16]	Received CRC1

Table 19 – Chunk 3 Field Extraction

3.10.13Formation Conventions

Prefix	Meaning
r_	Register (flip-flop, sequential logic)
w	Wire (combinational logic)
i_	Input port
o_	Output port

Table 20 - formation Conventions

3.10.14Modules Description

File	Description
UC_MB_Mainband_pkg.sv	Package containing all parameters and typedefs
UC_MB_CRC_Generator.sv	16-bit CRC computation block
UC_MB_Packer.sv	TX path — flit assembly and header insertion
UC_MB_Unpacker.sv	RX path — flit decomposition and CRC verification

UC_MB_Mainband.sv	Top-level module connecting Packer and Unpacker
-------------------	---

Table 21 – Description of mainband system RTL modules

3.11 Results

3.11.1 TX

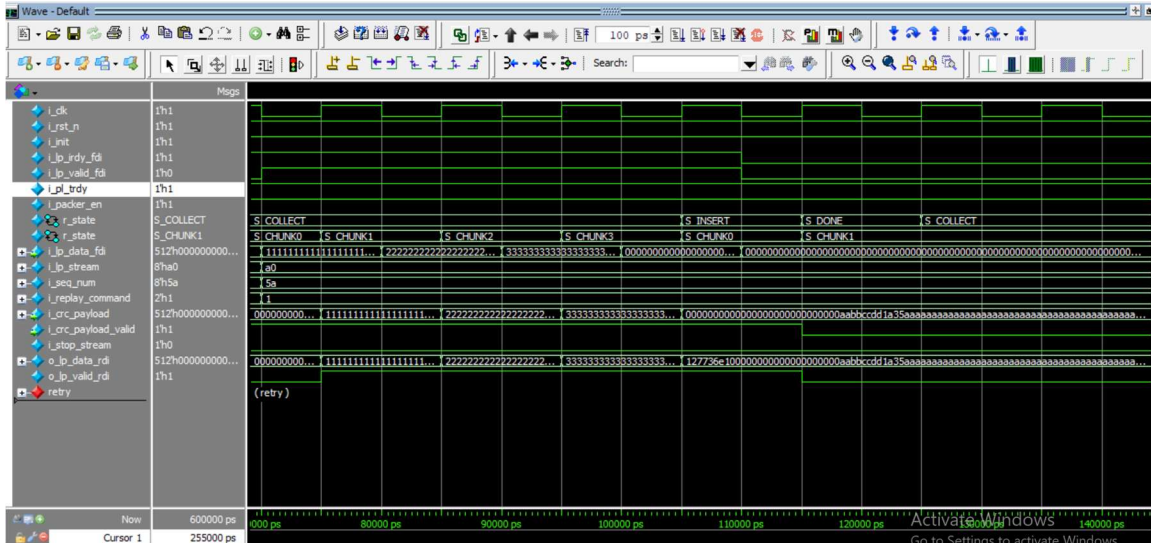


Figure 24 – waveform of mainband top module

```

Transcript
VSIM 47> run -all
# INITIALIZE DONE
# RESET DONE
# RETRY TX DONE
# NORMAL TX DONE
# NORMAL TX DONE
# ** Note: $stop : E:/questasim64_2021.1/examples/Course/GP/Packer/UC_MB_Packer_tb.sv(369)
# Time: 790 ns Iteration: 1 Instance: /UC_MB_Packer_tb
# Break in Module UC_MB_Packer_tb at E:/questasim64_2021.1/examples/Course/GP/Packer/UC_MB_Packer_tb.sv line 369
# Compile of UC_MB_CRC_Generator.sv was successful with warnings.
# Compile of UC_MB_Mainband_pkg.sv was successful with warnings.
# Compile of UC_MB_Packer.sv was successful with warnings.
# Compile of UC_MB_Packer_tb.sv was successful with warnings.
# 4 compiles, 0 failed with no errors.

```

Figure 23 – transcript of TX of mainband top module

3.11.2 RX

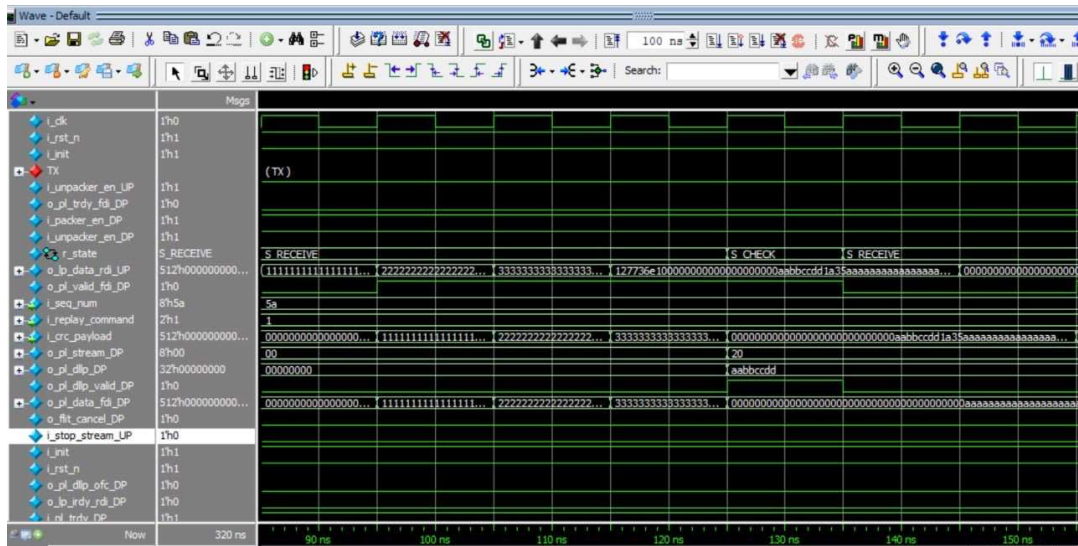


Figure 26 – waveform of RX of mainband top module

```

Transcript
File Edit View Bookmarks Window Help
Transcript
v$im(paused)> run -all
# RESET DONE
# INITIALIZATION DONE
# NORMAL RX DONE
# DATA RECEIVED CORRECT
# HEADER EXTRACTION DONE
# DLLP EXTRACTION DONE
# CRC CHECK PASS
** Note: $stop      : E:/questasim64_2021.1/examples/Course/GP/MainBand/UC_MB_Top_tb.sv(226)
# Time: 320 ns Iteration: 1 Instance: /UC_MB_Top_tb
# Break in NamedBeginStat main at E:/questasim64_2021.1/examples/Course/GP/MainBand/UC_MB_Top_tb.sv line 226
# Compile of UC_MB_CRC_Generator.sv was successful.
# Compile of UC_MB_Mainband_pkg.sv was successful.
# Compile of UC_MB_Mainband_Top.sv was successful.
# Compile of UC_MB_Packer.sv was successful.
# Compile of UC_MB_Top_tb.sv was successful.
# Compile of UC_MB_Unpacker.sv was successful.
# 6 compiles, 0 failed with no errors.

```

Figure 25 – transcript of RX of mainband top module

3.11.3 Mainband After Integration

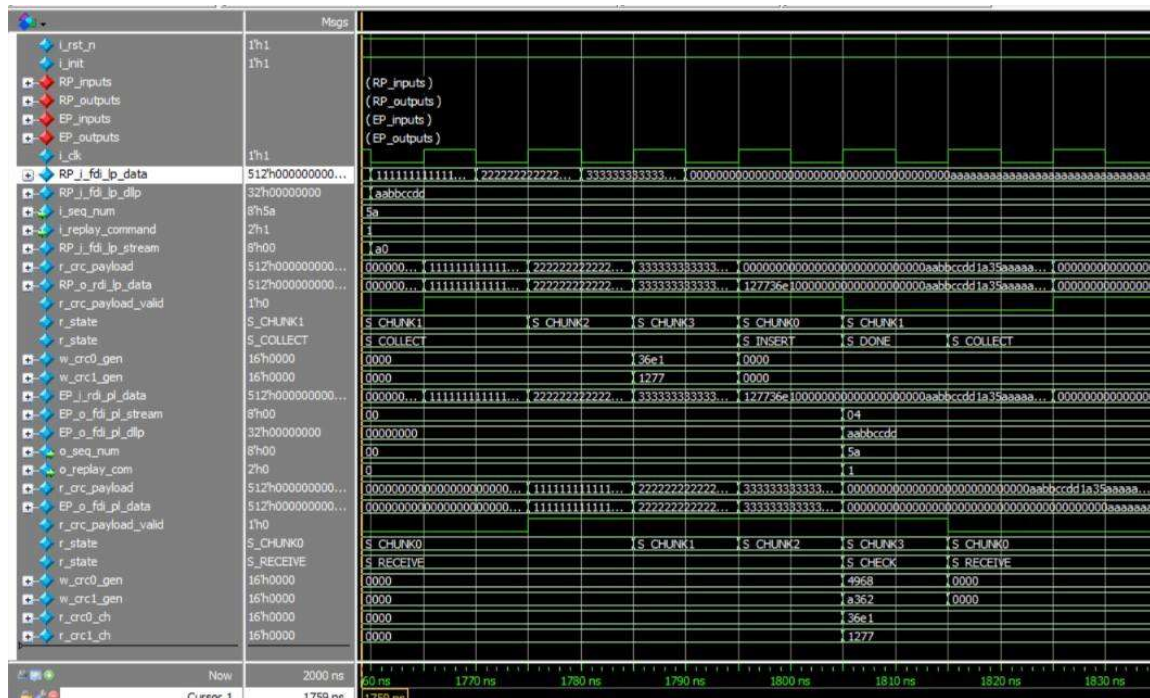


Figure 27 – mainband Top level Module waveform

3.12 Overview of Retry Block

The UCle retry mechanism is a Link-level reliability protocol implemented within the Die-to-Die (D2D) Adapter. Its purpose is to ensure valid data delivery across the physical link by detecting errors (using CRC) and re-transmitting corrupted data (Replay). The UCle retry block utilizes a simplified version of the standard PCIe Flit Mode Retry mechanism, heavily optimized for the low-latency requirements of chiplet-to-chiplet communication. Support for Retry is advertised and enabled during Stage 3 of Link Initialization (Adapter Parameter Exchange). Retry rules use sliding window and go back in mechanism. Retry mechanism has mainly 2 sub-phases: normal exchange phase, sequence number handshake. the retry block is housed within the Die-to-Die (D2D) Adapter and integrates with the mainband—the primary high-speed physical data path—by orchestrating the flow, formatting, and reliability of Flits before they cross the Raw Die-to-Die Interface (RDI).

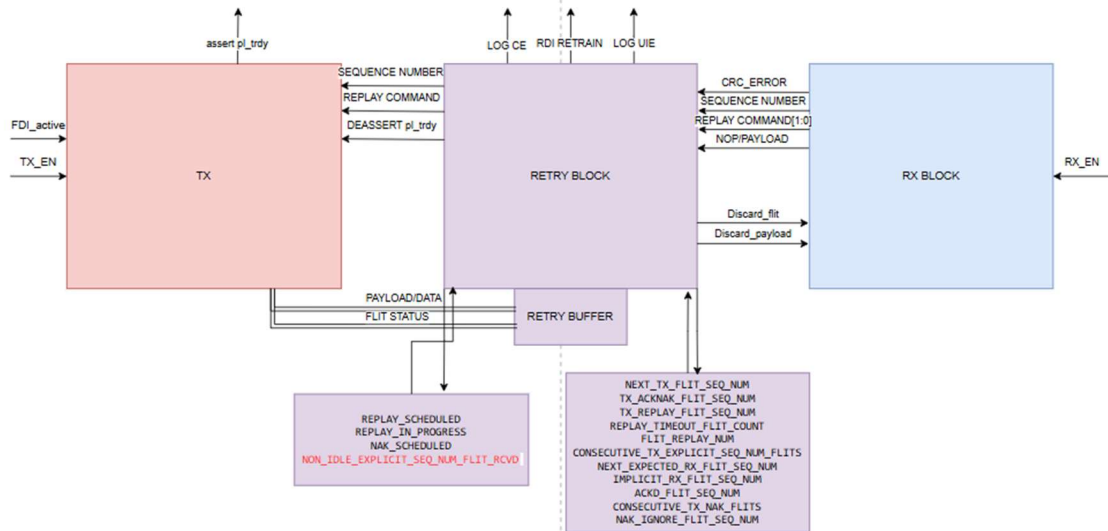


Figure 28 – Retry block in mainband

As the retry rules are from PCIe protocol but modified to fit the UCle specs, here is what changed to implement based on rules:

Flit Sequence Number	10-bit (1–1023)	8-bit (1–255)
Modular Arithmetic	mod 1023	mod 255
Max Unacknowledged Flits	≤ 511	127
Explicit/ACK Alternation	3 explicit + 1 ACK/NAK	1 explicit + 1 ACK/NAK
SNH Init Sequence Number	10'h3FF	8'hFF
Data Rate Categories	64 GT/s, ≤32 GT/s	32 GT/s, 16 GT/s
Types of replaying of data	Selective Replay and Standard Replay	Only Standard Replay

Table 22 – UCle simplified version of Retry mechanism

3.12.1 Retry External Interfaces

Signal name	Direction	Source / Dest
clk	in	External clock
rst_n	in	External reset
init	in	External software reset
fdi_active	in	External system control
tx_en	in	External system control
rx_en	in	External system control
data_rate	in	External system control
rx_flit_valid	in	External RX boundary pulse
tx_flit_valid	in	External TX boundary pulse
flush	in	External buffer flush control
drain	in	External buffer drain control
rx_crc_error	in	External RX mainband receiver
rx_seq_num[7:0]	in	External RX mainband receiver
rx_replay_command	in	External RX mainband receiver
rx_flit_type	in	External RX mainband receiver
tx_i_data[DATA_WIDTH-1:0]	in	External TX buffer input
tx_i_stream[STREAM_WIDTH-1:0]	in	External TX buffer input
tx_o_data[DATA_WIDTH-1:0]	out	External TX buffer output
tx_o_stream[STREAM_WIDTH-1:0]	out	External TX buffer output
buffer_state	out	Internal buffer status from u_buffer
pl_trdy_control	out	To transmitter, from u_transmitter_rules
tx_replay_command	out	To transmitter, from u_transmitter_rules
tx_seq_num[7:0]	out	To transmitter, from u_transmitter_rules
retry_in_use	out	Retry active status from u_buffer
discard_flit	out	To error/status handling, from u_receiver_rules
discard_payload	out	To error/status handling, from u_receiver_rules
log_uie	out	To error/status handling, OR of log_uie_a and log_uie_b
log_cie	out	To error/status handling, OR of log_cie_a and log_cie_c
rdr_retrain	out	To error/status handling, from u_transmitter_rules

Table 23 – Retry TOP interface

3.12.2 Retry Counters and Flags

Counter	Size	Description
next_tx_flit_seq_num	Unsigned 8 bits	tracks the Flit Sequence Number to be assigned to the next Payload Flit transmitted
tx_acknak_flit_seq_num	Unsigned 8 bit	holds the Flit Sequence Number to be included in the next Ack or Nak Flit transmitted
tx_replay_flit_seq_num	Unsigned 8 bit	holds the Flit Sequence Number of the first Flit to be re-transmitted from the Tx Retry Buffer during a replay event
replay_timeout_flit_count	Unsigned 9 bit saturate at 1FFh	tracks the number of Payload and NOP Flits sent—and idle Flit times elapsed—since the last acknowledgment
flit_replay_num	Unsigned 3 bit	tracks the number of consecutive Replay attempts initiated without making forward progress to detect a link failure condition
consecutive_tx_explicit_seq_num_flits	Unsigned 2 bit	tracks the number of consecutive Explicit Sequence Number Flits sent to enforce the UCle-specific rule requiring alternating Data and Ack/Nak Flits (limit < 1)
next_expected_rx_flit_seq_num	Unsigned 8 bit	stores the expected Flit Sequence Number for the next valid, non-IDLE, non-duplicate Flit to be received
implicit_rx_flit_seq_num	Unsigned 8 bit	tracks the implicit Flit Sequence Number assigned to currently received Flits that do not carry an explicit sequence number
ackd_flit_seq_num	Unsigned 8 bit	holds the Flit Sequence Number from the most recently received valid Ack or Nak Flit, used to purge acknowledged Flits from the Tx Retry Buffer
consecutive_tx_nak_flits	3-bit unsigned	tracks the number of consecutive Nak Flits sent by the Transmitter to ensure the required burst (typically 3) is transmitted
nak_ignore_flit_seq_num	8 bits	holds a specific Flit Sequence Number for which incoming Nak Flits should be ignored to prevent redundant replays during the Nak Ignore Window

Table 24 – Retry mechanism counters

flag	description
non_idle_explicit_seq_num_flit_rcvd	this flag indicates that a non-idle explicit sequence number flit has been received since the last exit from the reset or retrain state (UCle replaces the PCIe "idle flit handshake phase" with this condition)

replay_scheduled	this flag indicates that a flit has been scheduled to be re-transmitted from the TX retry buffer
replay_scheduled_type	always set to standard replay
replay_in_progress	this flag indicates that the transmitter is currently busy sending flits from the TX retry buffer
nak_scheduled	this flag indicates that a nak flit has been scheduled to be transmitted to the remote partner
nak_scheduled_type	always set to standard nak
nak_withdrawal_allowed	this flag is always cleared to 0b in UCle

Table 25 – Retry Mechanism flags

3.13 Precedence of Retry

The core of the retry block is the TX Retry Buffer, which temporarily stores information for every transmitted Payload Flit until the remote Link partner acknowledges its successful receipt.

When the transmitter accepts a new Transaction Layer Packet (TLP) from the Protocol Layer, it packs it into a Payload Flit, assigns it an Explicit Flit Sequence Number, and stores the TLP Bytes, Flit Status, and Sequence Number in the TX Retry Buffer.

The receiver evaluates incoming Flits using dual Cyclic Redundancy Checks (CRC). If the data is pristine, the receiver sends an Ack Flit containing the sequence number of the latest good Flit, which allows the transmitter to safely purge all older Flits from its TX Retry Buffer.

To maintain low latency and minimize buffer sizes, the UCle protocol strictly mandates that the transmitter must alternate between sending Explicit Sequence Number Flits and Ack/Nak Flits, ensuring rapid acknowledgment turnaround.

If the receiver detects a CRC error, it drops the corrupted Flit and schedules a Nak Flit.

Crucial UCle Simplification: Unlike standard PCIe, the Selective Nak and RX Retry Buffer mechanisms are strictly prohibited and must not be implemented in UCle. UCle relies entirely on the Standard Nak, which forces the transmitter to rewind and replay the entire data stream starting from the exact point of failure.

A Replay from the TX Retry Buffer can be triggered by two primary events: the receipt of a Nak Flit or a Replay Timer Timeout. **Replay from Nak:** When a valid Nak Flit is received, the transmitter sets its `REPLAY_SCHEDULED` flag, stops accepting new TLPs from the Protocol Layer, and executes a Standard Replay. It rewinds the `TX_REPLAY_FLIT_SEQ_NUM` to the Flit immediately following the last successfully acknowledged sequence number and blasts out all unacknowledged Flits directly from the TX Retry Buffer. **Replay Timeout:** The Adapter maintains a 9-bit `REPLAY_TIMEOUT_FLIT_COUNT` counter (saturating at 1FFh) to track the time since the last Ack. This counter increments every time a Payload or NOP flit is sent. If this counter reaches 375 without receiving an Ack or Nak, a Replay Timer Timeout is triggered,

automatically forcing a Standard Replay from the buffer. To prevent buffer overflow, the transmitter closely monitors the MAX_UNACKNOWLEDGED_FLITS threshold. If the number of unacknowledged Flits hits this limit, the transmitter must instantly throttle the Protocol Layer, blocking new TLPs and sending only NOP Flits to keep the link alive until an incoming Ack frees up buffer space. If consecutive Replays fail to make forward progress due to heavy noise, a 3-bit FLIT_REPLAY_NUM counter will eventually roll over. It results logging an Uncorrectable Internal Error and directly initiating a Link Retrain sequence on the Raw Die-to-Die Interface (RDI) to re-establish the connection.

3.13.1 Transmitter Rules

3.13.1.1 Sequence Number Handshake Phase

This phase is used to establish the initial sequence numbers between the two Link partners and must be performed as part of every Flit-Aware Die-to-Die Interface (FDI) Active Entry Handshake. Flit Types Used: The Transmitter is permitted to use either Payload Flits or NOP Flits to complete this handshake. Payload flits are sent after FDI reach Active. If no payload, transmitter must autonomously generate NOP Flits to complete the phase.

Flit Replay Command Rules (Priority): The Transmitter decides what to place on the wire based on the following priority

1. Explicit Sequence Number Flit: Sent if the number of consecutive Explicit Flits sent is < 1 .
2. Standard Nak Flit: Sent if nak_scheduled is 1b and conditions for sending an Explicit Flit are not met.
3. Ack Flit: Sent if neither Explicit Flits nor Nak Flits are required to be sent.

Sequence Numbering: When transmitting an ack or nak flit, the sequence number is set to the current value of tx_acknak_flit_seq_num. when transmitting an explicit sequence number flit, a non-replayed payload flit uses next_tx_flit_seq_num, a nop flit uses next_tx_flit_seq_num - 1, and a replayed payload flit uses the exact sequence number stored for it in the tx retry buffer.

phase exit conditions: the transmitter transitions into the normal flit exchange phase once it has either transmitted 3 or more ack flits with non-zero sequence numbers (after receiving an explicit flit with a non-zero number) and it has transmitted 9 or more explicit sequence number flits with non-zero sequence numbers. if 128 flits are transmitted without successfully exiting this phase, a timeout occurs and forces a link retrain.

3.13.1.2 Normal Flit Exchange Phase

Once the initial sequence numbers are synchronized, the Transmitter enters the Normal Flit Exchange Phase for standard data transfer.

Flit Replay Command Rules (Priority): The Transmitter evaluates its state to determine the next Flit to send:

1. Explicit Sequence Number flit: this takes top priority if a replay is scheduled (`replay_scheduled = 1b`) or currently in progress (`replay_in_progress = 1b`). it is also sent if the nak ignore window is active (`nak_ignore_flit_seq_num != 000h`), or if the ucie alternation rule permits it (`consecutive_tx_explicit_seq_num_flits < 1` and `consecutive_tx_nak_flits` is 0 or > 2).
2. Standard Nak Flit: sent if `nak_scheduled` is 1b.
3. Ack Flit: sent if the conditions for sending an explicit flit or a standard nak flit are not met.

Consecutive Nak Transmission: If the Transmitter begins sending a Nak Flit, it is mandated to send a minimum of three consecutive Nak Flits unless interrupted by an exit from the L0 state or a reset of the Replay Schedule. The `consecutive_tx_nak_flits` counter rigorously tracks this; it increments every time a Nak Flit is transmitted and instantly resets to 0b if any non-Nak Flit is transmitted.

Sequence Numbering: Similar to the Handshake phase, any Ack or Nak Flit transmitted during normal exchange must strictly set its sequence number to the current value held in the `tx_acknak_flit_seq_num` register.

3.13.2 Receiver Rules

3.13.2.1 ACK, NAK, Discard Schedules

The Receiver evaluates incoming Flits against its `implicit_rx_flit_seq_num` and `next_expected_rx_flit_seq_num` counters to determine how to manage the Link state and buffers: Invalid Flits: If a Flit fails validation (e.g., CRC error), the Receiver executes a Nak Schedule. It discards the corrupted Flit, sets the `nak_scheduled` flag, and prepares to transmit a Standard or Selective Nak back to the sender pointing to the last successfully received Flit. Valid Flits (Forward Progress): If a valid Payload Flit is received in the correct order, the Receiver executes an Ack Schedule. It consumes the Flit, increments the expected sequence number, and schedules an Ack Flit for transmission. Duplicate Flits: If the Receiver receives a Payload Flit with a sequence number it has already consumed, it executes a Flit Discard Schedule, safely discarding the payload to prevent data corruption while still scheduling an Ack/Nak to help the Transmitter clear its buffer.

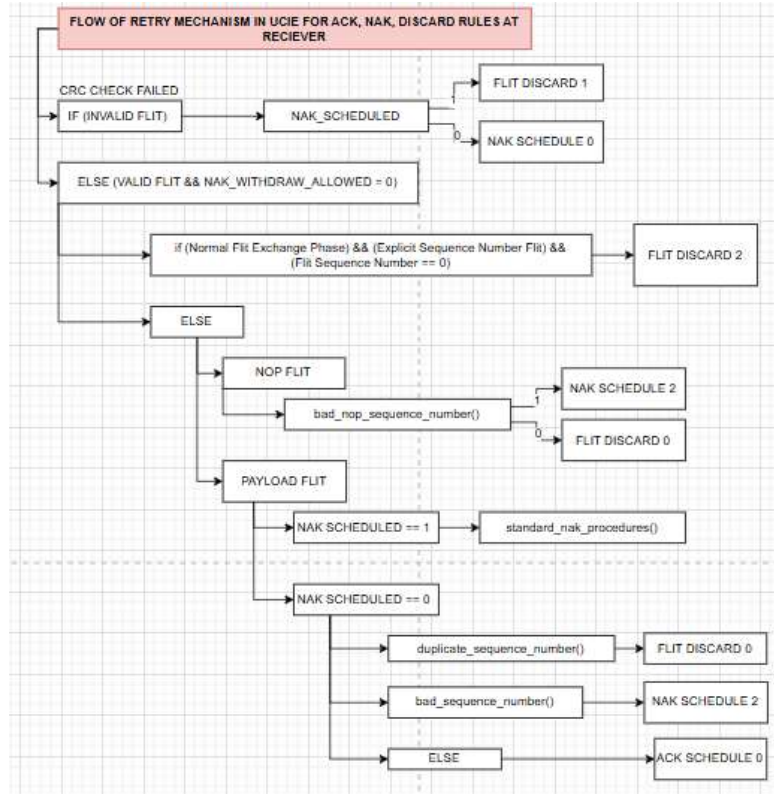


Figure 30 – Reviewer flow in Retry

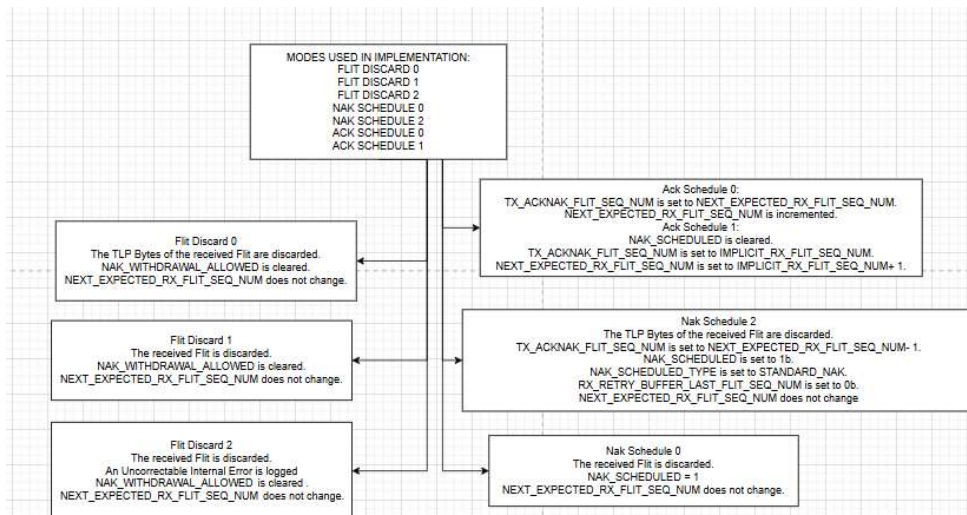


Figure 29 – Schedules implemented

3.13.2.2 Processing Received Acks and Naks

When the Receiver captures an Ack or Nak Flit originating from the remote Link partner, it must validate the sequence number and clear its own Transmit (TX) Retry Buffer: **Zero Sequence Numbers:** Any received Ack or Nak Flit carrying a Flit Sequence Number of 0 is strictly ignored. **Out-of-Bounds Sequence Numbers:** If an Ack or Nak Flit carries a Sequence Number that falls outside the maximum unacknowledged Flits window, the Receiver ignores it and logs a uncorrectable internal error. **Buffer Purging:** if a valid ack or nak flit is received with a valid sequence number n , the receiver must immediately purge all flits stored in its tx retry buffer that are older than and including the flit with sequence number n . **Forward Progress Validation:** if received sequence number indicates forward progress, the receiver clears its `flit_replay_num` error counter and updates its last acknowledged sequence number tracker. **Replay Trigger:** if the received flit is a nak, the receiver sets its `nak_ignore_flit_seq_num` appropriately and immediately schedules a replay sequence from its TX retry buffer to recover the link.

3.13.3 Replay Schedule and Transmit

The Transmitter continuously monitors its state and incoming acknowledgments to determine if a Replay from the TX Retry Buffer must be scheduled based on two primary rules: **Replay Schedule Rule 0: Replay Timeout** This rule triggers a Replay if the Transmitter has not received an acknowledgment within the permitted timeframe. **Replay Schedule Rule 1: Received Nak for Payload Flit** This rule triggers a Replay when the remote Link partner explicitly indicates a corrupted Flit was received.

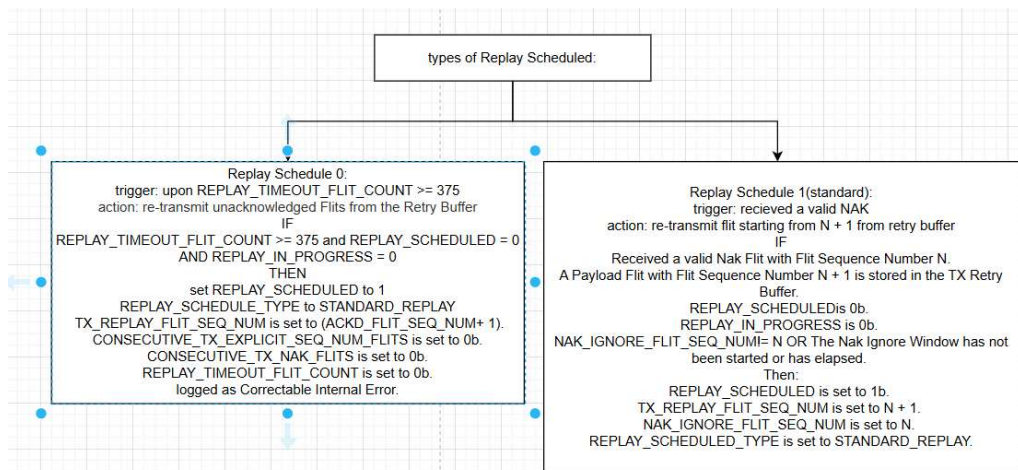


Figure 31 – Replay types

Once a Replay is scheduled (`replay_scheduled = 1b`), the Transmitter evaluates the following rules to execute the physical re-transmission of the Flits onto the wire according to these rules: **Flit Replay Transmit Rule 0: Standard Replay Scenario** This rule governs

the standard execution of flushing the TX Retry Buffer back onto the wire. Flit Replay Transmit Rule 2: Replay Rollover Scenario This rule handles fatal error escalation if the Link is so noisy that consecutive Replays fail to make forward progress.

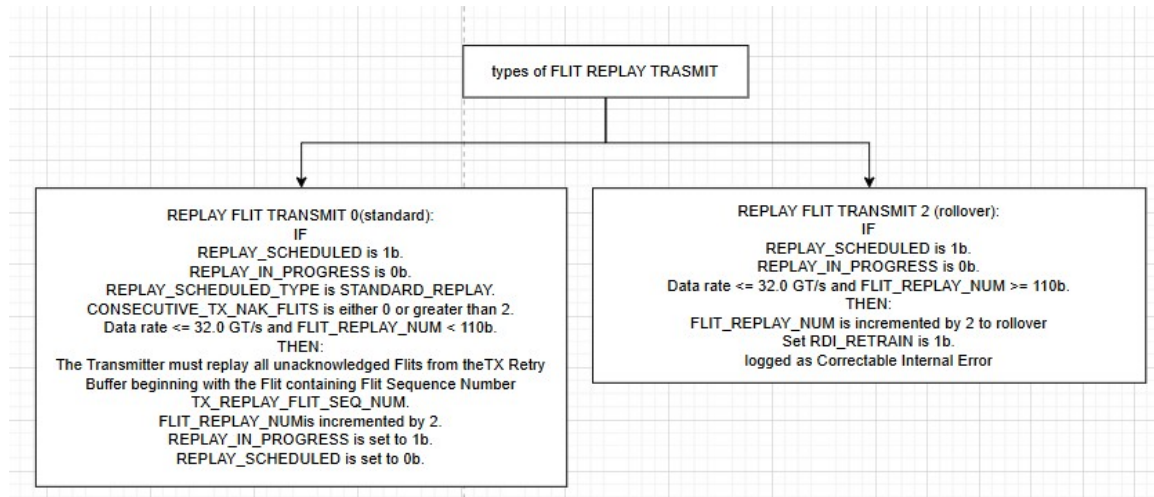


Figure 32 – Flit replay transmit implemented

3.13.4 Retry Internal Block

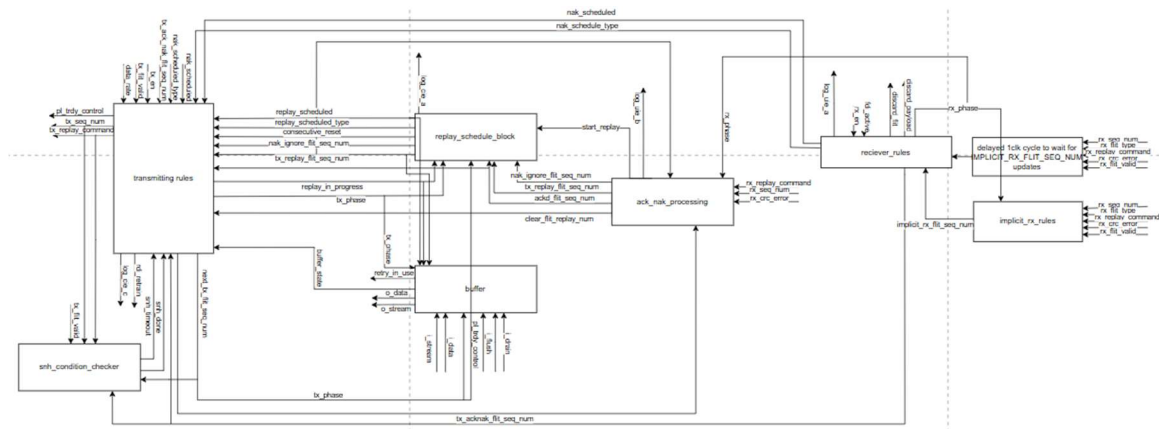


Figure 33 – Retry block internal subblock architecture

The top module Retry Top block instantiates these 7 submodules: implicit RX rules, receiver rules, Ack/Nak processing. Replay schedule, buffer, transmitting rules, SNH condition checker.

3.13.4.1 Implicit RX rules

3.13.4.1.1 Purpose

Tracks the implicit receive flit sequence number — the sequence number that the receiver derives without it being explicitly carried in every flit. This is used by the receiver rules module for sequence number validation.

3.13.4.1.2 How It Works

- First explicit sequence number: When a flit arrives with `replay_command == explicit`, no CRC error, and `seq_num != 0`, the module latches `rx_seq_num` as the baseline implicit sequence number and sets `r_non_idle_explicit_seq_num_flit_rcvd = 1`.
- Subsequent payload flits: After the baseline is established, each valid payload flit (no CRC error, non-zero `seq_num`, non-explicit command) causes the implicit sequence number to auto-increment by 1.
- Reset: On hardware reset, software reset (`init = 0`), or when `rx_phase == R_IDLE`, the counter resets to 0.

3.13.4.1.3 Input/Output Signal

Signal name	Direction	Source / Dest
clk	in	Top-level clk
rst_n	in	Top-level rst_n
init	in	Top-level init
i_rx_seq_num	in	Top-level rx_seq_num
i_crc_error	in	Top-level rx_crc_error
i_replay_command	in	Top-level rx_replay_command
i_rx_flit_type	in	Top-level rx_flit_type
i_rx_phase	in	Internal rx_phase from u_receiver_rules
i_flit_valid	in	Top-level rx_flit_valid
o_implicit_rx_flit_seq_num	out	Internal implicit_rx_flit_seq_num to u_receiver_rules.

Table 26 - signal description of *implixit rx rules block in retry*

3.13.4.2 Receiver Rules

3.13.4.2.1 Purpose

Validates every received flit and decides whether to accept it (schedule an ACK), reject it (schedule a NAK), or discard it. Also manages the RX-side phase state machine.

3.13.4.2.2 How it Works

Condition	Action	Task Called
CRC error + NAK already scheduled	Discard entire flit	flit_discard_1()
CRC error + no NAK scheduled	Schedule a standard NAK	nak_schedule_0()
Explicit command with seq_num == 0 in NORMAL EXCHANGE	Discard + log UIE	flit_discard_2()
NOP flit with bad sequence number	Schedule NAK	nak_schedule_2()
NOP flit with good sequence number	Discard payload (NOPs carry no payload)	flit_discard_0()
Payload flit + NAK already pending	Run standard nak_procedure()	
Payload flit + duplicate seq num	Discard payload	flit_discard_0()
Payload flit + bad seq num	Schedule NAK	nak_schedule_2()
Payload flit + good seq num	Schedule ACK	ack_schedule_0()

Table 27 - receiver Rules for retry and schedules

3.13.4.2.3 Input/Output Signals

Signal name	Direction	Source / Dest
clk	in	Top-level clk
rst_n	in	Top-level rst_n
init	in	Top-level init
i_rx_crc_error	in	Top-level rx_crc_error
i_rx_flit_type	in	Top-level rx_flit_type
i_rx_replay_command	in	Top-level rx_replay_command
i_rx_seq_num	in	Top-level rx_seq_num
i_snh_done	in	Internal snh_done from u_snh_condition_checker
i_snh_timeout	in	Internal snh_timeout from u_snh_condition_checker
i_fdi_active	in	Top-level fdi_active
i_rx_en	in	Top-level rx_en
i_implicit_rx_flit_seq_num	in	Internal implicit_rx_flit_seq_num from u_implicit_rx_rules
i_flit_valid	in	Top-level rx_flit_valid
o_rx_phase	out	Internal rx_phase to u_implicit_rx_rules, u_ack_nak_processing, u_replay_schedule, u_buffer
o_log_uie	out	Internal log_uie_a to top-level OR logic
o_discard_flit	out	Top-level discard_flit
o_discard_payload	out	Top-level discard_payload
o_nak_scheduled	out	Internal nak_scheduled to u_transmitter_rules
o_nak_schedule_type	out	Internal nak_schedule_type to u_transmitter_rules

o_tx_acknak_flit_seq_num	out	Internal tx_acknak_flit_seq_num to u_transmitter_rules, u_snh_condition_checker
--------------------------	-----	---

Table 28 - signal description of receiver rules block in retry

3.13.4.3 ACK/NAK Processing

3.13.4.3.1 Purpose

Processes incoming ACK and NAK responses from the remote side (received via the mainband RX path) and updates the local transmitter state accordingly. This module works on the TX side's view of the remote receiver's acknowledgments.

3.13.4.3.2 How It Works

When a flit arrives without CRC error and with replay_command == ack or nak: Validity check: The sequence number is validated using valid_sequence_number() — it must fall within the window [ackd_flit_seq_num, next_tx_flit_seq_num - 1] modulo 255. If invalid, logs UIE. Seq num == 0: Silently ignored (initial value). Valid ACK/NAK common path: If rx_seq_num advances past ackd_flit_seq_num, update ackd_flit_seq_num and assert clear_flit_replay_num (reset the replay counter in transmitter rules). ACK-specific: Clear nak_ignore_flit_seq_num to 0. If the ACK covers the replay window, clear tx_replay_flit_seq_num. NAK-specific: Set nak_ignore_flit_seq_num to suppress duplicate NAK replays. Assert o_start_replay to trigger a replay.

3.13.4.3.3 Input/Output Signal

Signal name	Direction	Source / Dest
clk	in	Top-level clk
rst_n	in	Top-level rst_n
init	in	Top-level init
i_rx_replay_command	in	Top-level rx_replay_command
i_rx_seq_num	in	Top-level rx_seq_num
i_rx_crc_error	in	Top-level rx_crc_error
i_next_tx_flit_seq_num	in	Internal next_tx_flit_seq_num from u_transmitter_rules
i_tx_replay_flit_seq_num	in	Internal tx_replay_flit_seq_num_u2 from u_replay_schedule
i_rx_phase	in	Internal rx_phase from u_receiver_rules
o_clear_flit_replay_num	out	Internal clear_flit_replay_num to u_transmitter_rules
o_ackd_flit_seq_num	out	Internal ackd_flit_seq_num to u_replay_schedule, u_buffer, u_transmitter_rules
o_tx_replay_flit_seq_num	out	Internal tx_replay_flit_seq_num_u1 to u_replay_schedule
o_nak_ignore_flit_seq_num	out	Internal nak_ignore_flit_seq_num_u1 to u_replay_schedule
o_start_replay	out	Internal start_replay to u_replay_schedule
o_log_uie	out	Internal log_uie_b to top-level OR logic

Table 29 - signal description of ACK/NAK processing block in retry

3.13.4.4 Replay Schedule

3.13.4.4.1 Purpose

Decides when and from where to start a replay of buffered flits. It arbitrates between timeout-triggered replays and NAK-triggered replays.

3.13.4.4.2 How it Works

Trigger	Condition	Task	Behavior
Timeout	replay_timeout_flit_count ≥ 375 and no replay active	replay_schedule_0()	Start standard replay from ackd_flit_seq_num + 1; reset consecutive counters; log CIE
NAK	start_replay asserted, no replay active, and not a duplicate NAK	replay_schedule_1()	Start standard replay from rx_seq_num + 1; set NAK ignore
Passthrough	Replay in progress	—	De-assert replay_scheduled; forward upstream values

Table 30 - replay schedule mechanism

3.13.4.4.3 Input/Output Signals

Signal name	Direction	Source / Dest
clk	in	Top-level clk
rst_n	in	Top-level rst_n
init	in	Top-level init
i_tx_phase	in	Internal tx_phase from u_transmitter_rules
i_replay_in_progress	in	Internal replay_in_progress from u_transmitter_rules
i_start_replay	in	Internal start_replay from u_ack_nak_processing
i_rx_seq_num	in	Top-level rx_seq_num
i_ackd_flit_seq_num	in	Internal ackd_flit_seq_num from u_ack_nak_processing
i_nak_ignore_flit_seq_num	in	Internal nak_ignore_flit_seq_num_u1 from u_ack_nak_processing
i_replay_timeout_flit_count	in	Internal replay_timeout_flit_count from u_transmitter_rules
i_tx_replay_flit_seq_num	in	Internal tx_replay_flit_seq_num_u1 from u_ack_nak_processing
o_tx_replay_flit_seq_num	out	Internal tx_replay_flit_seq_num_u2 to u_ack_nak_processing, u_buffer, u_transmitter_rules
o_nak_ignore_flit_seq_num	out	Internal nak_ignore_flit_seq_num_u2 to u_transmitter_rules
o_consecutive_reset	out	Internal consecutive_reset to u_transmitter_rules
o_log_cie	out	Internal log_cie_a to top-level OR logic
o_replay_scheduled	out	Internal replay_scheduled to u_buffer, u_transmitter_rules
o_replay_scheduled_type	out	Internal replay_scheduled_type to u_transmitter_rules

Table 31 - signal description of replay schedule block in retry

3.13.4.5 Retry Buffer

3.13.4.5.1 Purpose

Manages the TX retry buffer — a circular storage that holds transmitted flits until they are ACK'd. It supports three operations:

Operation	Description
Write	Store outgoing flit data/stream as the transmitter sends them
Replay	Read back previously-sent flits for retransmission
Purge	Logically free buffer entries after ACK (via advancing ackd_data_ptr)

Table 32 - retry buffer operation

3.13.4.5.2 Pointer Managements

Pointer	Purpose	Computed From
write_data_ptr	Next write address for new flits	$4 \times (\text{next_tx_flit_seq_num} - 1) + \text{chunk_counter}$
replay_data_ptr	Current replay read address	$4 \times (\text{tx_replay_flit_seq_num} - 1)$, then increments
acked_data_ptr	Last purged address	$4 \times (\text{ackd_flit_seq_num} - 1)$

Table 33- ram pointers in retry buffer

3.13.4.5.3 Parameters

DATA WIDTH	64 bytes = $64 * 8 = 512$
STREAM WIDTH	5 bits
ADDR DATA WIDTH	10
ADDR STREAM WIDTH	8
DATA DEPTH	$256 * 4 = 1024$
STREAM DEPTH	256

Table 34 - parameters of memory

3.13.4.5.4 Input/Output Signal

Signal name	Direction	Source / Dest
clk	in	Top-level clk
rst_n	in	Top-level rst_n
init	in	Top-level init
i_tx_phase	in	Internal tx_phase from u_transmitter_rules
i_tx_replay_flit_seq_num	in	Internal tx_replay_flit_seq_num_u2 from u_replay_schedule
i_ackd_flit_seq_num	in	Internal ackd_flit_seq_num from u_ack_nak_processing
i_replay_scheduled	in	Internal replay_scheduled from u_replay_schedule

i_replay_in_progress	in	Internal replay_in_progress from u_transmitter_rules
i_flush	in	Top-level flush
i_drain	in	Top-level drain
i_pl_trdy_control	in	Internal pl_trdy_control from u_transmitter_rules
i_next_tx_flit_seq_num	in	Internal next_tx_flit_seq_num from u_transmitter_rules
i_data	in	Top-level tx_i_data
i_stream	in	Top-level tx_i_stream
i_dout	in	Top-level ram_i_dout
o_write_enable	out	Top-level ram_o_write_enable
o_addr	out	Top-level ram_o_addr
o_din	out	Top-level ram_o_din
o_data	out	Top-level tx_o_data
o_stream	out	Top-level tx_o_stream
o_buffer_state	out	Top-level buffer_state, also to u_transmitter_rules
o_retry_in_use	out	Top-level retry_in_use

Table 35- signal description of buffer block in retry

3.13.4.6 Transmitter Rules

3.13.4.6.1 Purpose

The central TX control module. It manages:

- The TX-side phase state machine.
- Flit sequence number assignment (next_tx_flit_seq_num).
- Replay command selection (explicit / ack / nak) per flit.
- Flow control (pl_trdy_control) — backpressure to the data source.
- Replay execution (driving replay_in_progress).
- Excessive replay detection → triggers link retrain.
- Replay timeout counter (replay_timeout_flit_count).

3.13.4.6.2 How it Works

SNH: Assert pl_trdy_control = 1 (backpressure ON — no payload accep with replay_command = explicit and seq_num = 0xFF (SNH handshake flits).

SNH FDI Active: Begin accepting payload (pl_trdy_control = 0) unless the unacknowledged window exceeds MAX_UNACKNOWLEDGED_FLITS. Apply the replay command rotation: every other flit alternates between explicit (carrying the sequence number) and ack/nak (carrying acknowledgment). If a replay is scheduled, enter the replay path instead.

Normal Exchange: The main operating state. The replay command selection follows this priority: Explicit — if normal_exchange_explicit_condition() is true (handles replay path,

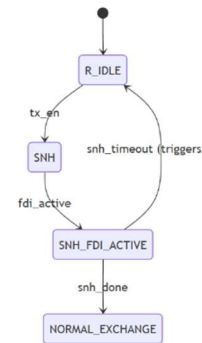


Figure 34 – phases in retry

NAK-ignore, and consecutive-count logic). NAK — if nak_scheduled && nak_schedule_type == standard_nak. ACK — default fallback.

Replay Execution (flit_replay_transmit_0): When a standard replay is scheduled: Assert pl_trdy_control = 1 (stop new data). Set replay_in_progress = 1. Set tx_seq_num to the replay start point. Increment flit_replay_num by 2 (counts replay attempts). The replay runs until tx_seq_num catches up with next_tx_flit_seq_num - 1.

Excessive Replay Detection (flit_replay_transmit_2): If flit_replay_num >= 6 (3 replay attempts at 32 GT/s): Assert o_rdi_retrain — request link retraining. Assert o_log_cie — log a Correctable Internal Error.

3.13.4.6.3 Input/Output Signals

Signal name	Direction	Source / Dest
clk	in	Top-level clk
rst_n	in	Top-level rst_n
init	in	Top-level init
i_snh_done	in	Internal snh_done from u_snh_condition_checker
i_snh_timeout	in	Internal snh_timeout from u_snh_condition_checker
i_replay_scheduled	in	Internal replay_scheduled from u_replay_schedule
i_replay_scheduled_type	in	Internal replay_scheduled_type from u_replay_schedule
i_nak_scheduled	in	Internal nak_scheduled from u_receiver_rules
i_nak_schedule_type	in	Internal nak_scheduled_type from u_receiver_rules
i_consecutive_reset	in	Internal consecutive_reset from u_replay_schedule
i_tx_acknak_flit_seq_num	in	Internal tx_acknak_flit_seq_num from u_receiver_rules
i_tx_en	in	Top-level tx_en
i_buffer_state	in	Internal buffer_state from u_buffer
i_fdi_active	in	Top-level fdi_active
i_flit_valid	in	Top-level tx_flit_valid
i_data_rate	in	Top-level data_rate
i_clear_flit_replay_num	in	Internal clear_flit_replay_num from u_ack_nak_processing
i_tx_replay_flit_seq_num	in	Internal tx_replay_flit_seq_num_u2 from u_replay_schedule
i_nak_ignore_flit_seq_num	in	Internal nak_ignore_flit_seq_num_u2 from u_replay_schedule
i_ackd_flit_seq_num	in	Internal ackd_flit_seq_num from u_ack_nak_processing
o_next_tx_flit_seq_num	out	Internal next_tx_flit_seq_num to u_ack_nak_processing, u_buffer
o_replay_timeout_flit_count	out	Internal replay_timeout_flit_count to u_replay_schedule

o_tx_replay_command	out	Top-level tx_replay_command, also to u_snh_condition_checker
o_pl_trdy_control	out	Top-level pl_trdy_control, also to u_buffer
o_tx_phase	out	Internal tx_phase to u_replay_schedule, u_buffer, u_snh_condition_checker
o_tx_seq_num	out	Top-level tx_seq_num, also to u_snh_condition_checker
o_rdi_retrain	out	Top-level rdi_retrain
o_replay_in_progress	out	Internal replay_in_progress to u_replay_schedule, u_buffer
o_log_cie	out	Internal log_cie_c to top-level OR logic

Table 36 - signal description of transmitter rules block in retry

3.13.4.7 SNH Condition Checker

3.13.4.7.1 Purpose

Determines when the Sequence Number Handshake (SNH) phase is complete, or if it has timed out. The SNH is the startup handshake that both sides of the link perform before entering normal data exchange.

3.13.4.7.2 SNH Completion Condition

The SNH is considered done when both criteria are met: tx_ack_flit_count ≥ 3 — at least 3 ACK flits have been sent with a non-zero sequence number while the remote also sent a non-zero sequence number. tx_explicit_flit_count ≥ 9 — at least 9 explicit flits have been sent with a non-zero sequence number.

3.13.4.7.3 SNH Timeout

A snh_flit_counter increments on every flit sent. If snh_flit_counter ≥ 128 and the SNH condition has NOT been met then o_snh_timeout = 1. Timeout causes the link to go back to R_IDLE and triggers retraining.

3.13.4.7.4 Input/Output Signals

Signal name	Direction	Source / Dest
clk	in	Top-level clk
rst_n	in	Top-level rst_n
init	in	Top-level init
i_tx_phase	in	Internal tx_phase from u_transmitter_rules
i_flit_sent	in	Top-level tx_flit_valid
i_tx_replay_command	in	Internal tx_replay_command from u_transmitter_rules
i_tx_flit_seq_num	in	Internal tx_seq_num from u_transmitter_rules
i_tx_acknak_flit_seq_num	in	Internal tx_acknak_flit_seq_num from u_receiver_rules
o_snh_done	out	Internal snh_done to u_receiver_rules, u_transmitter_rules
o_snh_timeout	out	Internal snh_timeout to u_receiver_rules, u_transmitter_rules

Table 37 - signal description of SNH condition checker block in retry

3.13.5 Retry Flow Chart

3.13.5.1 TX Path

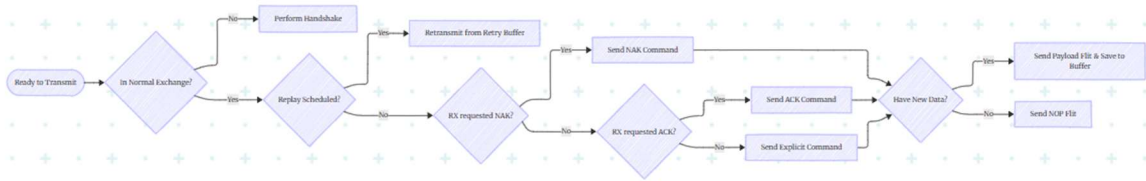


Figure 35 – TX Path in retry

3.13.5.2 RX Path

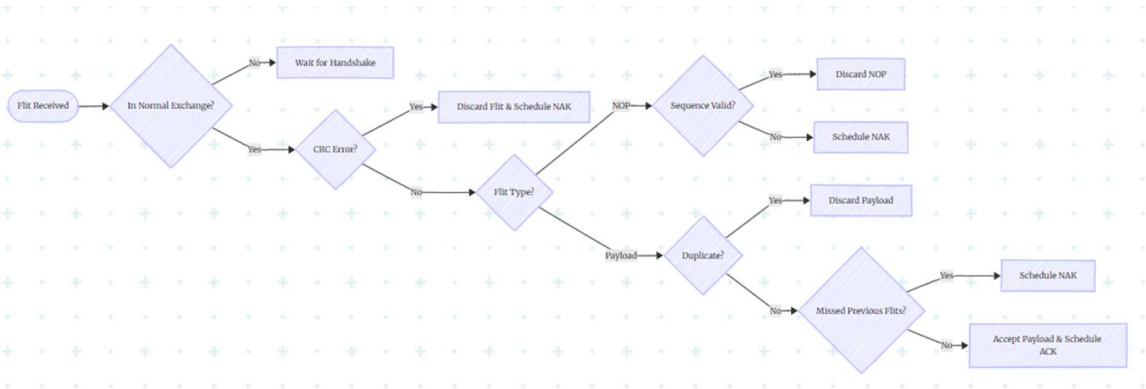


Figure 36 – RX path in retry

3.13.5.3 Data Flow Summary

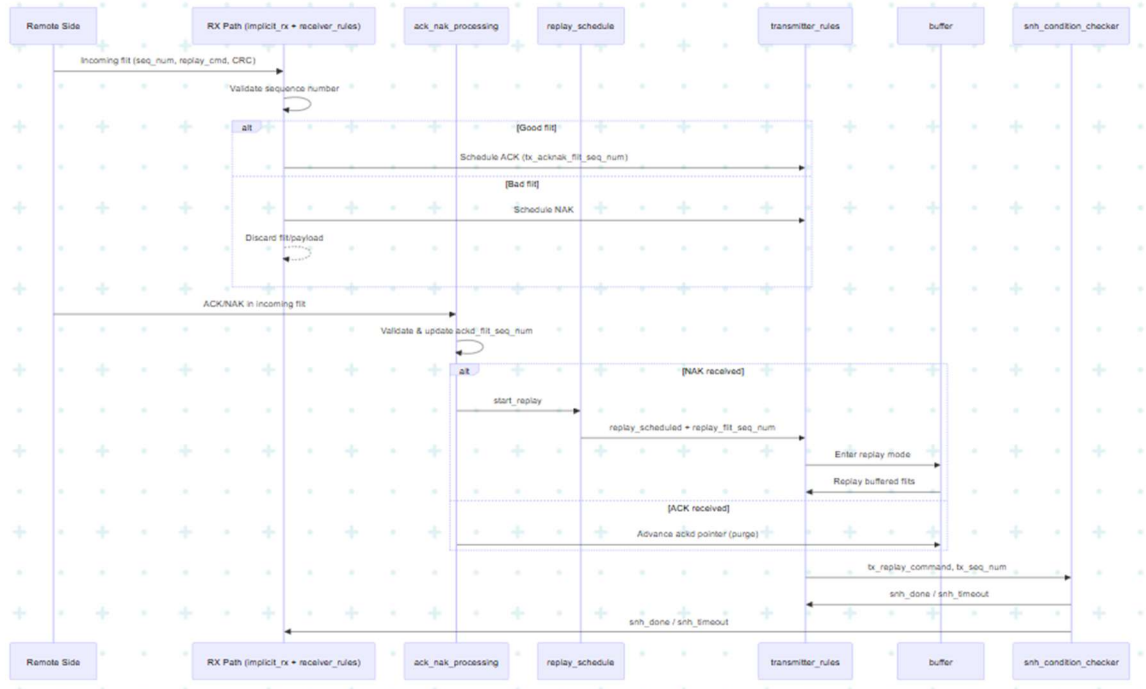


Figure 37 – data flow in retry

3.13.6 Test Cases

Test case	Stimulus	Expected result	Expected log / note
Reset	RDI reset/init state	Clear retry state, counters, and pending replay activity	Retry logic reinitialized and ready for SNH
Enter SNH	Sequence Number Handshake state	Start handshake sequencing	Sending NOP flits with seq num = 255
Enter SNH active	Active state from FDI with retry sequencing established	Sending payload flits and monitoring the flits to satisfy SNH transition conditions	Payload flits starting with seq num = 1. SNH completed successfully
SNH timeout	SNH transition conditions not satisfied	Trigger rdi_retrain and exit to idle/reset path	Handshake timeout condition
Enter Normal exchange	Normal retry-enabled operation	Accept payload flits with sequence/CRC monitoring	Normal traffic flow active
First CRC error	Crc error = 1 of a received flit	Schedule NAK (nak_schedule_0)	CRC error detected, starting replay flow
Second CRC while NAK scheduled	Ignore NAK	Discard affected flit (flit_discard_1)	Repeated CRC error while NAK pending
NAK received	Replay active	Assert pl_trdy=1, block new data, replay until complete, then resume	Replay started and completes successfully
Bad/Out-of-range seq 0xF0	Error handling path	Schedule NAK (nak_schedule_2) and discard payload	Invalid sequence number rejected
Duplicated already	Duplicate ACKed flit	Discard flit (flit_discard_0) and discard payload	Duplicate acknowledged flit rejected
Explicit seq=0 in NORMAL	Error path	Discard flit and log UIE	Reserved explicit sequence number treated as invalid
Flush/drain	Retry buffer drain mode	Set RAM read mode, write enable=0	Buffer contents drained without new writes
Valid payload	ACK scheduling path	Schedule ACK (ack_schedule_0) and clear NAK	Payload accepted and acknowledgement queued
Reset with init	Initialized reset state	Reinitialize retry state for next operation	Clean restart of retry mechanism

Table 38 - Retry Test Cases

3.13.7 Results of Retry

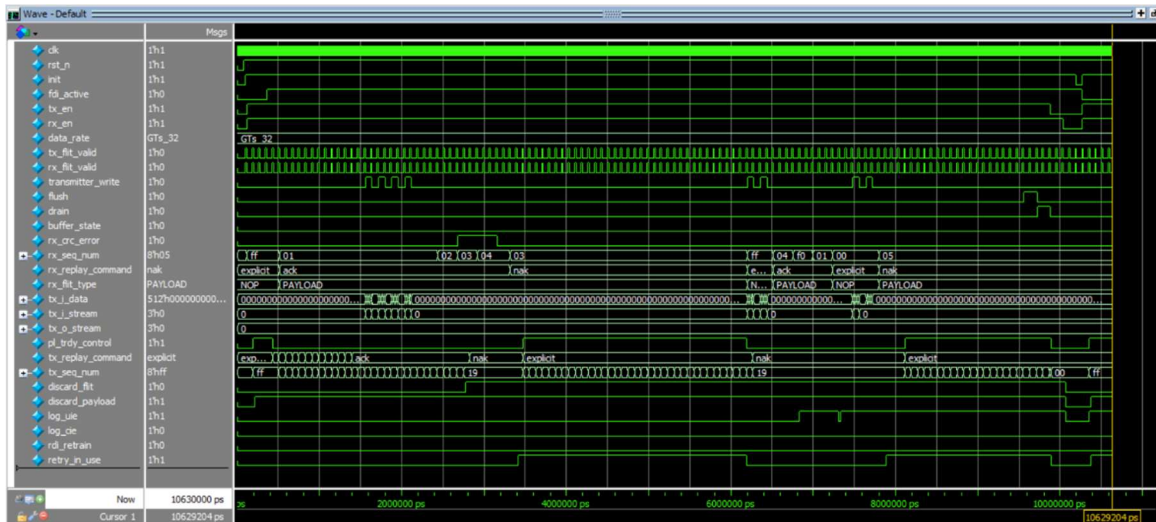


Figure 39 - Retry Top Waveform

```

# =====
#
#      Simulation Complete
#      PASSED: 38   FAILED: 0   TOTAL: 38
#      >>> ALL ASSERTIONS PASSED <<<
#
# =====
#
# ** Note: $stop      : retry_top_scenario_tb.sv(483)
#      Time: 10630 ns  Iteration: 1  Instance: /UC_MB_retry_top_scenario_tb

```

Figure 38 - Retry Top Testbench Transcript

Chapter 4: Sub-Block: Sideband

4.1 Overview

The UC1e Sideband interface is a dedicated communication channel used to exchange control and management information between chiplets. Unlike the high-speed data path, the sideband carries low-bandwidth transactions such as register accesses, link management, parameter exchange, configuration, acknowledgments, and error reporting. It is also responsible for link initialization, transaction completion, and flow control, ensuring reliable communication between the local and remote dies. The sideband protocol supports several packet categories, including Register Access Packets (Configuration and Memory Read/Write), Completion Packets (with or without returned data), Message Packets (Link Management, Parameter Exchange, Vendor-defined), Management Transport Messages (optional). Priority Sideband Traffic Packets for high-priority control operations. To better understand how these packet types are processed, let's examine the architecture and transaction flow of the Sideband block.

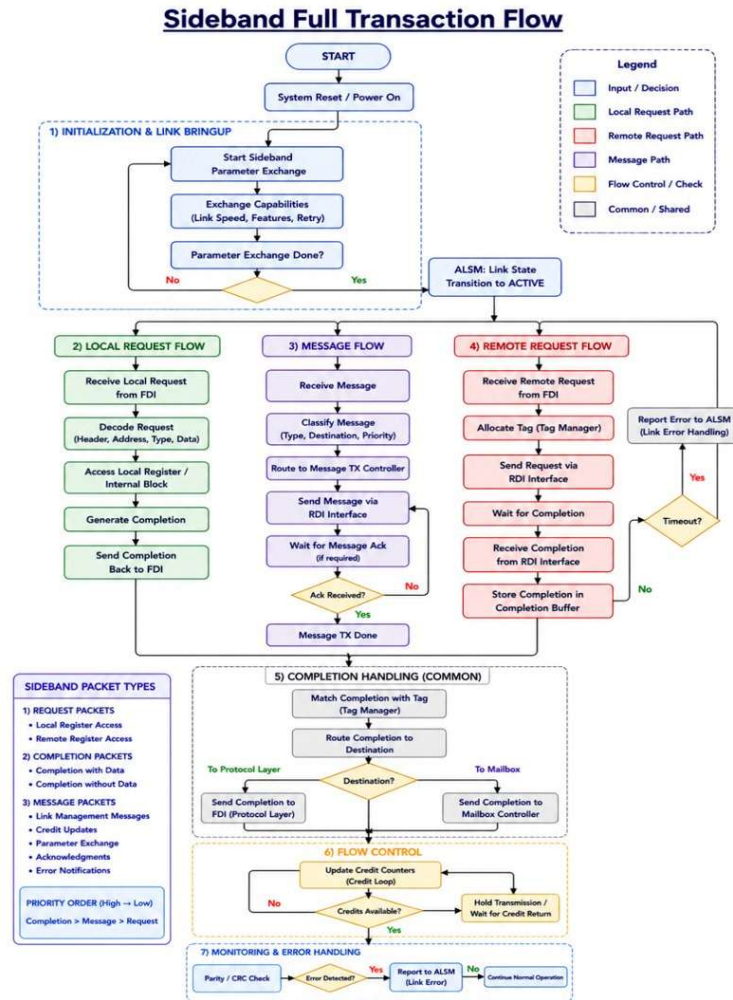


Figure 40 - sideband workflow

4.2 Sideband Final Architecture

The diagrams below present the complete Sideband architecture, showing the interconnection between all functional blocks across the FDI and RDI interfaces.

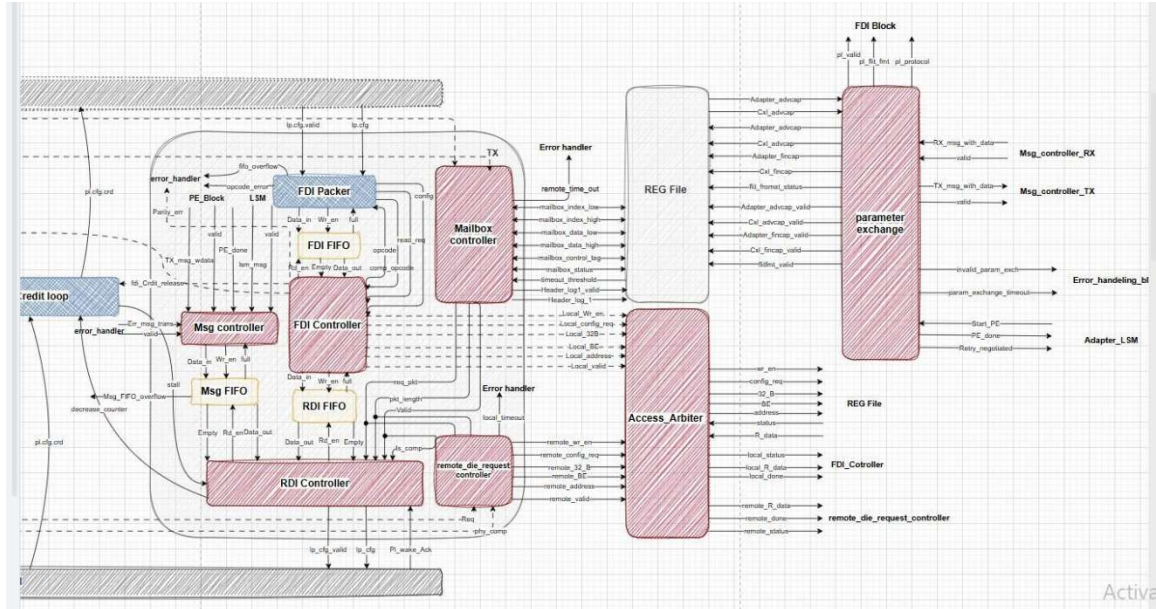


Figure 42 – sideband architecture 1

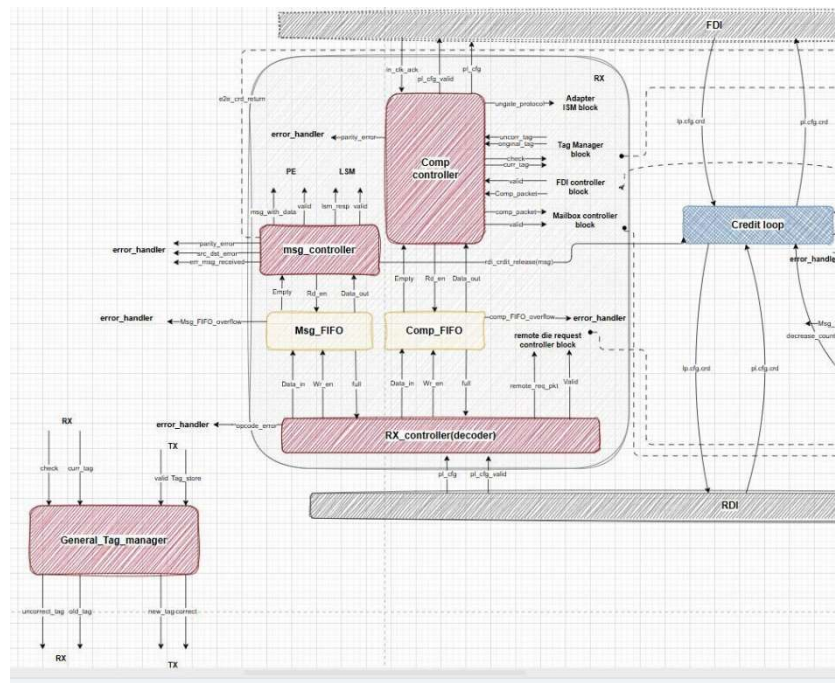


Figure 41 – sideband architecture 2

4.3 Block Documentation

4.3.1 Parameter Exchange Block

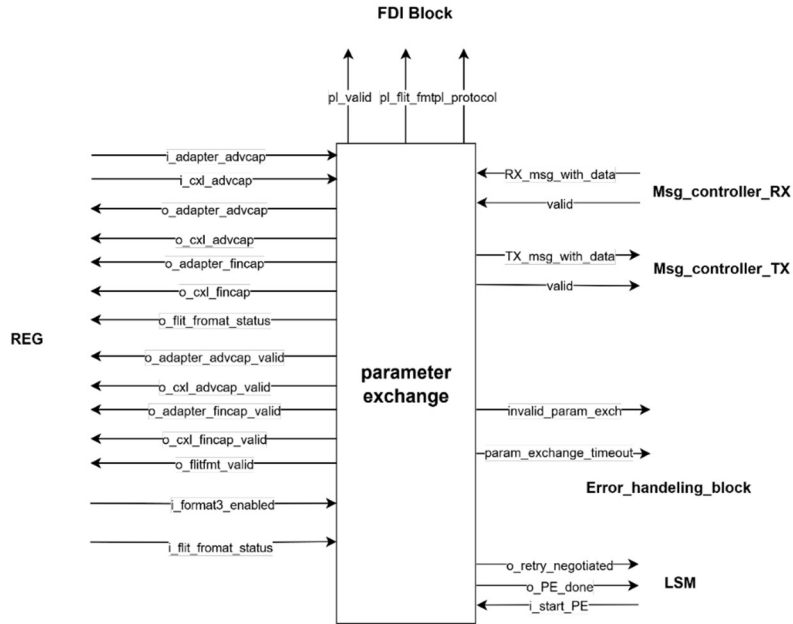


Figure 43 – parameter exchange block

4.3.1.1 Functionality

This block implements the UCIE Sideband Parameter Exchange sequence between the two link partners (RP and EP) during link bring-up. It is responsible for:

- Transmitting capability messages (Advertised / Finalized capabilities).
- Receiving and decoding incoming capability messages (Msg-with-data packets).
- Selecting the final negotiated protocol and flit format and forwarding them through FDI
- Logging advertised and finalized capabilities into registers.
- Handling timeout (8 ms) if the exchange does not complete.

4.3.1.2 Signal Description

Port	Dir	Width	Description
i_adapter_advcap	IN	64	Local Advertised Adapter Capabilities used for negotiation.
i_cxl_advcap	IN	64	Local Advertised CXL Capabilities used for CXL negotiation stage.
rx_msg_valid	IN	1	Valid flag for rx_msg_with_data.
rx_msg_with_data	IN	128	Received Msg-with-data packet carrying remote advertised/final capabilities in [127:64].

tx_msg_with_data	OUT	128	Transmitted capability Msg-with-data (Adv/Fin Adapter/CXL).
tx_msg_valid	OUT	1	Valid flag for tx_msg_with_data.
i_flit_fmt_status	IN	5	Current negotiated flit format from regfile (Link Status [25:22]), used when reporting to protocol layer.
o_flit_fromat_status	OUT	5	Writes final negotiated flit format into Link Status + Header Log2 locations.
i_format3_enabled	IN	1	Enable format 3
i_start_PE	IN	1	Start Parameter Exchange from LSM.
o_PE_done	OUT	1	Parameter Exchange completed successfully (valid negotiation).
invalid_param_exch	OUT	1	Invalid negotiation (no common protocol/format).
param_exchange_timeout	OUT	1	Timeout occurred (> 8ms).
o_Retry_negotiated	OUT	1	Retry negotiated status forwarded to LSM.
pl_protocol	OUT	4	Negotiated protocol indication to Protocol Layer.
pl_flit_fmt	OUT	4	Negotiated flit format indication to Protocol Layer.
pl_valid	OUT	1	valid for protocol.
o_adapter_advcap	OUT	64	Logging: advertised adapter cap value written to regfile.
o_cxl_advcap	OUT	64	Logging: advertised CXL cap value written to regfile.
o_adapter_fincap	OUT	64	Logging: finalized adapter cap written to regfile.
o_cxl_fincap	OUT	64	Logging: finalized CXL cap written to regfile.
o_adapter_advcap_valid	OUT	1	Write enable for advertised adapter cap log.
o_cxl_advcap_valid	OUT	1	Write enable for advertised CXL cap log.
o_adapter_fincap_valid	OUT	1	Write enable for finalized adapter cap log.
o_cxl_fincap_valid	OUT	1	Write enable for finalized CXL cap log.
o_flitfmt_valid	OUT	1	Write enable for flit format status register.

Table 39 - Signal Description of Parameter exchange block in sideband

4.3.2 Access Arbiter

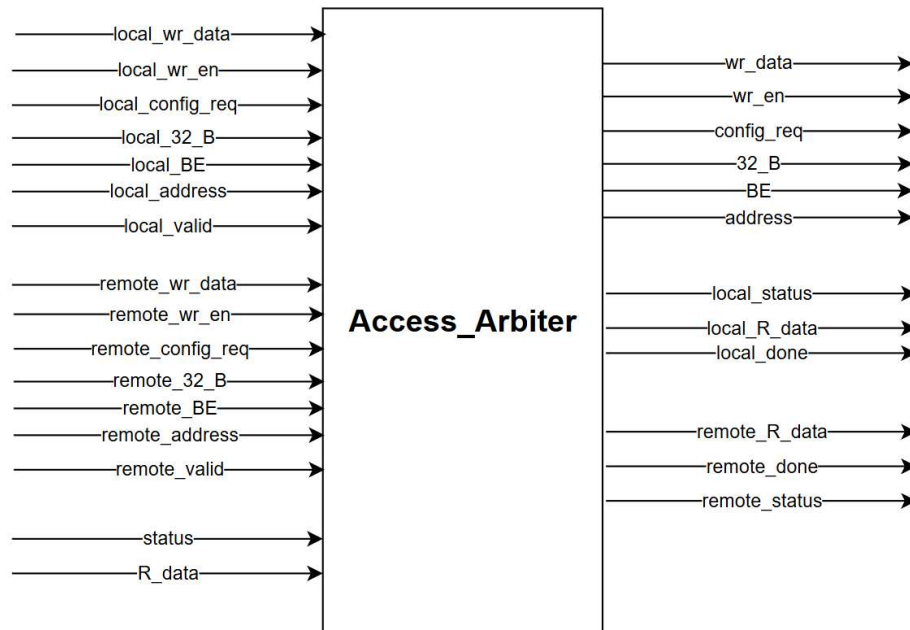


Figure 44 – access arbiter block

4.3.2.1 Functionality

The Access Arbiter Block is responsible for coordinating access to the Register File between multiple request sources. It acts as a centralized control unit to prevent conflicts during simultaneous access attempts. The block receives register access requests from two sources: Local Requests (via FDI Controller) Remote Requests (via Remote Die Request Controller), It applies a fixed priority scheme where remote requests are given higher priority than local ones.

4.3.2.2 Signal description

Port	Direction	Width	Description	Connected to
Local_wr_data	IN	Parameterized (32 or 64 bit)	Write data of the local request	FDI controller
Local_wr_en	IN	1	Write or read local request	FDI controller
Local_cofig_req	IN	1	Cofig or memory local request	FDI controller
Local_32_B	IN	1	32 or 64 bits for data	FDI controller
Local_BE	IN	8	Byte enable for local request	FDI controller
Local_address	IN	24	Address of local request	FDI controller

Local_valid	IN	1	There is local request or not	FDI controller
remote_wr_data	IN	Parameterized (32 or 64 bit)	Write data of the remote request	Remote die request controller
remote_wr_en	IN	1	Write or read remote request	Remote die request controller
remote_cofig_req	IN	1	Cofig or memory remote request	Remote die request controller
remote_32_B	IN	1	32 or 64 bits for data	Remote die request controller
remote_BE	IN	8	Byte enable for remote request	Remote die request controller
remote_address	IN	24	Address of remote request	Remote die request controller
remote_valid	IN	1	There is remote request or not	Remote die request controller
Status	IN	3	Completion status	Reg File
R_data	IN	Parameterized (32 or 64 bit)	Read data from register	Reg File
wr_data	OUT	Parameterized (32 or 64 bit)	Write data of the request	Reg File
wr_en	OUT	1	Write or read request flag	Reg File
cofig_req	OUT	1	Config or memory request flag	Reg File
32_B	OUT	1	32 or 64 bits for data flag	Reg File
BE	OUT	8	Byte enable for request	Reg File
address	OUT	24	Address of request	Reg File
Local_status	OUT	3	Completion status of local request	FDI controller
Local_R_data	OUT	Parameterized (32 or 64 bit)	Read data for local request	FDI controller
Local_done	OUT	1	Local request done	FDI controller
remote_status	OUT	3	Completion status of remote request	Remote die request controller
remote_R_data	OUT	Parameterized (32 or 64 bit)	Read data for remote request	Remote die request controller
remote_done	OUT	1	remote request done	Remote die request controller

Table 40 - signal description of access arbiter block in sideband

4.3.3 Mailbox Controller

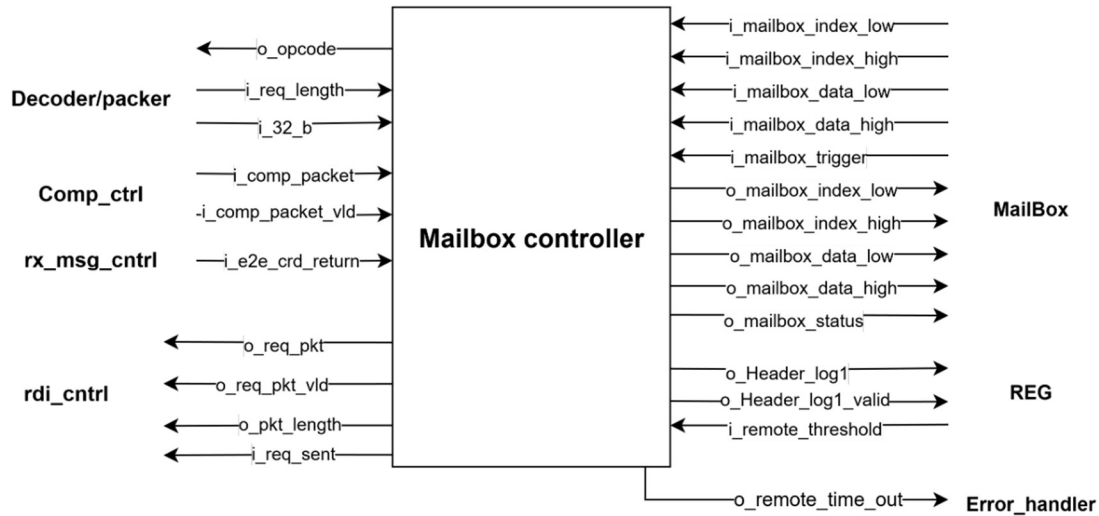


Figure 45 – mailbox controller block

4.3.3.1 Functionality

Mailbox Controller is responsible for initiating and managing remote register access transactions toward the link partner (EP) using the mailbox mechanism. Its operation can be summarized as follows: Wait for E2E Credit: RP controller stays idle until `i_e2e_crd_return` is asserted (credit available to send a remote request). Check Trigger: When credit exists, it monitors `i_mailbox_trigger` to start a new remote register access. Build Request Packet: It generates the request packet (opcode/address/BE/data) and calculates CP/DP parity. Send to RDI: It asserts `o_req_pkt` and `o_req_pkt_vld` until RDI confirms sending using `i_req_sent`. Wait for Completion or Timeout: It waits for `i_comp_packet_vld`; otherwise it counts timeout, updates timeout counter vs `i_remote_threshold`, and raises `o_remote_time_out` if the threshold is reached.

4.3.3.2 Signal Description

Port	Dir	Width	Description
<code>i_mailbox_index_low</code>	IN	32	Mailbox index low field. Used to generate opcode, byte enable, and part of address.
<code>i_mailbox_index_high</code>	IN	5	Mailbox index high field. Used as upper bits of remote address.
<code>i_mailbox_data_low</code>	IN	32	Mailbox payload data low (bits [31:0]).
<code>i_mailbox_data_high</code>	IN	32	Mailbox payload data high (bits [63:32]).
<code>i_mailbox_trigger</code>	IN	1	Trigger bit to start a remote register access request.
<code>i_req_length</code>	IN	1	Request type/length: 1 = 128-bit request, 0 = 64-bit request.

i_32_b	IN	1	Operation width: 1 = 32-bit, 0 = 64-bit.
i_comp_packet	IN	128	Completion packet received from Rx for the remote request.
i_comp_packet_vld	IN	1	Valid flag for received completion packet.
i_e2e_crd_return	IN	1	Indicates that EP returned an E2E credit, so RP can issue a remote request.
i_req_sent	IN	1	Indicates RDI has finished sending the remote request packet.
i_remote_threshold	IN	4	Timeout threshold value (maximum allowed remote request timeouts before raising timeout).
o_opcode	OUT	5	Opcode extracted from mailbox index to be decoded by opcode decoder.
o_req_pkt	OUT	128	Generated remote register access request packet sent to RDI controller.
o_req_pkt_vld	OUT	1	Valid flag for o_req_pkt.
o_pkt_length	OUT	1	Packet length : 1 = 128-bit, 0 = 64-bit.
o_mailbox_data_low	OUT	32	Returned data low from completion packet.
o_mailbox_data_high	OUT	32	Returned data high from completion packet.
o_mailbox_status	OUT	2	Mailbox status output indicating completion result or timeout condition.
o_Header_log1	OUT	64	Logged completion header/data for debug in case of error responses (UR/CA).
o_Header_log1_valid	OUT	1	Write enable for Header Log1 register.
o_remote_time_out	OUT	1	Asserted when timeout counter reaches the configured threshold
o_mailbox_data_vld	OUT	1	Enables writing received completion data into mailbox registers.
o_mailbox_trigger_en	OUT	1	Clears mailbox trigger after completion received or timeout.

Table 41 - signal description of mailbox controller block in sideband

4.3.4 FDI packer

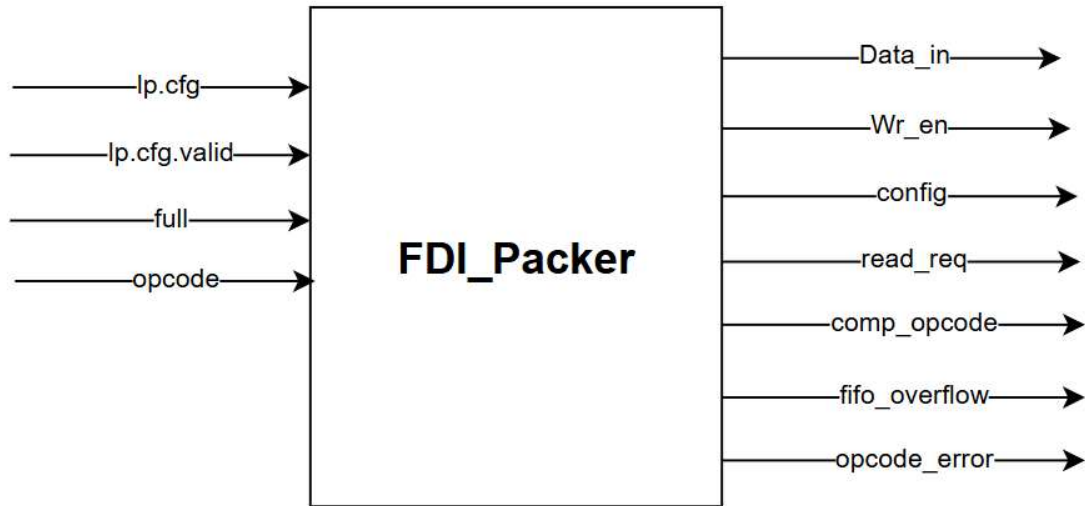


Figure 46 – FDI packer block

4.3.4.1 Functionality

The FDI Packer Block is responsible for handling local register access requests originating from the Protocol Layer. Its role is to construct the appropriate sideband packets for transmission over the sideband interface. **Input Interface:** The block receives requests through the `ip_cfg` interface, which can be 8-bit, 16-bit, or 32-bit wide, depending on the protocol configuration. **Packet Construction:** It constructs access request packets (such as register reads or writes) according to the sideband packet format, which may reach up to 128 bits. The block contains an Opcode Decoder, which interprets and validates the operation type for each incoming request. **Buffering:** Once constructed, packets are stored in the FDI FIFO, awaiting scheduling for transmission. **Error Signaling:** The block also detects and signals two error conditions: **FIFO Overflow:** Indicates the FIFO buffer is full and cannot accept more packets. **Opcode Error:** Raised when an invalid or unsupported opcode is detected in the incoming request.

4.3.4.2 Signal description

Port	Direction	Width	Description	Connected to
Lp_cfg	IN	Parameterized (8,16,32 Bit)	Incoming request data from FDI	FDI
Lp_cfg_valid	IN	1	Valid signal for lp_cfg	FDI
full	IN	1	FIFO full flag	FDI FIFO
opcode	IN	5	Request opcode	FDI controller
Data_in	OUT	128	Input packet data	FDI FIFO

Wr_en	OUT	1	FIFO write enable	FDI_FIFO
config	OUT	1	Config or memory request flag	FDI_controller
Read_req	OUT	1	Write or read request flag	FDI_controller
Comp_opcode	OUT	5	Completion opcode	FDI_controller
Fifo_overflow	OUT	1	FIFO overflow error	Error_handler
Opcode_error	OUT	1	Invalid opcode error	Error_handler

Table 42 - signal description of FDI packer block in sideband

4.3.5 FDI Controller

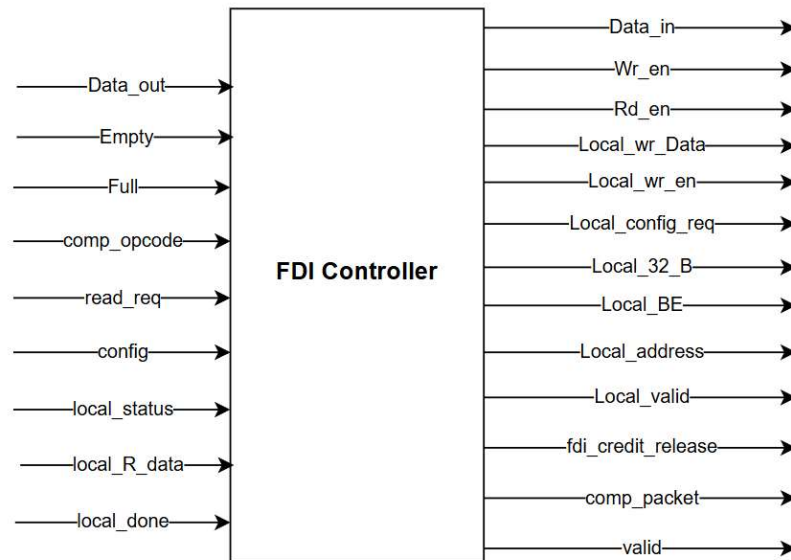


Figure 47 – FDI controller block

4.3.5.1 Functionality

The FDI Controller Block is responsible for managing the routing and execution of local register access requests, determining whether each request targets the PHY or the Adapter. Request Routing: Upon receiving a local request, the controller examines the destination address to decide whether the transaction should be: Forwarded to the PHY, Handled internally by the Adapter Adapter Access Handling: For requests targeting the Adapter: The controller accesses the corresponding registers, performing read or write operations as required. Interaction with Packer: The FDI Controller extracts the opcode from the request and forwards it to the FDI Packer. The Packer then responds with the interpreted request type (e.g., configuration read, memory write), enabling the controller to handle the request appropriately. Output to RDI FIFO: All formatted and processed requests are then forwarded to the RDI FIFO.

4.3.5.2 Signal description

Port	Direction	Width	Description	Connected to
Data_out	IN	128	Output data packet from fifo	FDI FIFO
Empty	IN	1	FIFO empty flag	FDI FIFO
Full	IN	1	FIFO full flag	RDI FIFO
Comp_opcode	IN	5	Opcode of the completion	FDI Packer
Read_Req	IN	1	Write or read request flag	FDI Packer
config	IN	1	Cofig or memory request	FDI Packer
Local_status	IN	3	Completion status of local request	Access Arbiter
Local_R_data	IN	Parameterized (32 or 64 bit)	Read data for local request	Access Arbiter
Local_done	IN	1	Local request done	Access Arbiter
Data_in	OUT	128	input data packet to fifo	RDI FIFO
Wr_en	OUT	1	FIFO write enable	RDI FIFO
Rd_en	OUT	1	FIFO read enable	FDI FIFO
Local_wr_data	OUT	Parameterized (32 or 64 bit)	Write data of the local request	Access Arbiter
Local_wr_en	OUT	1	Write or read local request	Access Arbiter
Local_config_req	OUT	1	Cofig or memory local request	Access Arbiter
Local_32_B	OUT	1	32 or 64 bits for data	Access Arbiter
Local_BE	OUT	8	Byte enable for local request	Access Arbiter
Local_address	OUT	24	Address of local request	Access Arbiter
Local_valid	OUT	1	There is local request or not	Access Arbiter
Fdi_credit_release	OUT	1	Enables Protocol to issue new local request	Credit loop
Comp_packet	OUT	128	response to local Protocol request (where dst of request is adapter)	Comp_controller
Valid	OUT	1	Valid for the comp_packet	Comp_controller

Table 43 - signal description of FDI controller block in sideband

4.3.6 RDI controller

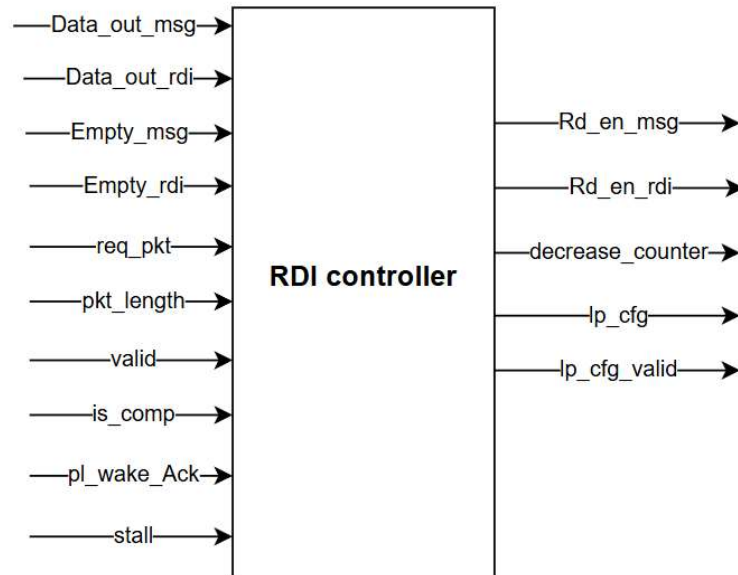


Figure 48 – RDI controller

4.3.6.1 Functionality

The RDI Controller Block is responsible for managing the transmission of Sideband Packets over the RDI interface. It ensures correct packet prioritization, credit tracking, and flow control during active link operation.

FIFO Arbitration and Prioritization: The controller receives sideband packets from FIFOs. Message FIFO and RDI FIFO. It enforces a strict priority order. Completions are sent first (highest priority), followed by Messages Then Requests. **Credit Loop Management:** The controller works with the Credit Loop Block to track available credits. For each credited packet sent (e.g., message or request), the controller decrements the credit counter. If a STALL is received, transmission is paused until new credits become available. **Note:** Completions are never blocked by credit status and must always be transmitted immediately, as they are guaranteed to be sunk by the receiver. **Link Management Signaling:** The controller is also responsible for notifying the Link State Machine (LSM) to take corrective action if the PL_WAKE_ACK signal is not asserted by the Physical Layer in time, (wake up physical)

4.3.6.2 Signal description

Port	Direction	Width	Description	Connected to
Data_out_msg	IN	128	Output data packet from msg fifo	Msg_FIFO

Data_out_rdi	IN	128	Output data packet from rdi fifo	RDI_FIFO
Empty_msg	IN	1	Empty flag of msg fifo	Msg_FIFO
Empty_rdi	IN	1	Empty flag of rdi fifo	RDI_FIFO
Req_pkt	IN	128	Request packet	Mailbox controller & remote die request controller
Pkt_lenght	IN	7	Indicate pkt length is 64 or 128	Mailbox controller & remote die request controller
valid	IN	1	Valid signal for req_pkt	Mailbox controller & remote die request controller
Is_comp	IN	1	Indicate if there is a completion or not	remote die request controller
Pl_wake_ack	IN	1	Indicate is a Physical layer wake or not	RDI
Stall	IN	1	paused the transmission until new credits available	Credit loop
Pl_cfg	OUT	Parameterized (8,16,32)	Incoming request data from FDI	RDI
Pl_cfg_valid	OUT	1	Valid signal for lp_cfg	RDI
Rd_en_msg	OUT	1	Msg FIFO read enable	Msg FIFO
Rd_en_rdi	OUT	1	RDI FIFO read enable	RDI_FIFO
Decrease_counter	OUT	1	Used to decrease available credit of sending packet across RDI	Credit loop

Table 44 - signal description of RDI controller in sideband

4.3.7 Msg Controller TX

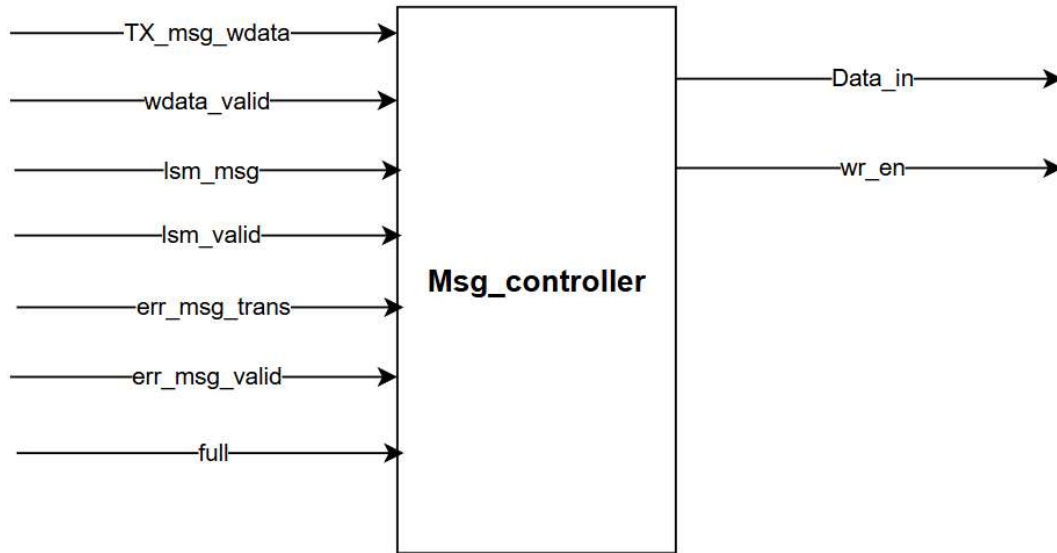


Figure 49 – msg controller TX

4.3.7.1 Functionality

The Message Controller Block assembles complete sideband packets in the correct format, supporting both: Messages with Data (e.g., Parameter Exchange), Messages without Data (e.g., Link Management)

Message Handling Priority: Messages with Data are handled first, and originate from the Parameter Exchange (PE) Block. The controller waits for the PE_DONE signal before continuing. After transmitting all data-bearing messages and receiving confirmation, the controller begins handling Messages without Data, which come from the Link State Machine (LSM).

Error Message Injection: The block also accepts high-priority error messages from the Error Handler, signaled via: error_msg_trans: carries the actual error message (e.g., Correctable, Non-Correctable, or Fatal error), error_msg_valid: asserted to indicate that the error message is valid and must be sent immediately. When error_msg_valid is high, the error message takes highest priority and is sent before all other message types.

FIFO Management: All constructed messages are stored in a FIFO queue, pending transmission through the sideband interface.

4.3.7.2 Signal description

Port	Direction	Width	Description	Connected to
------	-----------	-------	-------------	--------------

TX_msg_wdata	IN	128	Message with data	Parameter exchange
Wdata_valid	IN	1	Indicates that TX_msg_wdata is valid and ready to send	Parameter exchange
Lsm_msg	IN	—	Message without data	Adapter lsm
Lsm_valid	IN	1	Indicates that Lsm_msg is valid and ready to send	Adapter lsm
Err_msg_trans	IN	128	Error message (Correctable/NonCorrectable/Fatal)	Error Handler
Err_msg_valid	IN	1	Signals that Err_msg_trans is valid and must be sent immediately.	Error Handler
full	IN	1	Indicates that FIFO is full	Msg FIFO
Data_in	OUT	128	input sideband packet to msg fifo	Msg FIFO
Wr_en	OUT	1	Write enable for Msg FIFO	Msg FIFO

Table 45 - signal description of msg controller TX in sideband

4.3.8 Remote die request controller

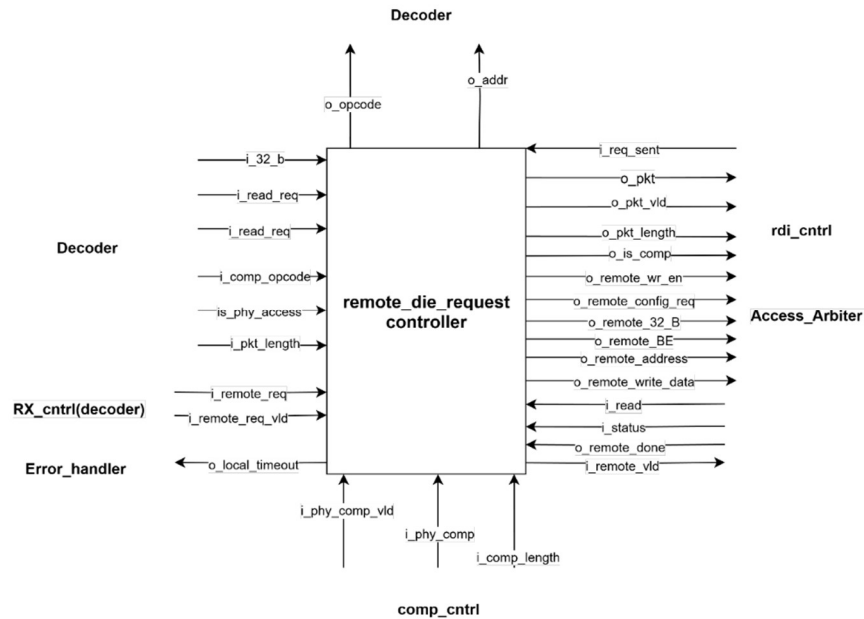


Figure 50 – remote die request controller block

4.3.8.1 Functionality

The Remote Die Request Controller receives remote access requests from the RX and determines whether the request targets the adapter registers or the PHY. For adapter accesses, it performs register read or write operations and locally generates the corresponding completion packet. For PHY accesses, it forwards the request to the PHY through the RDI interface and waits for the returned completion. The controller

formats and sends all requests and completions to RDI with correct packet type, parity, and length information. A timeout mechanism is implemented to detect missing PHY completions and report a local timeout to the Error_handler block.

4.3.8.2 Signal description

Port	Dir	Width	Description
i_remote_req	IN	128	Remote request packet from RX.
i_remote_req_vld	IN	1	Indicates valid remote request.
i_read_req	IN	1	Indicates read/write operation.
i_comp_opcode	IN	5	Completion opcode from decoder.
is_phy_access	IN	1	Indicates PHY access (1 = PHY, 0 = adapter).
i_pkt_length	IN	1	Packet length indicator (64-bit / 128-bit).
i_phy_comp	IN	128	Completion packet received from PHY.
i_phy_comp_vld	IN	1	Indicates valid PHY completion.
i_comp_length	IN	1	Completion packet size (128-bit / 64-bit).
i_status	IN	3	Status from register file .
i_read_data	IN	64	Read data from register file.
i_req_sent	IN	1	Indicates packet accepted by RDI.
i_remote_done	IN	1	Access arbiter operation done.
o_opcode	OUT	5	Decoded opcode sent to decoder.
o_addr	OUT	24	Decoded address sent to decoder.
o_remote_write_data	OUT	64	Write data sent to register file.
o_remote_wr_en	OUT	1	Register file write enable.
o_remote_address	OUT	24	Register file address.
o_remote_BE	OUT	8	Byte enable for register access.
o_remote_config_req	OUT	1	Configuration register request.
o_remote_32_B	OUT	1	Indicates 32-bit register access.
o_remote_vld	OUT	1	Valid request to access arbiter.
o_pkt	OUT	128	Packet sent to RDI.
o_pkt_vld	OUT	1	Packet valid signal.
o_pkt_length	OUT	1	Packet size indicator (128-bit / 64-bit).
o_is_comp	OUT	1	Indicates completion packet.
o_local_timeout	OUT	1	Local timeout indication to Error_handler.

Table 46 - signal description of remote die request controller block in sideband

4.3.9 Credit loop

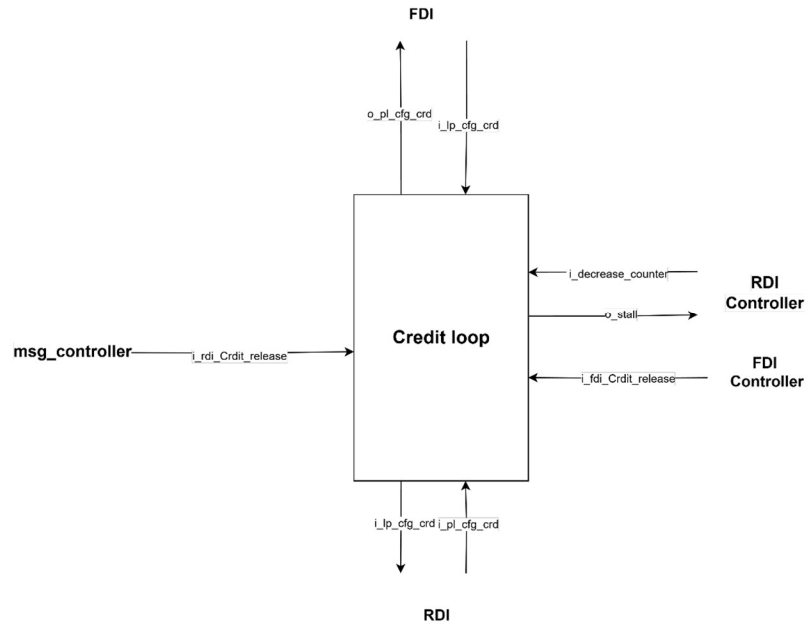


Figure 51 – credit loop block

4.3.9.1 Functionality

The Sideband Credit Loop manages credit exchange between the adapter, protocol layer, and PHY for sideband traffic over the FDI and RDI interfaces. It maintains independent credit counters for the protocol layer and the PHY, initializing them to their maximum values after reset. Credits are decremented when sideband buffers are consumed and incremented when credit return signals are received. The controller generates stall indications to prevent transmission or reception when the corresponding credit pool is exhausted. Credit return signals are transparently forwarded between the adapter, protocol layer, and PHY to close the credit loop and ensure lossless flow control.

4.3.9.2 Signal description

Port	Dir	Width	Description
i_rdi_Crdit_release	IN	1	Indicates that the adapter has released a location from its RDI FIFO.
i_fdi_Crdit_release	OUT	1	Indicates that the adapter has released a location from its FDI FIFO.
i_decrease_counter	IN	1	Indicates credit consumption from PHY or protocol buffer.
o_stall	OUT	1	Stall indication when no credits are available and stop transmtion.

i_lp_cfg_crd	IN	1	Credit return from protocol layer to adapter.
o_pl_cfg_crd	OUT	1	Credit return forwarded from adapter to protocol layer.
i_pl_cfg_crd	IN	1	Credit return from PHY to adapter.
o_lp_cfg_crd	OUT	1	Credit return forwarded from adapter to PHY.

Table 47 - signal description of credit loop block in sideband

4.3.10 Comp controller

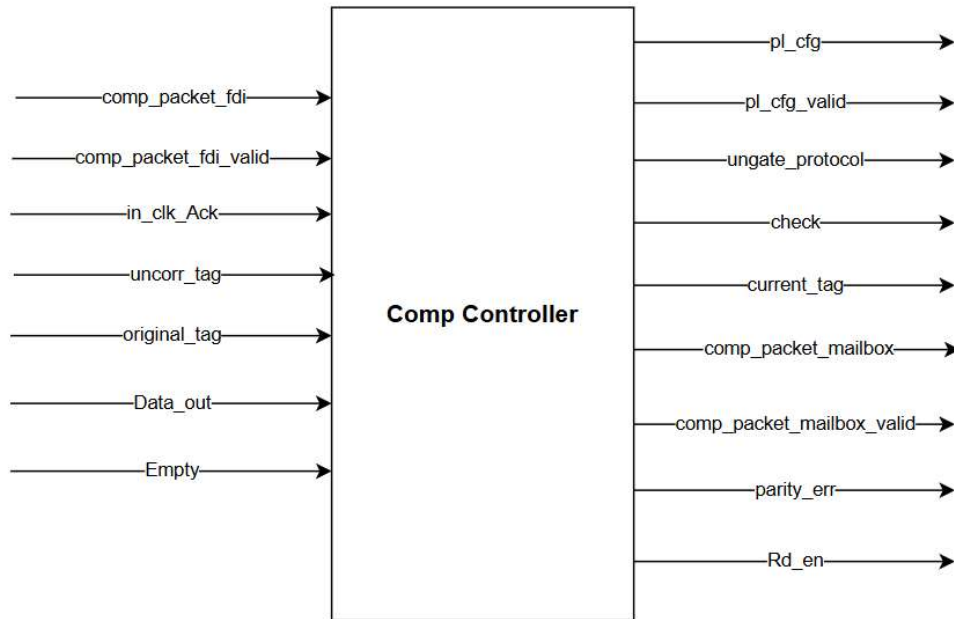


Figure 52 – comp controller block

4.3.10.1 Functionality

The Completion Controller Block is responsible for managing completion packets from both local and remote sources and forwarding them to the appropriate destinations. Dual-Sided Completion Handling: Local Completions from FDI Controller: These are completions for register access requests initiated by the Protocol Layer. The controller forwards them to the Protocol Layer via the pl_cfg interface. Remote Completions from Remote Die: These completions correspond to remote register access requests issued through the Mailbox mechanism. They are routed to the Mailbox Controller, which processes and updates the appropriate Mailbox Registers. Protocol Layer Check: Before sending completions to the Protocol Layer, the controller checks the in_clk_ack signal from the Protocol Layer to ensure it is active and ready. If this signal is not asserted, the controller notifies the Adapter LSM by asserting the ungate_protocol signal.

4.3.10.2 Signal description

Port	Direction	Width	Description	Connected to
Comp_packet_fdi	IN	128	Completion from FDI Controller (reply to protocol request targeting adapter)	FDI controller
Comp_packet_fdi_valid	IN	1	Indicates that Comp_packet_fdi is valid and ready	FDI controller
In_clk_Ack	IN	1	Indicates protocol layer is awake and can receive data	FDI
Uncorr_tag	IN	—	Signal indicating received tag is invalid	Tag manager
Original_tag	IN	5	Tag value remapped back to its original by tag manager.	Tag manager
Data_out	IN	128	Completion packet received over RDI.	Comp FIFO
Empty	IN	1	Indicates the completion FIFO is empty.	Comp FIFO
Pl_cfg	OUT	Parameterized (8,16,32)	Output signal to send completion (from the same die) to protocol layer.	FDI
Pl_cfg_valid	OUT	1	Indicates Pl_cfg contains valid data.	FDI
Ungate_protocol	OUT	1	Signal to wake up protocol layer if In_clk_Ack is low	Adapter lsm
check	OUT	1	Triggers tag manager to validate the given tag.	Tag manager
Current_tag	OUT	5	Tag sent to tag manager for checking.	Tag manager
Comp_packet_mailbox	OUT	128	Remote Completion packet for mailbox controller.	Mailbox controller
Comp_packet_mailbox_valid	OUT	1	Indicates that mailbox completion packet is valid.	Mailbox controller
Parity_err	OUT	1	Indicates a parity error was detected.	Error handler
Rd_en	OUT	1	Enables reading data from completion FIFO.	Comp FIFO

Table 48 - signal description of comp controller block in sideband

4.3.11 Msg controller RX

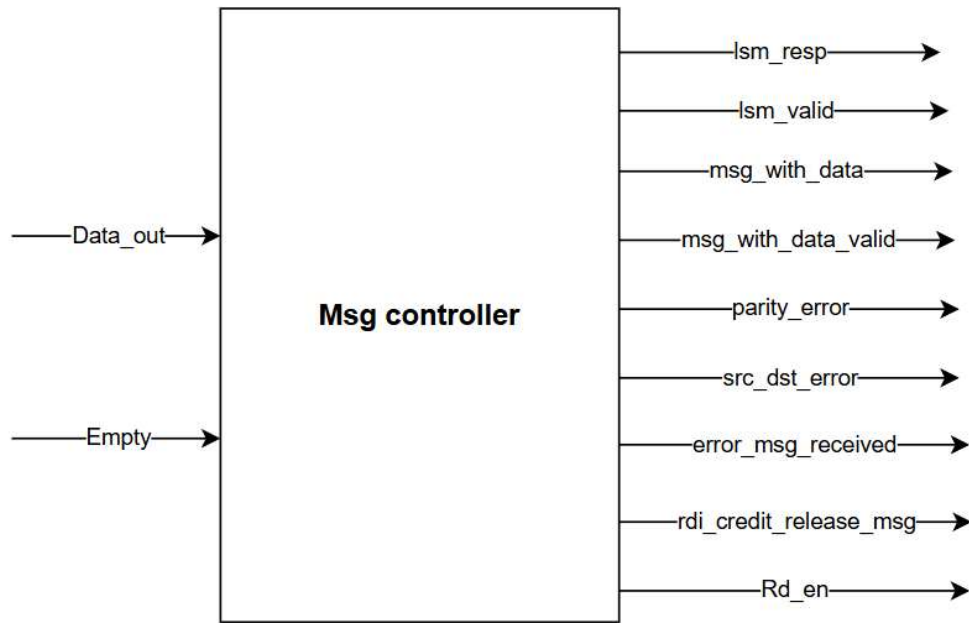


Figure 53 – msg controller RX block

4.3.11.1 Functionality

The block reads sideband messages from the Message FIFO and determines their type based on the packet format: Messages with Data (Parameter Exchange messages), Messages without Data (Link State control messages), Error messages. Messages with Data are routed to the Parameter Exchange (PE) Block, where they are used in link capability negotiation. Messages without Data are forwarded to the Link State Machine (LSM) Block, which manages the link's operational state transitions. Error messages like parity error , src or dst error and error message received from the remote die are forwarded to Error handler block to deal with them.

4.3.11.2 Signal description

Port	Direction	Width	Description	Connected to
Data_out	IN	128	Message received over RDI from PHY or remote die.	Msg_FIFO
Empty	IN	1	Indicates that the Msg_FIFO is empty.	Msg_FIFO
Lsm_resp	OUT	3	Link State Machine response message (without data)	Adapter lsm
Lsm_valid	OUT	1	Indicates that Lsm_resp is valid.	Adapter lsm

Msg_with_data	OUT	128	Data message to Parameter Exchange block.	Parameter exchange
Msg_with_data_valid	OUT	1	Indicates that Msg_with_data is valid.	Parameter exchange
Parity_error	OUT	1	Signals a parity error was detected in the received message.	Error handler
Src_dst_error	OUT	1	Indicates incorrect source or destination in received message.	Error handler
Rdi_credit_release_msg	OUT	1	Signals availability of a new credit for RDI transmission.	Credit loop
Rd_en	OUT	1	Signals availability of a new credit for RDI transmission.	Msg_FIFO
Error_msg_received	OUT	128	Indicates an error Msg was received (correctable, uncorrectable, or fatal)	Error handler

Table 49 - signal description of msg controller RX block in sideband

4.3.12 RX Controller

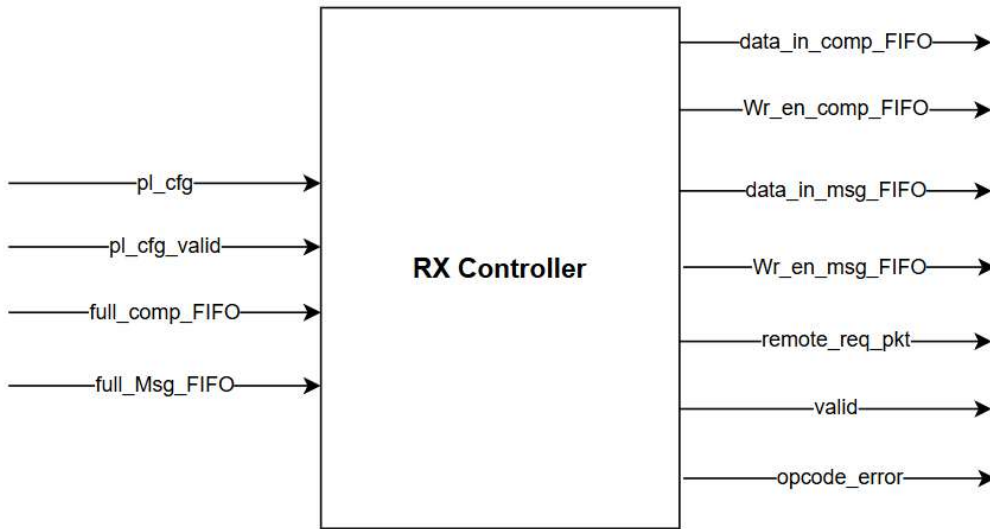


Figure 54 – RX controller block

4.3.12.1 Functionality

The RX Controller Block is responsible for parsing and classifying incoming sideband packets retrieved from the RX FIFO. Input: The block reads fully assembled Sideband Packets from the RX FIFO, as constructed by the RDI Packet Block. Opcode-Based Classification: For each packet, the controller inspects the Opcode field to determine the type of transaction: Message (with or without data), Register Access Request, Completion. Output: After classification, each packet is forwarded to its dedicated Block: Remote

request for remote die request controller block, Completions to comp FIFO, Messages to Msg FIFO.

4.3.12.2 Signal description

Port	Direction	Width	Description	Connected to
Pl_cfg	IN	Parameterized (8,16,32)	Packet received over RDI from either local PHY or remote die.	RDI
Pl_cfg_valid	IN	1	Indicates Pl_cfg contains valid data.	RDI
Full_comp_FIFO	IN	1	Indicates that the comp FIFO is full	Comp FIFO
Full_msg_FIFO	IN	1	Indicates that the msg FIFO is full	Msg FIFO
Data_in_comp_FIFO	OUT	128	Completion packet from physical layer or remote die	Comp FIFO
Wr_en_comp_FIFO	OUT	1	Write enable for comp FIFO	Comp FIFO
Data_in_Msg_FIFO	OUT	128	Message packet from physical layer or remote die	Msg FIFO
Wr_en Msg_FIFO	OUT	1	Write enable for msg FIFO	Msg FIFO
Remote_req_pkt	OUT	128	Request packet from the remote die	Remote die request controller
valid	OUT	1	Indicates the Remote_req_pkt is valid.	Remote die request controller
Opcode_error	OUT	1	Raised when an invalid or unsupported opcode is detected.	Error handler

Table 50 - signal description of RX controller block in sideband

4.3.13 Tag Manager



Figure 55 – Tag manager block

4.3.13.1 Functionality

TX Path – Tag Allocation and Storage: Receives the tag value from the transmitting block via the `tag_store` signal. Checks the validity and uniqueness of the tag. If the tag is valid and available, it asserts the `correct_tag` signal. If the tag is invalid, reused, or conflicting, it asserts `correct_tag = 0` and triggers tag remapping logic to assign a new valid tag. **RX Path – Completion Tag Validation:** When a completion packet is received, the block checks the received tag against its internal tracking table. If the tag is valid and was previously remapped: It outputs the original tag (pre-remapped) to ensure correct routing of the completion. If the tag is not recognized or mismatched: It asserts the `uncorrect_tag` signal to flag an error condition.

4.3.13.2 Signal description

Port	Direction	Width	Description	Connected to
Valid	IN	1	Indicates that the incoming tag is ready to be validated and stored	FDI packer
Tag_store	IN	5	The tag value to be stored or validated	FDI packer
Check	IN	1	Triggers validation for a received tag to start check it	Comp controller
Current_tag	IN	5	Tag received from completion controller to be checked	Comp controller
Correct	OUT	1	Set to 1 if the tag is valid and accepted.	FDI packer
New_tag	OUT	5	Outputs a new tag if the original one was invalid or reused	FDI packer
Uncorrect_tag	OUT	1	Set to 1 if the tag is not recognized or has an error	Comp controller

Old_tag	OUT	5	Outputs the original tag if the received one was previously remapped	Comp controller
----------------	-----	---	--	-----------------

Table 51 - signal description of tag manager block in sideband

4.3.14 FIFO

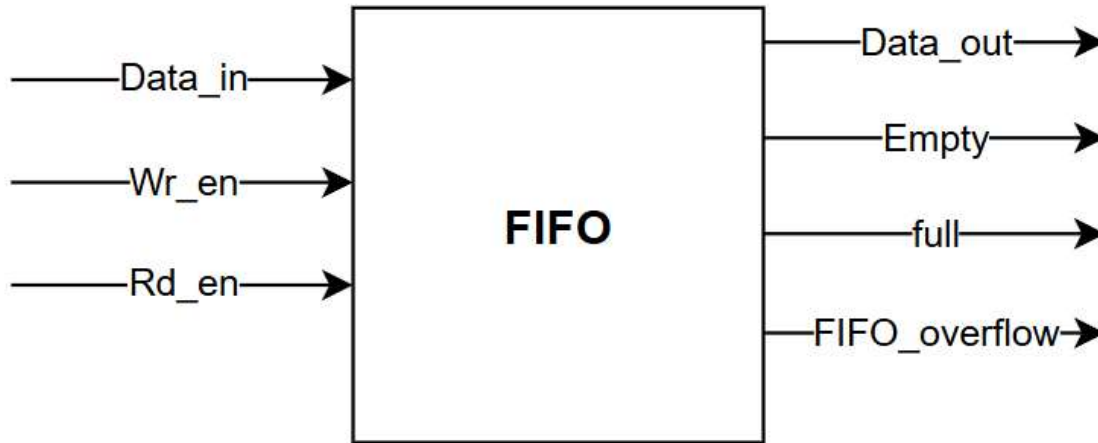


Figure 56 – FIFO block

4.3.14.1 Functionality

The FIFO (First-In First-Out) buffer stores data in the order it is written and releases it in the same order. It is used to handle data flow between blocks running at different speeds.

- When data is written (Wr_en is high), it is stored if the FIFO is not full.
- When data is read (Rd_en is high), the oldest data is output if the FIFO is not empty.
- It shows signals like full, empty, and FIFO_overflow to help manage proper data flow and avoid loss.

4.3.14.2 Signal description

Port	Direction	Width	Description
Data_in	IN	128	The data to be written into the FIFO
Wr_en	IN	1	Write enable. When high, writes data to FIFO if not full.
Rd_en	IN	1	Read enable. When high, reads data from FIFO if not empty.
Data_out	OUT	128	The oldest data stored, sent out on read
Empty	OUT	1	High when FIFO has no data to read.

Full	OUT	1	High when FIFO is full and cannot accept more data.
Fifo_overflow	OUT	1	Goes high if write is attempted when FIFO is full (error condition)

Table 52 - signal description of FIFO block in sideband

4.4.1 Sideband TX

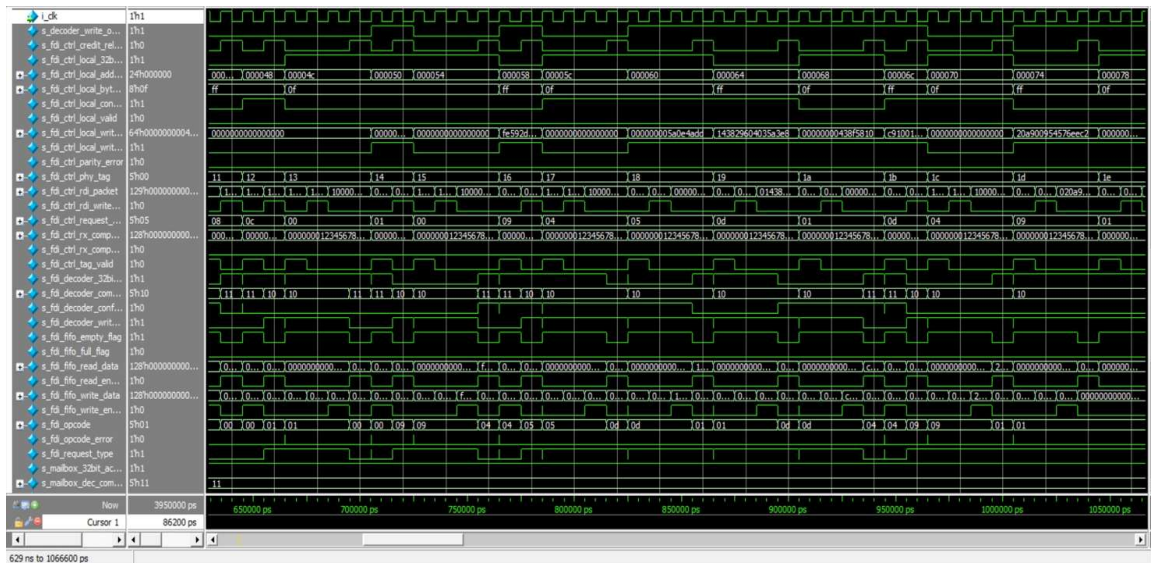


Figure 58 - sideband TX TOP waveform

```
#####
# FDI TO RDI OPCODE SCOREBOARD CHECK
#####
# Opcode Matched at index 0: 00001
# Opcode Matched at index 1: 00001
# Opcode Matched at index 2: 00000
# Opcode Matched at index 3: 00000
# Opcode Matched at index 4: 01101
# Opcode Matched at index 5: 01101
# Opcode Matched at index 6: 01101
# Opcode Matched at index 7: 00000
# Opcode Matched at index 8: 01001
# Opcode Matched at index 9: 01001
# Opcode Matched at index 10: 01001
# Opcode Matched at index 11: 00100
# Opcode Matched at index 12: 00100
# Opcode Matched at index 13: 00101
# Opcode Matched at index 14: 01001
# Opcode Matched at index 15: 00000
# Opcode Matched at index 16: 01000
# Opcode Matched at index 17: 01100
# Opcode Matched at index 18: 00000
# Opcode Matched at index 19: 00001
# Opcode Matched at index 20: 00000
# Opcode Matched at index 21: 01001
# Opcode Matched at index 22: 00100
# Opcode Matched at index 23: 00101
# Opcode Matched at index 24: 01101
# Opcode Matched at index 25: 00001
# Opcode Matched at index 26: 01101
# Opcode Matched at index 27: 00100
# Opcode Matched at index 28: 01001
# Opcode Matched at index 29: 00001
#
#####
# Scoreboard summary: sent=30 received=30
```

Figure 57 - sideband TX TOP transcript

4.4.2 Sideband RX TOP

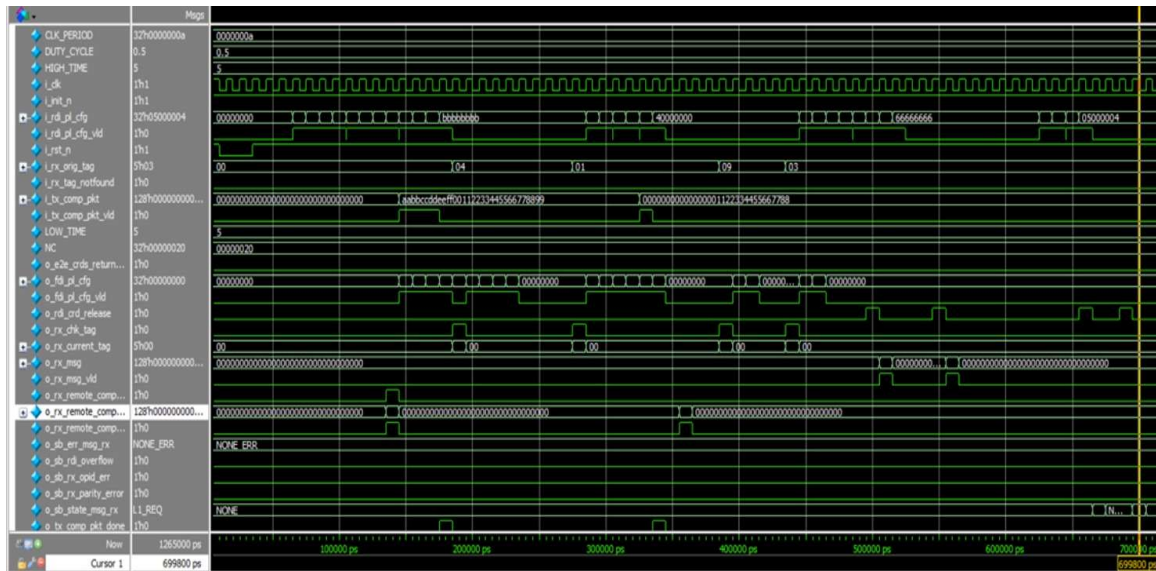
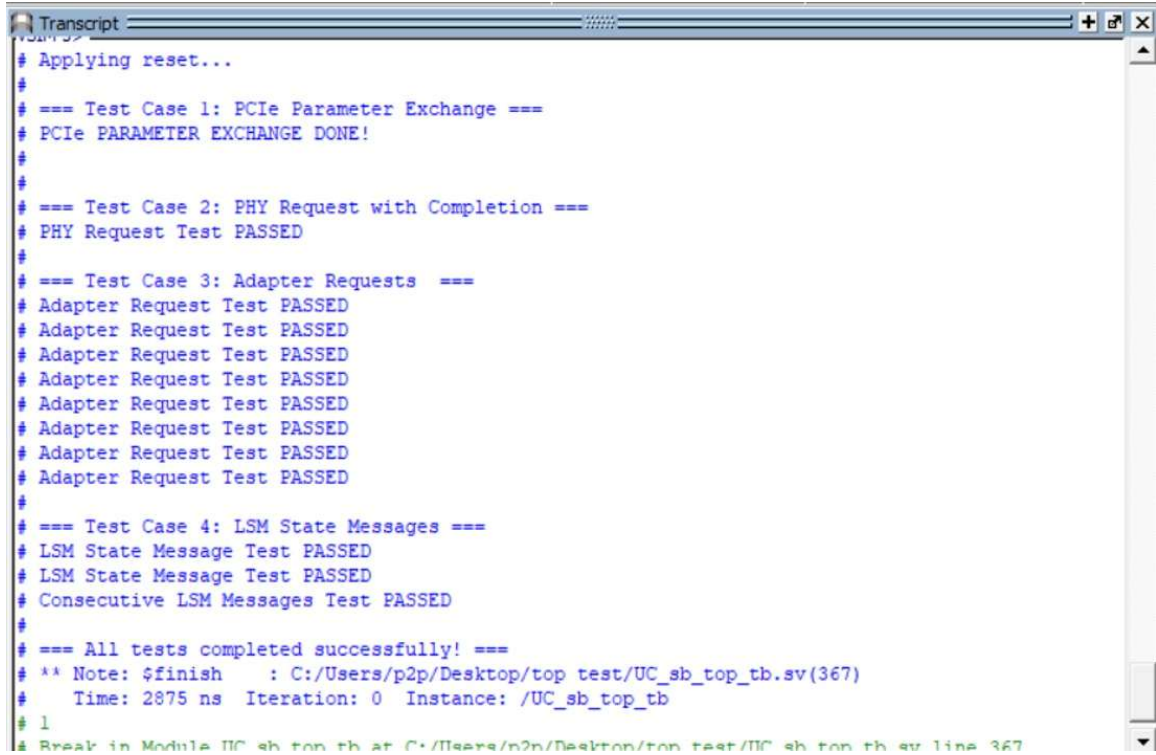


Figure 60 - sideband RX TOP waveform

```
VSIM 4> run -all
# [0] Applying HW reset...
# [550] HW reset released.
# [550] TC-1 START: Full Completion Packets
# [2850] TC-1 DONE
# [2850] TC-2 START: Half Completion Packets
# [4450] TC-2 DONE
# [4450] TC-3 START: Message Packets
# [6250] TC-3 DONE
# [6250] TC-4 START: LSM State Messages
# [7650] TC-4 DONE
# [12650] All test cases completed.
# ** Note: $stop      : C:/Users/p2p/Desktop/RX TEST/UC_rx_top_tb.sv(223)
#      Time: 1265 ns  Iteration: 0  Instance: /UC_rx_top_tb
# Break in Module UC_rx_top_tb at C:/Users/p2p/Desktop/RX TEST/UC_rx_top_tb.sv line 2
```

Figure 59 - sideband RX TOP transcript

4.4.3 Sideband Full Test



```
Transcript
# Applying reset...
#
# === Test Case 1: PCIe Parameter Exchange ===
# PCIe PARAMETER EXCHANGE DONE!
#
#
# === Test Case 2: PHY Request with Completion ===
# PHY Request Test PASSED
#
# === Test Case 3: Adapter Requests ===
# Adapter Request Test PASSED
# Adapter Request Test PASSED
# Adapter Request Test PASSED
# Adapter Request Test PASSED
# Adapter Request Test PASSED
# Adapter Request Test PASSED
# Adapter Request Test PASSED
# Adapter Request Test PASSED
#
# === Test Case 4: LSM State Messages ===
# LSM State Message Test PASSED
# LSM State Message Test PASSED
# Consecutive LSM Messages Test PASSED
#
# === All tests completed successfully! ===
# ** Note: $finish      : C:/Users/p2p/Desktop/top test/UC_sb_top_tb.sv(367)
#      Time: 2875 ns   Iteration: 0   Instance: /UC_sb_top_tb
# 1
# Break in Module UC_sb_top_tb at C:/Users/p2p/Desktop/top test/UC_sb_top_tb.sv line 367
```

Figure 61 - Sideband TOP transcript

Chapter 5: sub-block: ALSM & Register File

5.1 Block Diagram

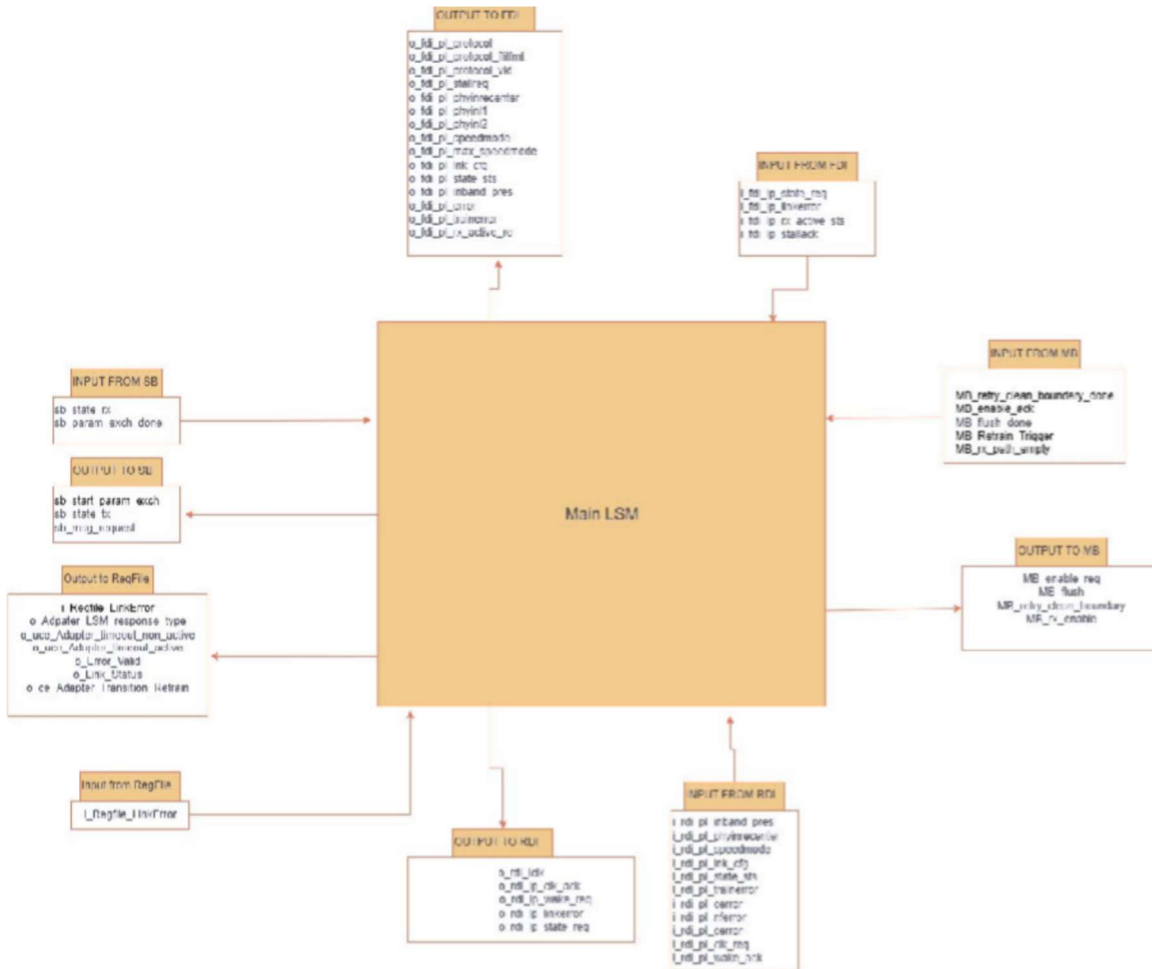


Figure 62 – ALSM block interfaces

5.2 Purpose

The Adapter Link State Machine (ALSM) is a foundational component of the Die-to-Die (D2D) Adapter responsible for coordinating high-level link management and synchronization between the local and remote dies. It serves as the primary mechanism for state negotiation, using dedicated sideband messages to ensure that both link partners reach a common understanding of the link's operational status.

The ALSM acts as a critical abstraction layer that consolidates the diverse states of the underlying hardware into a unified status for the upper Protocol Layer. It effectively shields the Protocol Layer from the Physical Layer's sub-states (which are themselves abstracted via the RDI state machine) while providing a consistent interface. This consolidation allows the system to manage transitions between complex states like Power Management (L1/L2)

or Retrain by ensuring that all internal buffers, such as the Retry buffer, are properly flushed/emptied and aligned at clean boundaries before the state change is finalized across

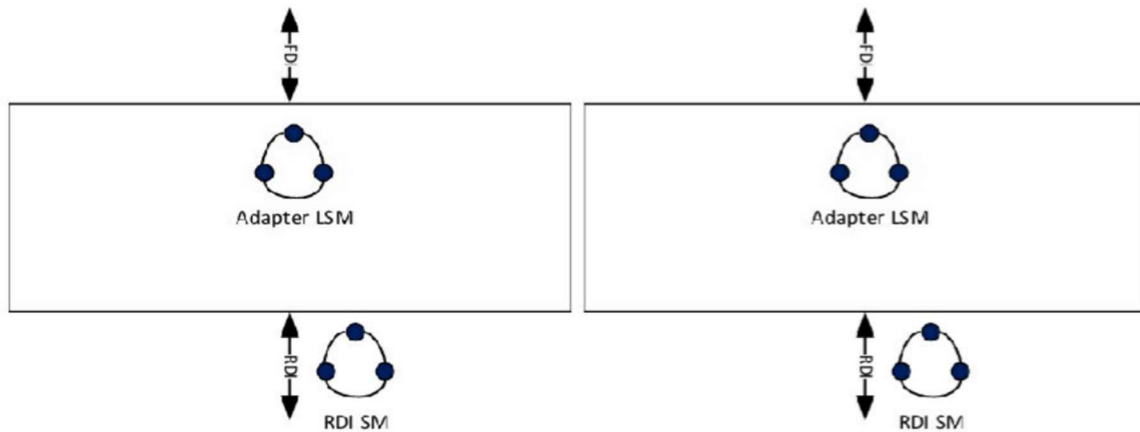


Figure 63 – LSM of adapter layer

the dies. Each die has its own FDI and RDI state machine like so:

Each State machine consists of the following states as per spec:

RDI State Machine

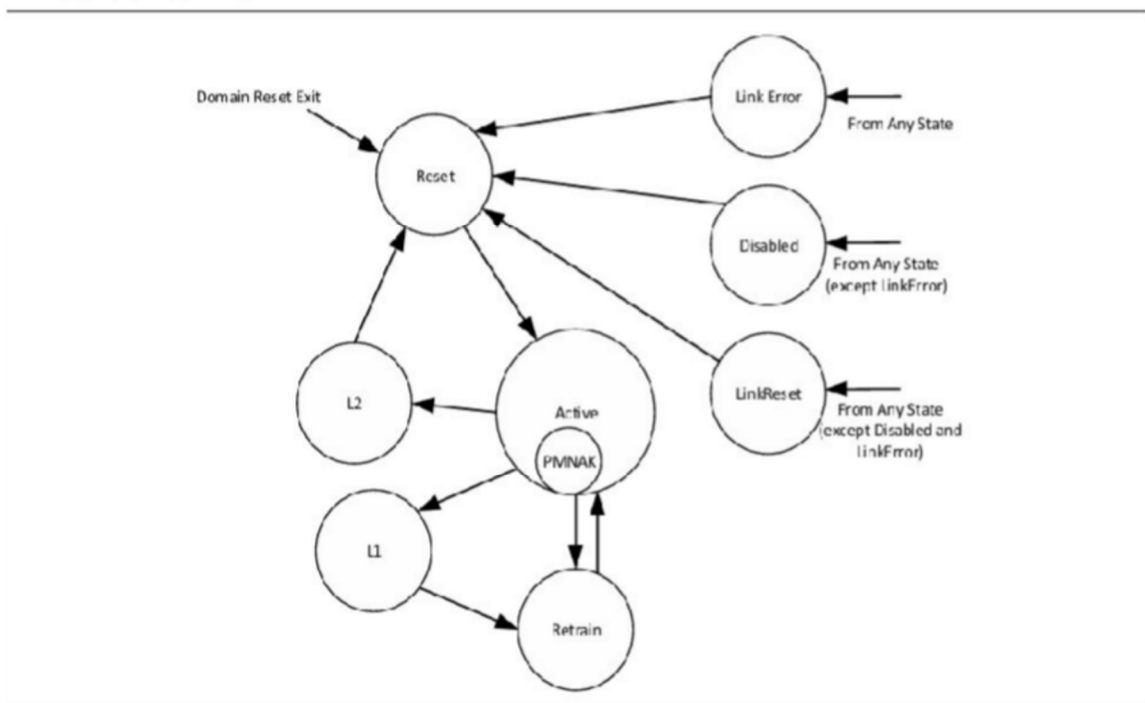


Figure 64 -RDI State Machine

FDI State Machine

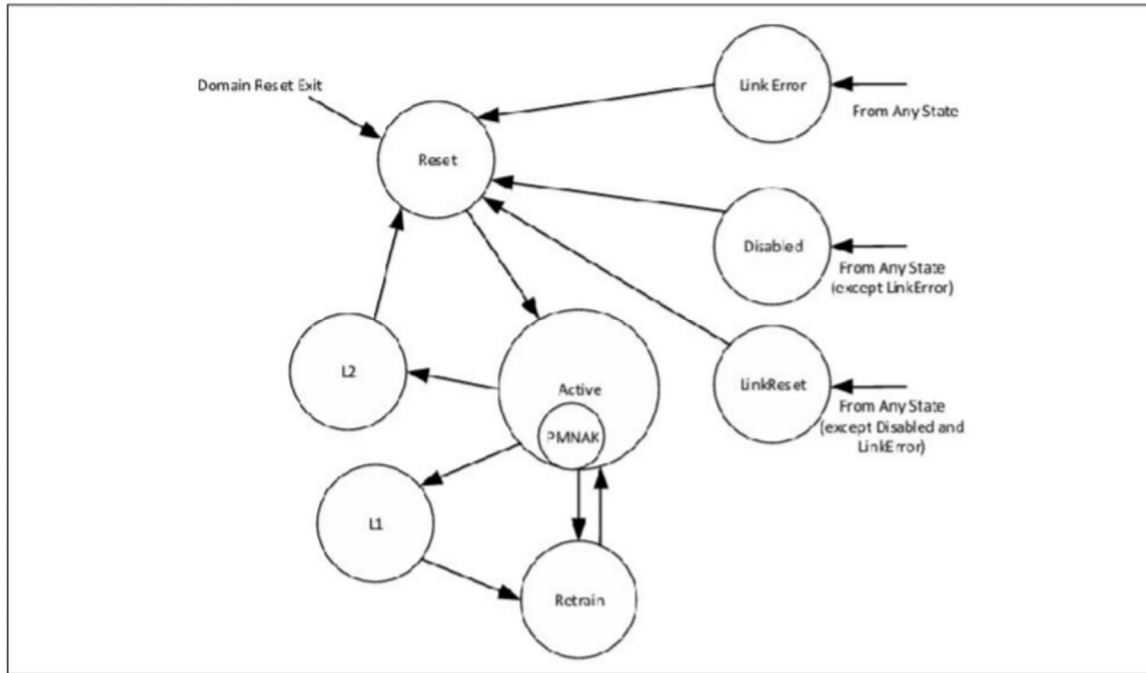


Figure 65 – FDI State Machine

The FDI and RDI state machines are the same although they represent different layers (the Protocol and PHY layer respectively). State transitions are to be negotiated on the FDI and RDI on the same die and on the Sideband for remote die state transition.

5.3 Functionality

FDI and RDI: the Adapter LSM block is responsible for handling the RDI and FDI state machine hierarchy. Implementing all states for RDI and FDI according to specification. **Mainband:** It controls the MB block for each state (when to start and when to halt communication) depending on the ALSM specification (Total Halt, Active Exit, Retrain Entry, PM Entry, Active Entry, LinkDown and LinkError). **Sideband:** The ALSM uses the Sideband block to communicate with the Remote link partner for State Transition messages and parameter exchange initialization. **Register File:** The Register File is used as a global error handler and is therefore given all possible errors that the ALSM can generate. The ALSM also uses the Uncorrectable Error Signal given out by the Register File to transition To/From LinkError State.

5.4 Signal Description

Port	Direction	Width	Description
i_rdi_pl_inband_pres	Input	1	RDI can only request active when this signal is high
i_rdi_pl_phyinrecenter	Input	1	RDI indication that phy is in train or retrain
i_rdi_pl_speedmode	Input	3	To be registered and given to the FDI
i_rdi_pl_lnk_cfg	Input	3	To be registered and given to the FDI
i_rdi_pl_state_sts	Input	4	Reflects the state of the RDI

i_rdi_pl_clk_req	Input	1	Phy starts the four way handshake of rdi ungating
i_rdi_pl_wake_ack	Input	1	Adapter starts the four way handshake of rdi ungating
i_rdi_pl_stall_req	Input	1	RDI request to adapter to stop transmission
i_rdi_pl_error	Input	1	PHY indication of error
i_rdi_pl_trdy	Input	1	PHY data backpressure on Adapter transmitter
o_rdi_lclk	Input	1	Clock at which the rdi interface operates
o_rdi_lp_clk_ack	Output	1	Registered rdi_pl_clk_req
o_rdi_lp_wake_req	Output	1	Adapter starts the four way handshake of rdi ungating (tied to 1b)
o_rdi_lp_linkerror	Output	1	Indication to PHY error occurred
o_rdi_lp_state_req	Output	4	RDI state request to PHY
o_rdi_lp_stall_ack	Output	1	Adapter confirmation to PHY that transmission stopped
i_fdi_lp_state_req	Input	4	FDI state request to Adapter
i_fdi_lp_linkerror	Input	1	Indication to Phy error occurred
i_fdi_lp_rx_active_sts	Input	1	Reflects the state of the Rx path of the Protocol Layer
i_fdi_lp_stall_ack	Input	1	Indicates that Protocol has stopped transmitting on a flit boundary
i_fdi_lp_clk_ack	Input	1	Indication that FDI is ungated
i_fdi_lp_wake_req	Input	1	Protocol requests ungating of FDI
o_fdi_pl_stallreq	Output	1	Stall signal on FDI when entering PM
o_fdi_pl_phyinrecenter	Output	1	(FDI in retrain OR rdi_pl_phyinrecenter) and lp_clk_ack
o_fdi_pl_phyinl1	Output	1	Indication when phy in L1 PM
o_fdi_pl_phyinl2	Output	1	Indication when phy in L2 PM
o_fdi_pl_speedmode	Output	3	Registered fdi_pl_speedmode
o_fdi_pl_max_speedmode	Output	1	Registered fdi_pl_max_speedmode
o_fdi_pl_lnk_cfg	Output	3	Registered o_fdi_pl_lnk_cfg
o_fdi_pl_state_sts	Output	4	Indication of FDI state
o_fdi_pl_inband_pres	Output	1	FDI can only request Active if this signal is high and clk_req handshake is done
o_fdi_pl_rx_active_req	Output	1	Requests that protocol opens its Rx path
o_fdi_pl_clk_req	Output	1	Tied to 1
o_fdi_pl_wake_ack	Output	1	Registered fdi_lp_wake_req
i_sb_state_rx	Input	4	SB message to be sent
i_sb_param_exch_done	Input	1	SB indication that parameter exchange is done
o_sb_start_param_exch	Output	1	ALSM trigger to start parameter exchange
o_sb_msg_request	Output	1	ALSM trigger to send SB message
o_sb_state_tx	Output	4	ALSM state request over the sideband
i_MB_retry_clean_boundary_done	Input	1	MB indication that a clean boundary is reached and pl_trdy is asserted
i_MB_flush_done	Input	1	Indication that the Tx retry buffer is empty
i_MB_Retrain_Trigger	Input	1	MB trigger for the ALSM to enter Retrain
i_MB_rx_path_empty	Input	1	Indication that the rx path of the MB is empty
o_MB_flush	Output	1	ALSM trigger for the MB to deassert trdy and flush

			the retry buffer
o_MB_retry_clean_boundary	Output	1	ALSM trigger for the MB to assert trdy and reach a flit boundary and stop transmitting
o_MB_tx_enable	Output	1	ALSM for the MB to enable the TX path
o_MB_rx_enable	Output	1	ALSM for the MB to enable the RX path
i_Regfile_LinkError	Input	1	Global Uncorrectable Error Signal
i_Regfile_Start_Retrain	Input	1	SW retrain through Register File
o_Adapter_LSM_response_type	Output	3	Error code of the ALSM transition
o_uce_Adapter_timeout_non_active	Output	1	Timeout not in active state
o_uce_Adapter_timeout_active	Output	1	Timeout in non-active state
o_Error_Valid	Output	1	Error Valid trigger
o_Link_Status	Output	1	UP/DOWN status
o_uce_Adapter_Transition_Retrain	Output	1	Indicator when ALSM goes to retrain

Table 53 - singal description of ALSM block interface

5.5 Implementation

5.5.1 Link Bringup

Link Bringup is the process of taking a link from Reset to Active state. Bringup process must occur on the FDI and RDI state machines like so for the Link to start Active like so:

Figure 10-20. FDI Bring up flow

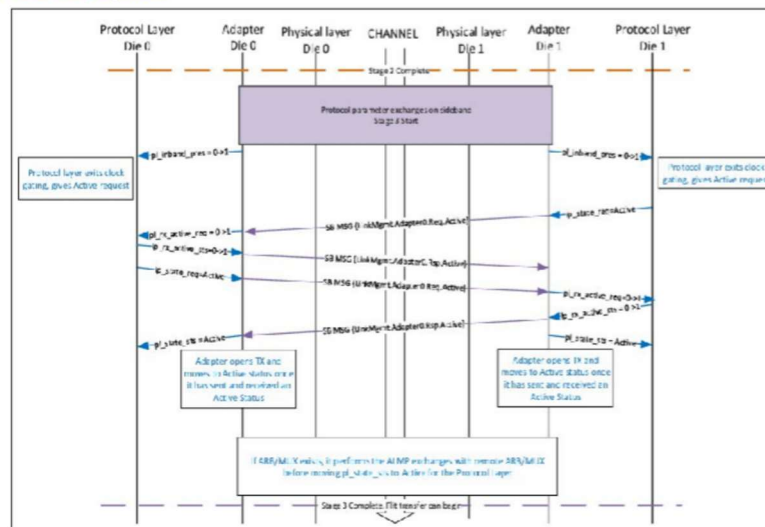


Figure 66 – Link bringup flow on RDI

Note that there is a separate RDI and FDI bringup flows where the RDI flow is negotiated by the PHY and reflected on the RDI interface and the FDI flow is negotiated by the Adapter Sideband messages.

The implemented flow in the ALSM is like the follow

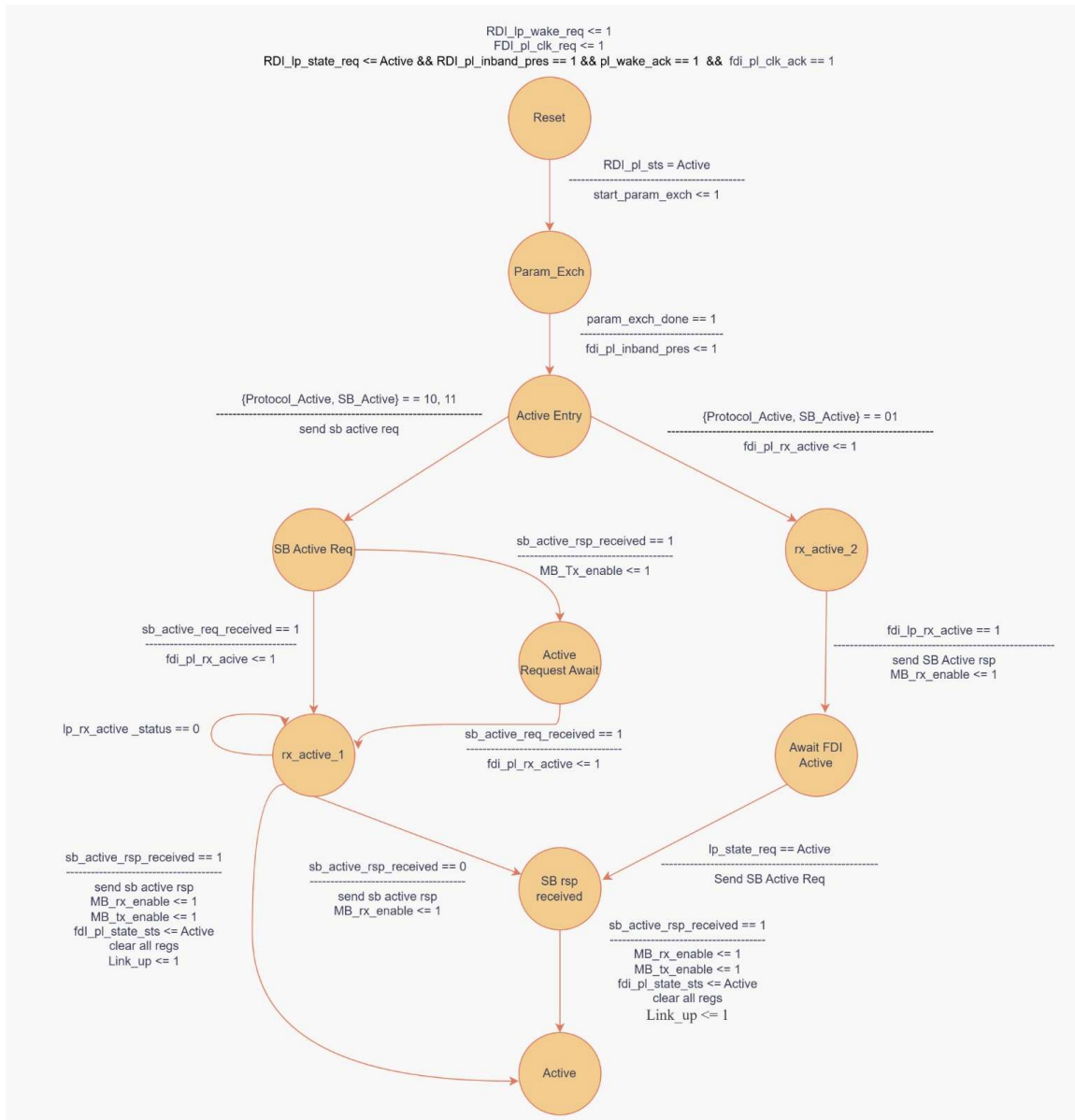


Figure 68 – Bringup implemented flow

Our ALSM implementation consolidates the RDI and FDI bringup flows and ensures proper activation of Mainband Tx and Rx paths after parameter exchange while accounting for different scenarios concerning the order at which the Sideband Messages arrive.

5.5.2 Retrain

Retrain is sometimes necessary when operating the Mainband. Cases which they are are when there are many CRC errors or many flits are replayed due to errors. Retrain can be started from the remote link partner or from the Mainband.

In order to train/retrain the link the Mainband must be off. This is achieved by communicating to the RDI about turning off the Tx path on a flit boundary (not to send partial flits) and then enter Retrain once the Rx path is empty. Retrain State machine is implemented like the following:

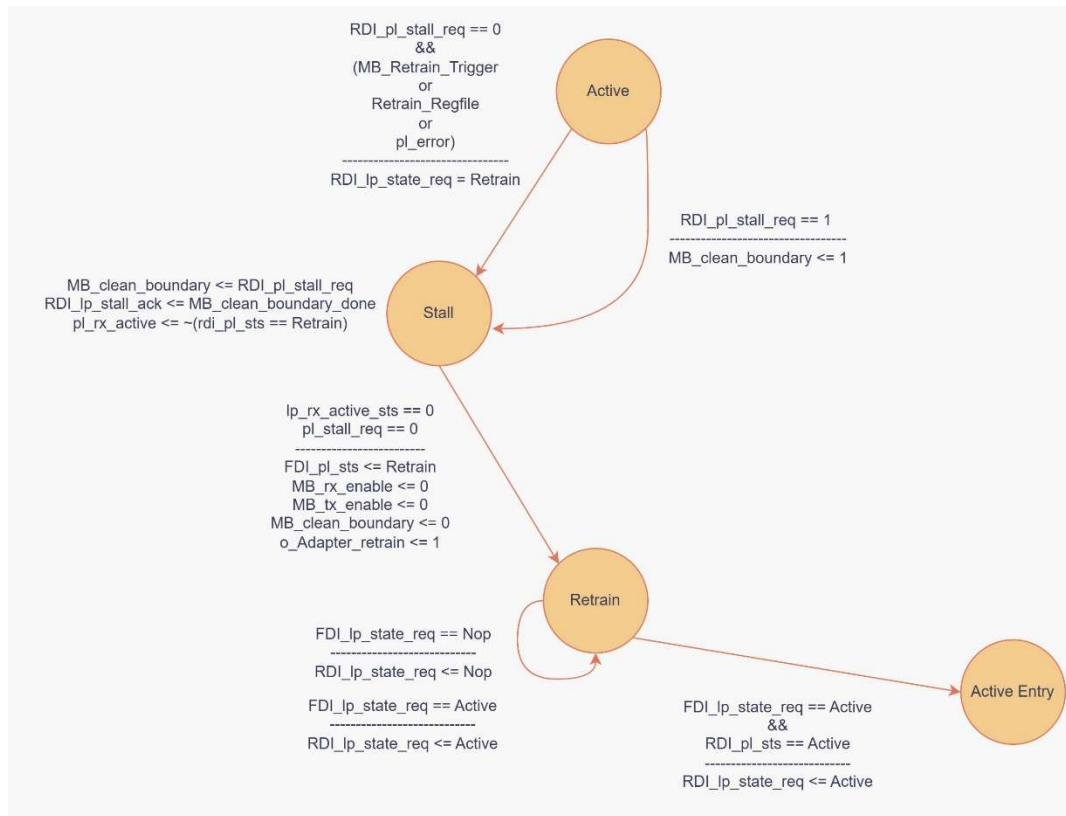


Figure 69 – Retrain Implemented flow

Retrain can only be entered by the stall_req/ack handshake offered by the RDI, this ensures that the Tx path is empty. To go back to Active the Protocol Layer must first request it on the lp_state_req signal on the FDI and then the state machine can go to active but NOT redoing parameter exchange.

5.5.3 LinkError

The UCIe offers a state for when Uncorrectable errors occur in a Link. Sources of Errors can be invalid opcodes on the Sideband, framing errors from the PHY or the Mainband, Parity Errors from the Sideband, overflow Errors from the Sideband or Mainband, and many other Link-Defined cases.

LinkError can be requested from upper layers, but entered from the bottom layers first and then LinkError entry is propagated upwards. To enter LinkError state, the Mainband must flush the Tx buffer before being disabled. An example of LinkError from the spec is:

Figure 10-32. LinkError example

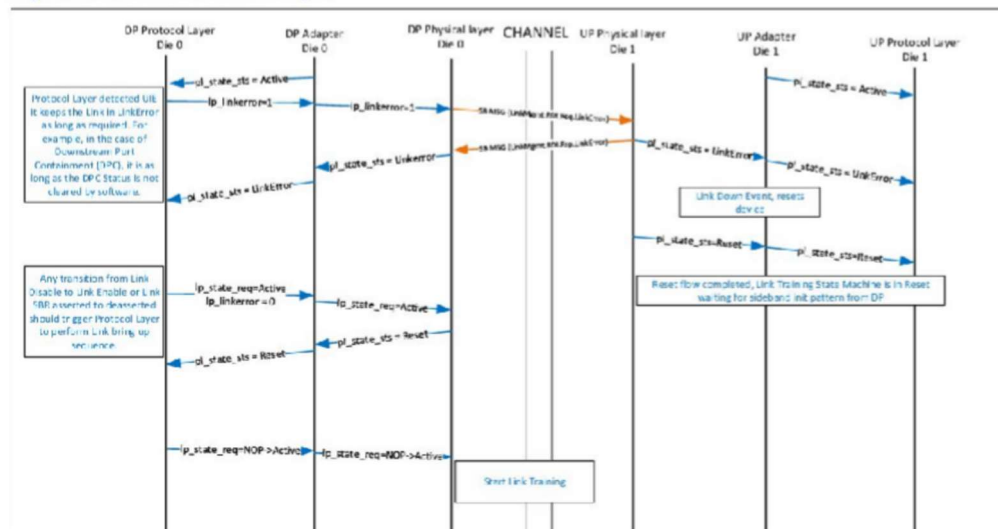


Figure 70 – LinkError flow

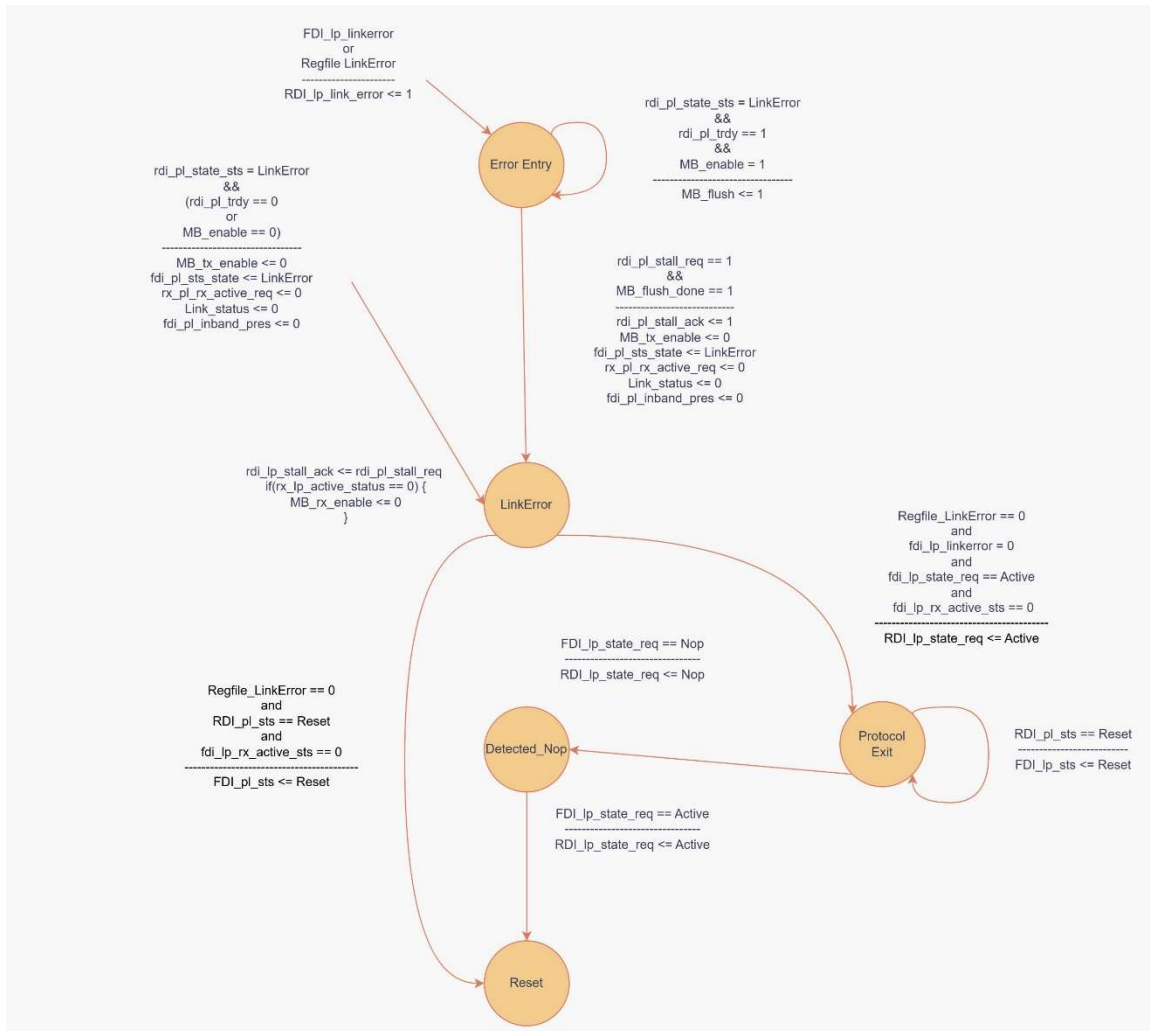


Figure 71 – LinkError Implemented flow

To exit LinkError the Software MUST clear out the Uncorrectable Error Registers in the Register file and the Protocol layer MUST stop requesting LinkError on lp_linkerror.

5.5.4.2 Disable

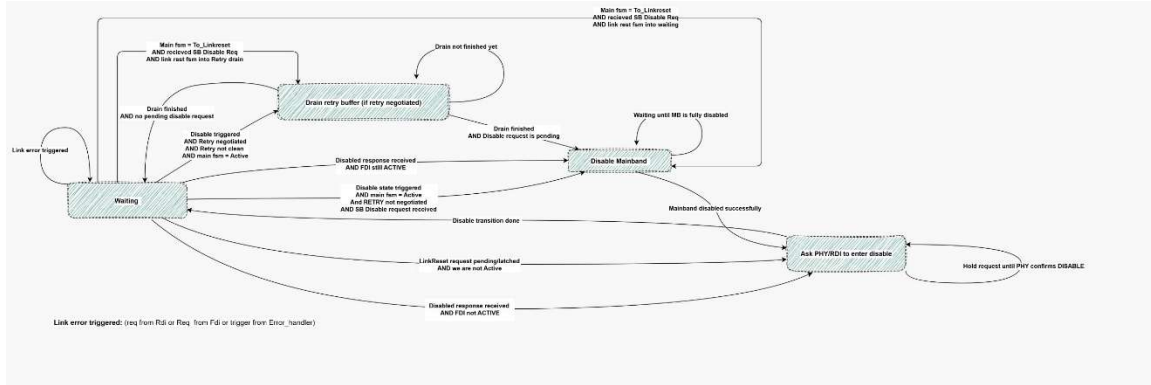


Figure 74 – LinkDisable implemented flow

The following is an example of the ALSM module simulation going from and to various

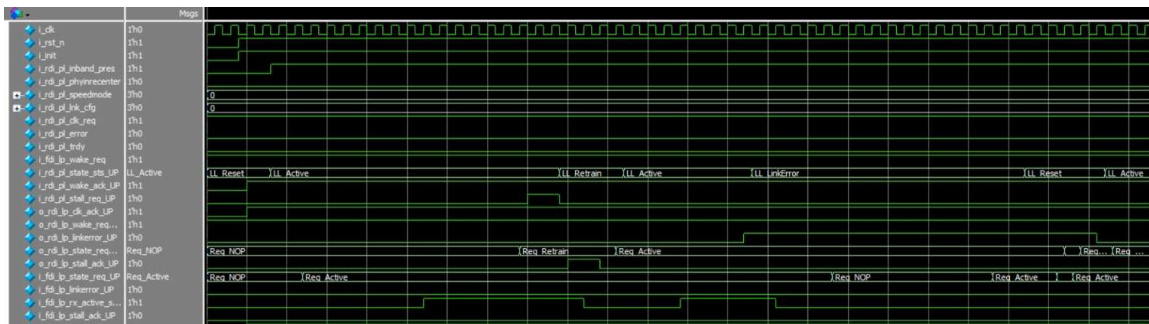


Figure 75 – ALSM TOP level waveform 1

states, testing scenarios according to specification and scenarios given in the spec.

following is an example of the ALSM module simulation going from and to various states, testing scenarios according to specification and scenarios given in the spec.

Adhering to spec requires scenarios where a D2D layer sends a request expecting a response but receiving a request in return. This can be because the other die has a request of the same type at the same time or sent a different type of request.

Active Entry, PARAM exchange should be fulfilled and active entry handshake between dies be done.

This gives a starting signal for the Mainband to start the Sequence Number Handshake

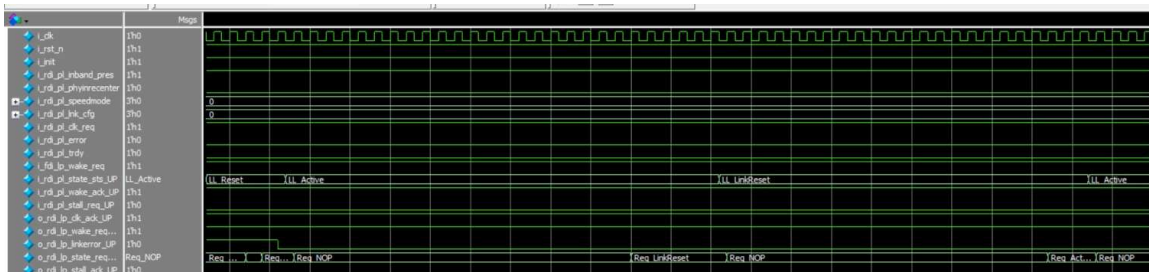


Figure 76 – ALSM TOP level waveform 2

For Retrain, the ALSM must signal to the MB to stop at a flit boundary before communicating with RDI the Entry to Retrain. Once Retrain is finished (RDI signals this) the ALSM returns to Active.

For LinkError the MB can either flush or just reset. To enter Reset, the SW must clear the Error registers using the SB interface, this in turn gets signaled to the ALSM.

Down-States are entered according to the `LinkError` example in the specification. This is to be followed on both RP and EP and accounted for in the MB interface. MB should honor the drain signal and start draining its Retry Buffer.

5.6 Overview of Register File

Register file architecture is designed to seamlessly leverage existing PCIe and CXL software investments while introducing the specific controls required for multi-die, chiplet-based configurations. To achieve this, the register file is split into two primary access domains: standard Configuration Space (accessed via configuration reads and writes) and Memory-Mapped I/O (MMIO) Space (accessed via standard memory reads and writes).

5.6.1 Register Discovery and Architectural Mapping

UCIe-aware software discovers the presence and capabilities of UCIe Links using specific discovery tables and capabilities mapped to different device types: Host Root Ports (UiRB): On the host side, registers reside in a statically located, host-specific block called the UCIe Register Block (UiRB). Each UCIe Link has a separate UiRB base address, which is enumerated to the Operating System via the ACPI-standardized UCIe Early Discovery Table (UEDT). Switch Downstream Ports (UiSRB): Registers reside in the UCIe Switch Register Block (UiSRB). The base address for the UiSRB is provided within the UiSRB DVSEC capability located in the Switch's Upstream Port (Dev0/Fn0). Endpoints: The basic UCIe Link capabilities are discovered via the UCIe Link DVSEC located in Dev0/Fn0 of the Endpoint.

5.6.2 Register Attributes and Field Types

UCIe registers utilize strict attributes to define software access rules, ensuring safe chiplet operation: RO (Read Only) & ROS (Read Only Sticky): Bits cannot be altered by software. Sticky bits preserve their values even if the link goes down, resetting only upon a Domain Reset. RW (Read-Write) & RW1C (Read-Write-One-To-Clear): Standard read/write bits. RW1C bits indicate status events and are cleared by writing a 1b. RWL (Read-Write Lock) & RWO (Read-Write-One-To-Lock): These follow standard RW behavior until they are locked, after which software cannot alter them. HwInit (Hardware Initialized): Used for capabilities that vary based on package integration (e.g., maximum link speed achievable based on routing). These are configured by package-level straps or firmware. RsvdP (Reserved and Preserved) & RsvdZ (Reserved and Zero): Reserved fields. RsvdZ requires software to write 0b, while RsvdP requires software to preserve the previously read value.

5.6.3 Configuration Space: UCIe Link DVSEC

The core configuration structure is the UCIe Link DVSEC (Designated Vendor-Specific Extended Capability) as it allows software to differentiate a UCIe port from a standard PCIe or CXL port. UCIe Link Capability & Control: Identifies supported Flit Formats (e.g., 68B, Standard 256B, Latency-Optimized 256B), maximum link width (up to x256), maximum link speeds (up to 64 GT/s), and multi-protocol capabilities. The Control register houses the crucial Start UCIe Link Training bit used to initiate Link bring-up. UCIe Link Status: Reports the currently negotiated Link speed, negotiated width, and real-time Link status (Up/Down). It also tracks hardware-autonomous bandwidth changes. Error and

Event Notification Control: Configures whether link events and correctable/uncorrectable errors trigger interrupt messages (like MSI/MSI-X) or sideband error messages.

5.6.4 MMIO Space: D2D Register Block

D2D Adapter Registers (First 4 KB): Manages protocol multiplexing, Retry mechanisms, and parameter exchanges. It includes: Error Status and Masks: Dedicated registers for Uncorrectable Errors (e.g., Adapter Timeout, Receiver Overflow) and Correctable Errors (e.g., CRC Error, Runtime Link Testing Parity Error). Header Logs: The Header Log 1 and Header Log 2 registers capture the headers of failed sideband accesses and specific error syndromes. Capability Logs: Read-only logs capturing the exact Advertised and Finalized capabilities for Adapter, CXL, and Multi-Protocol parameter exchanges.

5.6.5 Remote Access via the Sideband Mailbox Mechanism

Registers residing on a remote Link partner or an off-package UCIe Retimer cannot be directly accessed by host memory reads/writes. Instead, they are accessed via the Sideband Mailbox Mechanism located within the local UCIe Link DVSEC. Software populates the Sideband Mailbox Index registers with the target address offset and operational Opcode (e.g., 32-bit/64-bit Memory Read/Write). If performing a write, software places the payload in the Sideband Mailbox Data registers. Software then sets the Write/Read Trigger bit in the Sideband Mailbox Control register. The Adapter automatically translates this into a sideband packet, sends it across the 800 MT/s sideband link, and clears the trigger bit once the access completes successfully or fails with a Completer Abort (CA) or Unsupported Request (UR).

UCIe Link DVSEC

PCI Express Extended Capability Header		
Designated Vendor Specific Header 1		
Capability Descriptor	Designated Vendor Specific Header 2	
UCIe Link Capability		
UCIe Link Control ^f		
UCIe Link Status		
Error Notification Control	Link Event Notification Control	
Register Locator 0 Low		
Register Locator 0 High		
...		
...		
Reserved		
Sideband Mailbox Index Low		
Sideband Mailbox Index High		
Sideband Mailbox Data Low		
Sideband Mailbox Data High		
Reserved	Sideband Mailbox Status	Sideband Mailbox Control
Requester ID/Reserved		
Reserved		
Associated Port Numbers (1-N)		
...		

- a. Applies to UCIe-EP, UCIe-USP, UCIe-Retimer.
b. Applies to UCIe-EP, UCIe-USP when paired with a retimer.
c. Applies to UCIe-RP.
d. Applies to UCIe-DSP.
e. Software writes to this register need to be broadcast to both D2D Adapter and PHY blocks because some registers could be implemented in either block or both blocks.

Figure 77 - UCIe DVSEC

5.6.6 UCIe Registers Offsets

Register Name	Address Offset
PCI Express Extended Capability Header	0h
Designated Vendor Specific Header 1	4h
Designated Vendor Specific Header 2	8h
Capability Descriptor	Ah
UCIe Link Capability	Ch
UCIe Link Control	10h
UCIe Link Status	14h
Link Event Notification Control	18h
Link Event Notification Control	1Ah

Register Locator 0 Low	1Ch
Register Locator 0 High	20h
Register Locator 1 Low	24h
Register Locator 1 High	28h
Sideband Mailbox Index Low	3Ch
Sideband Mailbox Index High	40h
Sideband Mailbox Data Low	44h
Sideband Mailbox Data High	48h
Sideband Mailbox Control	4Ch
Sideband Mailbox Status	4Dh
Requester ID/Reserved	4Fh
UCIe Register Block Header	0h
Uncorrectable Error Status Register	10h
Uncorrectable Error Mask Register	14h
Uncorrectable Error Severity Register	18h
Correctable Error Status Register	1Ch
Correctable Error Mask Register	20h
Header Log 1 Register	24h
Header Log 2 Register	2Ch
Error and Link Testing Control Register (Offset 30h)	30h
Advertised Adapter Capability Log	54h
Finalized Adapter Capability Log	5Ch
Advertised CXL Capability Log	64h

Finalized CXL Capability Log	6Ch
UCIe Register Block Header	0h
D2D Implementation interrupt	10
UCIe Soft Reset	14h
Retry Control Register	18h
Remote Adapter Capability	1Ch
Remote CXL Capability	24h

Table 54 - UCIe Register File Offset

Chapter 6: Adapter Top Level Integration & Verification

6.1 Overview of Final Architecture

The architecture is divided into Mainband, Sideband, ALSM, FDI, RDI, and a Register File

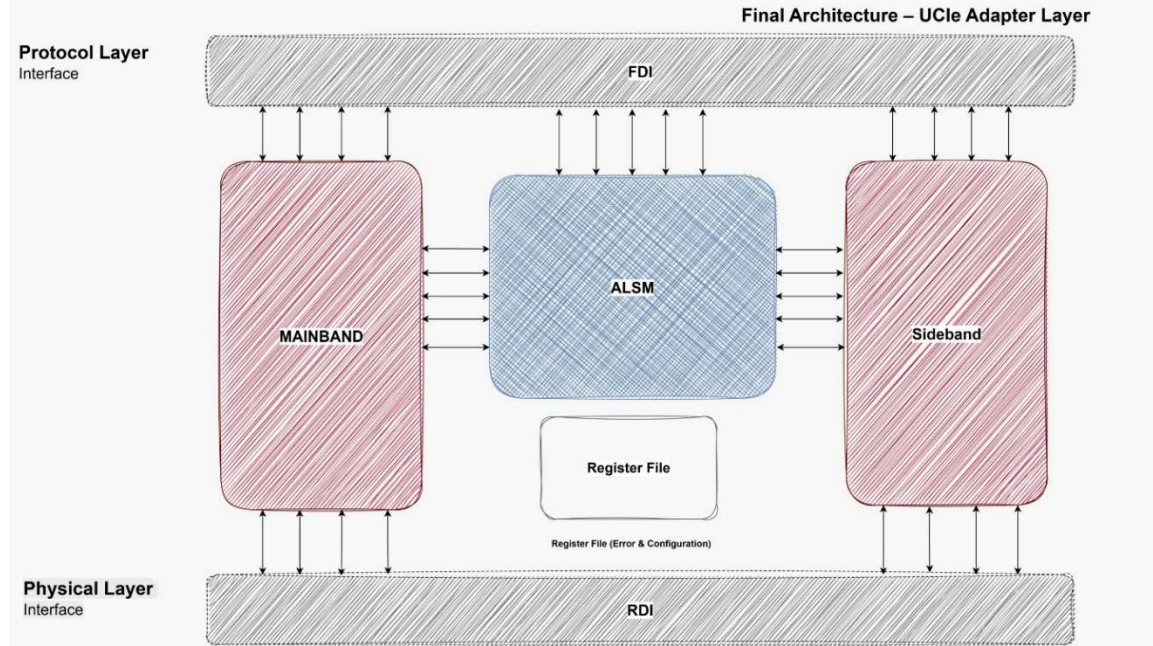


Figure 78 – UCle Implementation Top level

to separate control functions from data transfer functions and to reflect the UCle layered initialization model. Sideband provides the control and negotiation channel used for initialization, capability exchange, state synchronization, power management, and recovery, while Mainband carries the operational payload after the link becomes active. ALSM serves as the central coordination block that supervises state transitions and connects protocol-facing behavior on FDI with PHY-facing behavior on RDI. The Register File stores configuration, status, and error information, allowing the design to remain modular, configurable, and easier to verify. Mainband: This is the primary data path of the UCle link, responsible for transferring high-bandwidth protocol data (such as PCIe or CXL flits). Ensure error control and flow control through Retry mechanism from PCIe Rule book, and optimized and modified for UCle specs. Sideband: This low-speed connection is used for parameter exchanges, register accesses (for debug/compliance), and coordinating link training and management with the remote partner. LSM (Link State Machine): The Adapter LSM manages and coordinates the operational and power states (such as Active, PM, Retrain, LinkError) of the link with the remote partner via sideband messages. Register File / Register Blocks: These memory-mapped configuration spaces store link capabilities, operational control settings, and error logs. They are accessed by system software or via the sideband mailbox mechanism for initialization, telemetry, and debugging. FDI (Flit-Aware Die-to-Die Interface): This is the standardized interface that connects the Protocol Layer to the Die-to-Die (D2D) Adapter. RDI (Raw Die-to-Die

Interface): This is the standardized interface that connects the Die-to-Die (D2D) Adapter to the Physical Layer.

The proposed design operates as a unified layered system in which Sideband, ALSM, Mainband, FDI, and RDI cooperate to complete link bring-up and runtime data transfer. Sideband is responsible for initialization, capability negotiation, and management coordination between link partners, while RDI abstracts the Physical Layer bring-up, training, and repair process. ALSM uses the outcomes of these two domains to control link-state progression and determine when the Adapter may transition to the Active state. Once the link is fully initialized, FDI exposes the Adapter to the Protocol Layer and Mainband carries the operational traffic, with Sideband remaining available for management, retrain, and power-transition support throughout normal operation.

6.2 Testing Methodology

To ensure the correct behavior of our UCle Adapter Layer implementation, we adopted a structured testing methodology that involved multiple verification stages:

- Stage 1: Unit Testing of Internal Sub-blocks of Main Blocks Each internal sub-block such as Mainband, Sideband, and LSM sub-blocks was tested across all possible scenarios to validate its individual functionality.
- Stage 2: Block-Level Integration Testing After verifying each sub-block, the internal modules of every main block were integrated and tested to ensure that the main blocks like Mainband, Sideband, and LSM blocks function correctly as cohesive units.
- Stage 3: Full Adapter Layer Integration Testing Finally, the complete Adapter Layer was assembled and tested to validate the correctness of interconnections and interactions between the main blocks.

Stage 1 and 2 were illustrated in each block chapter, In the final stage of verification (stage 3), two instances of the UCle Adapter Layer are instantiated to interact with each other, simulating a complete end-to-end communication path. For each instance, the Protocol Layer and Physical Layer are modeled within the testbench to replicate realistic operating conditions.

6.3 Results

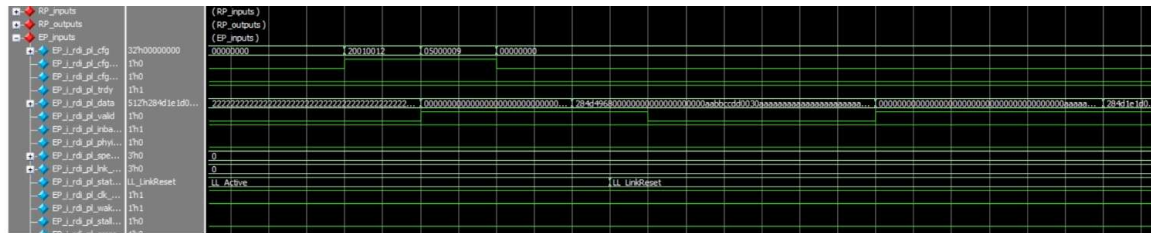


Figure 79 - top module waveform 1

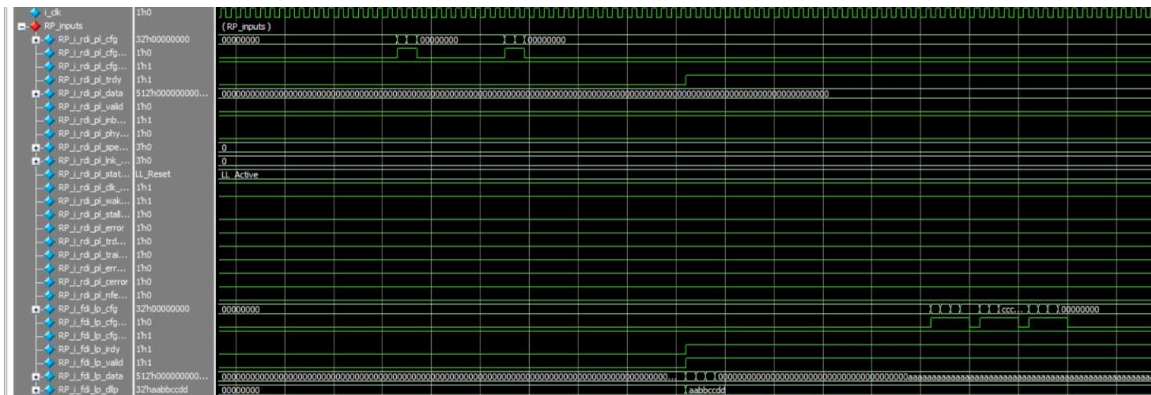


Figure 80 - top module waveform 2

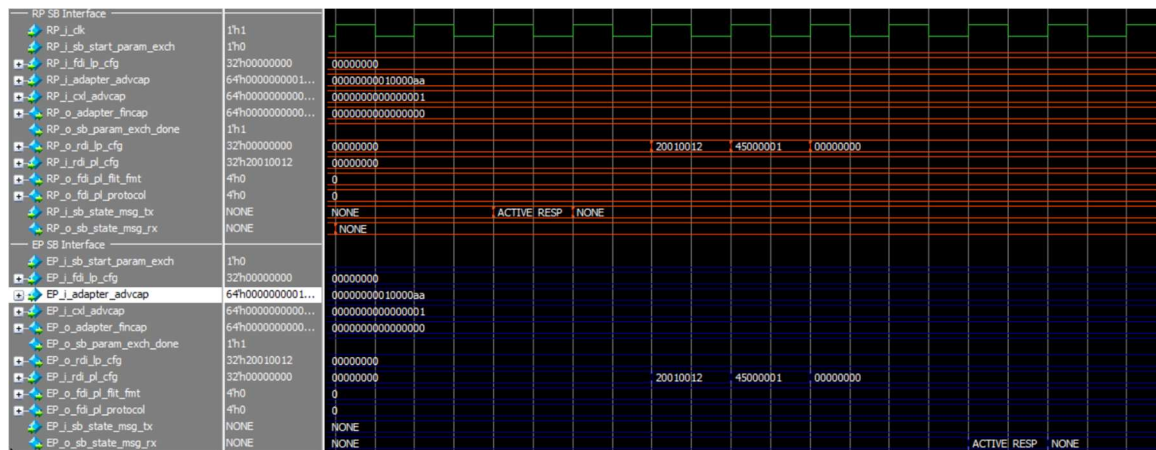


Figure 81 - active bringup on top level 1



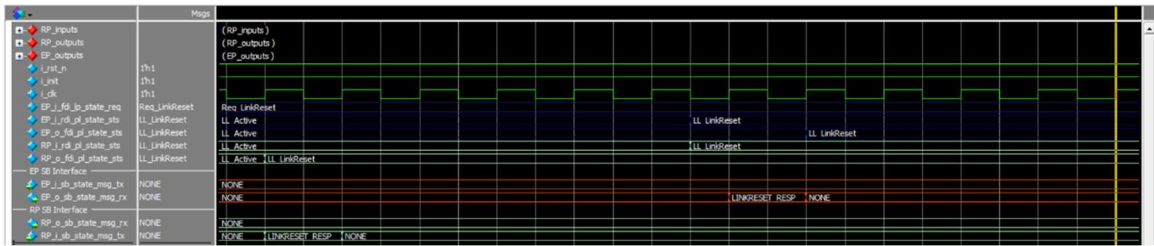


Figure 88 - linkreset entry and exit 3

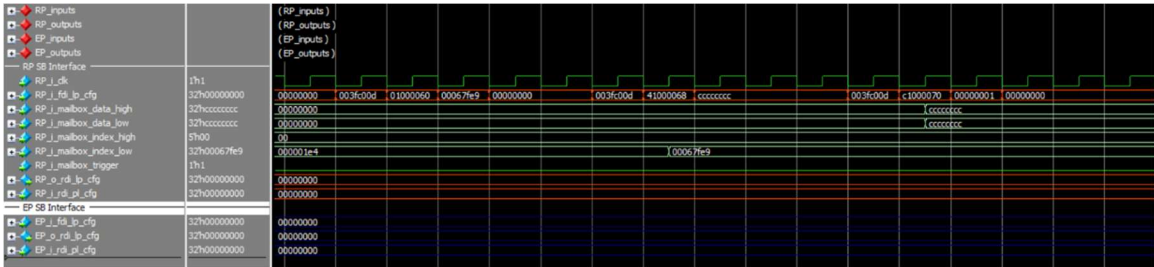


Figure 87 - mailbox on top level

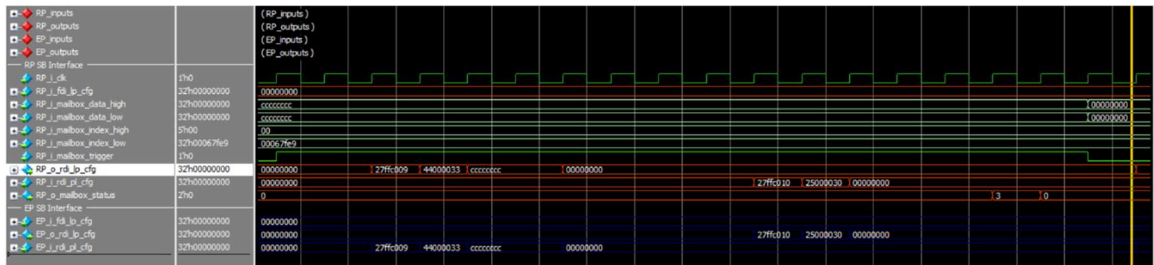


Figure 86 - request and completion on top level

Chapter 7: ASIC Flow

7.1 RTL Linting using synopsys Spyglass Linting Tool

This goal checks basic connectivity issues, such as: Floating inputs and Width mismatches.

This goal checks for simulation issues, such as: Incomplete sensitivity lists. Incorrect use of blocking/non-blocking assignments. Possible functional errors. Possible simulation hangs cases. Simulation race cases.

This goal identifies structural issues in the design that affect the post implementation performance and/or functionality of the design, examples of this include: Multiple drivers. High fan-in mux. Synchronous/asynchronous use of resets.

This goal reports un-synthesizable constructs in the RTL code.

This goal reports code that can cause RTL vs gate-level simulation mismatch.

7.1.1 Linting Results for Adapter Architecture

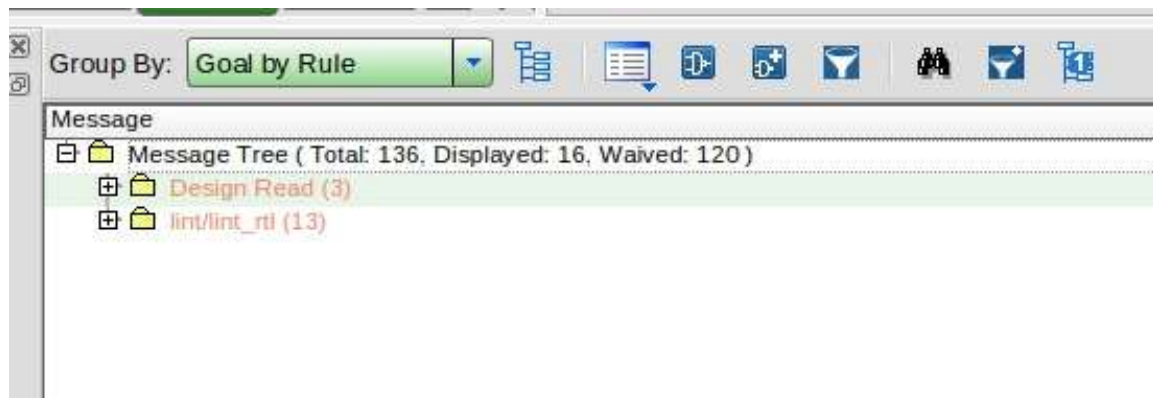


Figure 89 - RTL linting result

7.2 Synthesis

Logic synthesis is a critical phase in the digital design process that transforms RTL (Register Transfer Level) descriptions written in System Verilog into a gate-level netlist based on the constraints and the characteristics of the target standard cell library.

In this project, we are using 130nm library with NAND Gate Size 4.7068 um squared.

7.2.1 Design Constraints

Constraint Type	Target	Command / Setting
DC Variable	Fix multiple port nets	set_fix_multiple_port_nets -all - buffer_constants -feedthroughs

Master Clock	i_clk (REF_CLK_NAME)	create_clock -period 3.34 -waveform {0 1.67} [get_ports i_clk]
Clock Uncertainty	i_clk hold	set_clock_uncertainty -hold 0.1 [get_clocks i_clk]
Clock Transition	i_clk fall	set_clock_transition -fall 0.05 [get_clocks i_clk]
Input Delay	Functional inputs (i_*, ram_i_* minus i_clk/i_rst_n)	set_input_delay 0.668 -clock i_clk [filtered ports]
Output Delay	Functional outputs (o_*, ram_o_*)	set_output_delay 0.668 -clock i_clk [all_o]
Driving Cell	Functional inputs	set_driving_cell -lib_cell BUFX2M -pin Y (ss_1p08v_125c lib) [filtered ports]
Output Load	Functional outputs	set_load 0.5 [all_o]
Operating Condition	Min (hold) library	set_operating_conditions -min_library ff_1p32v_m40c -min ff_1p32v_m40c

Table 55 - constraints without DFT

7.2.2 Results

clock i_clk (fall edge)	1.85	1.85
clock network delay (ideal)	0.00	1.85
clock uncertainty	-0.10	1.75
	0.00	1.75 f
time borrowed from endpoint	1.64	3.39
data required time		3.39

data required time		3.39
data arrival time		-3.39

slack (MET)		0.00

Figure 91 - setup time report before DFT

Timing Path Group 'i_clk'		Cell Count	
-----		-----	
Levels of Logic:	11.00	Hierarchical Cell Count:	217
Critical Path Length:	3.39	Hierarchical Port Count:	20633
Critical Path Slack:	0.00	Leaf Cell Count:	43132
Critical Path Clk Period:	3.70	Buf/Inv Cell Count:	7258
Total Negative Slack:	0.00	Buf Cell Count:	3813
No. of Violating Paths:	0.00	Inv Cell Count:	3445
Worst Hold Violation:	0.00	CT Buf/Inv Cell Count:	0
Total Hold Violation:	0.00	Combinational Cell Count:	23948
No. of Hold Violations:	0.00	Sequential Cell Count:	19184
-----		Macro Count:	0

Area

Combinational Area:	249679.271301
Noncombinational Area:	
	508559.141834
Buf/Inv Area:	30431.815705
Total Buffer Area:	18009.39
Total Inverter Area:	12422.42
Macro/Black Box Area:	0.000000
Net Area:	0.000000

Cell Area:	758238.413135
Design Area:	758238.413135

Figure 90 - qor report before DFT

7.3 Design For Testability (DFT)

Design for Testability (DFT) is a critical step in the development of the UCle Adapter Layer, ensuring that the final integrated circuit (IC) can be thoroughly and efficiently tested after fabrication. DFT techniques are incorporated during the design phase to enhance the observability and controllability of internal nodes, making it feasible to detect and diagnose potential manufacturing faults.

After synthesizing the entire design and generating the gate-level netlist of the UCle Adapter, the DFT flow is applied to insert additional test structures, such as scan chains. These structures significantly improve the accessibility of internal logic, allowing for comprehensive testing of all critical paths and logic elements.

The primary objective of DFT is to validate that the chip functions correctly post-fabrication and is free from physical defects. Common fault types targeted include: Stuck-at-1 and Stuck-at-0 faults – representing lines permanently held at logic high or low. Bridging faults – caused by short circuits between adjacent wires. Open circuits – resulting from broken connections during manufacturing.

To support DFT, Automatic Test Pattern Generation (ATPG) tools are employed to generate test vectors and corresponding testbenches. These are used to simulate and validate the test patterns on the synthesized netlist, helping ensure high fault coverage and manufacturing test quality. By embedding DFT structures and running ATPG before tape-out, designers can guarantee that the UCle Adapter Layer is both testable and reliable, significantly reducing production costs and post-silicon debugging effort.

7.3.1 DFT Design Constraints

Constraint Type	Target	Command / Setting
Master Clock	scan_clk (DFT_CLK_NAME)	create_clock -period 10 -waveform {0 5} [get_ports scan_clk]
Dont Touch	DFT_CLK network	set_dont_touch_network [get_clocks DFT_CLK]
Clock Uncertainty	DFT_CLK setup	set_clock_uncertainty -setup 0.025 [get_clocks DFT_CLK]
Clock Uncertainty	DFT_CLK hold	set_clock_uncertainty -hold 0.05 [get_clocks DFT_CLK]
Clock Transition	DFT_CLK rise	set_clock_transition -rise 0.1 [get_clocks DFT_CLK]

Clock Transition	DFT_CLK fall	set_clock_transition -fall 0.1 [get_clocks DFT_CLK]
Clock Latency	DFT_CLK	set_clock_latency 0 [get_clocks DFT_CLK]
Clock Relationship	i_clk vs DFT_CLK	set_clock_groups -asynchronous -group i_clk -group DFT_CLK
Input Delay	DFT inputs	set_input_delay 1.002 -clock DFT_CLK [get_ports {SI SE test_mode}]
Output Delay	DFT output	set_output_delay 1.002 -clock DFT_CLK [get_ports SO]
Driving Cell	DFT inputs	set_driving_cell -lib_cell BUFEX2M -pin Y [get_ports {SI SE test_mode}]
Output Load	DFT output	set_load 0.5 [get_ports SO]
Case Analysis	test_mode	set_case_analysis 0 [get_ports test_mode]
Case Analysis	SE	set_case_analysis 0 [get_ports SE]

Table 56 - constraints for DFT

7.3.2 DFT Results

clock i_clk (rise edge)	3.34	3.34
clock network delay (ideal)	0.00	3.34
clock uncertainty	-0.10	3.24
	0.00	3.24 r
library setup time	-0.22	3.02
data required time		3.02

data required time		3.02
data arrival time		-3.02

slack (MET)		0.00

Figure 93 - setup timing report for DFT

Area	Cell Count

Combinational Area: 384465.553806	Hierarchical Cell Count: 52
Noncombinational Area:	Hierarchical Port Count: 20138
640095.401531	Leaf Cell Count: 60362
Buf/Inv Area: 44775.788918	Buf/Inv Cell Count: 10752
Total Buffer Area: 24537.73	Buf Cell Count: 5160
Total Inverter Area: 20238.06	Inv Cell Count: 5592
Macro/Black Box Area: 0.000000	CT Buf/Inv Cell Count: 0
Net Area: 0.000000	Combinational Cell Count: 41343

Cell Area: 1024560.955337	Sequential Cell Count: 19019
Design Area: 1024560.955337	Macro Count: 0

Timing Path Group 'i_clk'	

Levels of Logic: 14.00	
Critical Path Length: 3.02	
Critical Path Slack: 0.00	
Critical Path Clk Period: 3.34	
Total Negative Slack: 0.00	
No. of Violating Paths: 0.00	
Worst Hold Violation: 0.00	
Total Hold Violation: 0.00	
No. of Hold Violations: 0.00	

Figure 92 - qor report results for DFT

JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_ack_nak_processing/ackd_flit_seq_num_reg[1]/CK (SDFFRHQX4M)	0.00 #	0.00 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_ack_nak_processing/ackd_flit_seq_num_reg[1]/Q (SDFFRHQX4M)	0.31	0.31 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_ack_nak_processing/o_ackd_flit_seq_num[1] (UC_MB_retry_ack_nak_processing)	0.00	0.31 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/i_ackd_flit_seq_num[1] (UC_MB_retry_transmitter_rules_v2)	0.00	0.31 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/B[1] (UC_MB_retry_transmitter_rules_v2_DW01_sub_0)	0.00	0.31 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/U3/Y (CLKINX2M)	0.10	0.41 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/U2_1/CO (ADDFHX1M)	0.34	0.75 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/U2_2/CO (ADDFX2M)	0.33	1.08 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/U2_3/CO (ADDFHX2M)	0.24	1.32 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/U2_4/CO (ADDFX2M)	0.31	1.63 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/U2_5/CO (ADDFHX1M)	0.30	1.92 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/U1/Y (XOR2X2M)	0.15	2.08 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/r379/DIFF[6] (UC_MB_retry_transmitter_rules_v2_DW01_sub_0)	0.00	2.08 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/U474/Y (AND3X2M)	0.20	2.28 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/U472/Y (NAND2X2M)	0.06	2.34 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/U89/Y (AND3X1M)	0.25	2.59 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/U88/Y (OAI22X2M)	0.12	2.71 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/U90/Y (NAND3X1M)	0.14	2.85 f
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/U217/Y (CLKNAND2X2M)	0.08	2.93 r
JC_MB_Mainband_inst/U3_UC_MB_retry_top/u_transmitter_rules/U221/Y (OAI211X2M)	0.08	3.02 f

Figure 94 - Critical Path Post DFT

7.3.3 Summary of Design Report

Parameter	Value
Maximum Level of Logic (LOL)	14
Clock Frequency	300 MHz
Combinational Cell Count	41,343
Sequential Cell Count	19,019
Total Area	217,677 gates
DFT Coverage	99.9 %

Figure 95 - synthesis summary

7.4 Formal Verification

Formal verification is a method used to check that the Gate-Level Netlist (the output of the synthesis process) works exactly the same as the original RTL design (SystemVerilog files). The RTL is the reference, and the netlist is the actual implementation that will be fabricated.

We used the Formality tool to compare both versions and make sure no mistakes or changes were introduced during synthesis. This step is very important to confirm that the final hardware will behave exactly as designed.

The formal verification was successfully completed, with no failing points, aborted points, or unverified points reported. This confirms that the gate-level representation of the UCle Adapter Layer precisely matches its RTL specification.

```
Report          : failing_points

Reference       : Ref:/WORK/UC_TOP_EP
Implementation  : Imp:/WORK/UC_TOP_EP
Version        : L-2016.03-SP1
Date           : Mon Jun 29 11:45:05 2026
*****

No failing compare points.

1
```

Figure 96 - formal results

Chapter 8: Conclusions and Future Work

8.1 Conclusion

This project presented the design and RTL implementation of a UCle 3.0 Die-to-Die Adapter Layer that operates as the central coordination layer between the Protocol Layer and the Physical Layer, consistent with the structure and objectives of the attached report. The implemented architecture covered the major functional domains required by the adapter, including the mainband data path, sideband communication, Adapter Link State Machine (ALSM), retry and replay support, and the register file needed for control and manageability. The developed design targeted the key responsibilities assigned to the UCle adapter in the specification: providing a low-latency data path, coordinating link bring-up and parameter exchange, and supporting reliability features such as CRC and Retry whenever required by the operating conditions. In the report, these responsibilities were translated into concrete RTL sub-blocks, verified through block-level and top-level testing, and integrated into a complete adapter-layer implementation. The results of this work show that the adapter layer can successfully abstract the complexity of chiplet communication from upper protocol logic by bridging FDI toward the protocol side and RDI toward the physical side, while maintaining organized control over state transitions, sideband messaging, and protected flit transfer. This confirms that a modular implementation strategy is effective for building a standards-aligned UCle adapter and provides a strong foundation for future chiplet-based systems that require interoperability, scalability, and reliability. It is important to note that the technical reference for this project is the UCle Specification Revision 3.0, Version 1.0, and the implemented architecture, interfaces, and control behavior were developed with that specification as the primary design baseline.

8.2 Future Work

Several extensions can further strengthen this project and move it closer to a production-ready UCle subsystem. A first direction is full-system integration with a complete UCle Physical Layer and a real protocol stack such as PCIe, CXL, or a streaming protocol, since the UCle specification defines the adapter as the bridge between those layers and expects protocol-aware interoperability across them. A second direction is broader verification coverage. Although the report includes testing at both sub-block and integrated levels, future work should expand toward constrained-random verification, assertion-based verification, protocol compliance testing, corner-case timeout validation, and interoperability-oriented test scenarios derived directly from the UCle sideband, link management, and retry rules. Another valuable extension is deeper implementation work in physical design and silicon-oriented closure. This includes synthesis optimization, timing closure across all modes, power estimation, clock/reset robustness analysis, DFT enhancement, and post-layout validation so that the adapter can be evaluated under more realistic ASIC deployment conditions. Future versions can also add support for more advanced UCle capabilities. Examples include richer manageability flows, stronger error logging and recovery policies, broader register access support, retimer-aware operation, sideband-only management configurations, and adaptation for multi-module or off-package connectivity scenarios that are discussed in the UCle specification. Finally, this

work can be extended toward higher performance and broader architectural exploration. Potential directions include optimizing latency through the mainband pipeline, improving buffering and flow-control efficiency, studying scalability across multiple modules, and evaluating how the adapter can be adapted for future chiplet systems using advanced packaging, UCIe-3D integration, or long-reach retimer-based topologies, all while continuing to use the UCIe specification as the main technical reference.